

Antonia Strobl

Das quadratische Rucksackproblem

Anwendung und Performance metaheuristischer Lösungsansätze

Masterarbeit

zur Erlangung des akademischen Grades
eines Master of Science
der Studienrichtung Betriebswirtschaft
an der Karl-Franzens-Universität Graz

Betreuer: Ao.Univ.-Prof. Mag. Dr.rer.soc.oec. Peter Greistorfer

Institut: Institut für Operations und Information Systems

Graz, April 2021

Danksagung

Allem voran möchte ich Ao.Univ.-Prof. Mag. Dr.rer.soc.oec. Peter Greistorfer und Dipl.-Ing. Dr. techn. Rostislav Staněk für ihren inhaltlichen Input, ihre Expertise und ihre Geduld bei der Betreuung meiner Masterarbeit danken.

Im persönlichen Umfeld möchte ich meiner Familie danken, welche mich nicht nur beim Verfassen dieser Masterarbeit unterstützt hat, sondern mir ein Studium der Betriebswirtschaftslehre überhaupt erst ermöglicht hat. Meine Eltern, die mich stets inhaltlich, moralisch und finanziell unterstützen. Ein besonderer Dank gilt meiner Mutter, eine Meisterin der Beistrichsetzung, welche sich aufopfernd Zeit genommen hat beim Korrekturlesen dieser Arbeit. Mein Vater, ein wandelndes Lexikon, der mit seinem Wissensdurst und seiner unermüdlichen Konsequenz eine geistige Inspiration aller anderen darstellt. Mein Bruder und mein Vorbild seit Tag 1, dem ich in so vielen Dingen ähnele. Nicht zuletzt, was die Zeitspanne beim Verfassen einer wissenschaftlichen Arbeit betrifft.

Ich möchte auch meinen Großeltern danken, da sie in meiner gänzlich subjektiven Wahrnehmung die besten Großeltern sind, die man sich wünschen kann und auch mein finanzielles Polster in der Zeit des Studiums immer aufgebessert haben.

Darüber hinaus möchte ich meiner erweiterten Familie danken. Eine Familie, mit der ich nicht dasselbe Blut, aber so viele wunderschöne Erinnerungen teile. Ohne diese Freunde - die meine Meilensteine feiern, als wären es ihre eigenen - wäre mein Leben und insbesondere die Studienzeit nicht dasselbe gewesen. Meine zwei Affen, die unterschiedlicher nicht sein könnten, aber gleichermaßen einzigartig und beeindruckend sind. Wir haben philosophiert, gelernt, gelacht, manchmal auch gelitten und sind schlussendlich gemeinsam erwachsen geworden. Ihr seid meine wahren Lehrer und ich möchte keinen einzigen Meilenstein in meinem Leben mehr ohne euch begehen.

Die Studienzeit wäre auch nicht dasselbe gewesen ohne eine Freundin, mit der ich einen (noch viel zu) kleinen Teil der Welt bereisen durfte. Du hast mit deiner mutigen Art den Abenteurer in mir geweckt, meinen (kulinarischen) Horizont erweitert und ich freue mich unfassbar auch noch viele Chaos-Reisen. Ich durfte es auch erleben, wie eine Arbeitskollegin fernab von zu Hause eine Freundin wurde, die mich immer wieder durch ihre unendliche Gutmütigkeit beeindruckt. Ich möchte auch einer Person danken, die es durch ihre einzigartig authentische Art geschafft hat, aus einem unabsichtlichen Schlag ins Gesicht im Vorlesungssaal eine Freundschaft zu zaubern, die ich nicht mehr missen möchte. Und weil das Leben oftmals ernst genug ist, möchte ich zum Schluss meinem besten Freund danken, der jede noch so angespannte Stimmung durch einen unangebrachten Scherz entschärfen kann.

Inhaltsverzeichnis

Abbildungsverzeichnis	iii
Tabellenverzeichnis	iv
Algorithmenverzeichnis	v
1 Einleitung	1
2 Das Rucksackproblem	4
2.1 0–1 Rucksackproblem	4
2.2 0–1 Quadratisches Rucksackproblem	8
3 Literatur-Review	10
3.1 Optimale Lösungsverfahren	10
3.2 (Meta-)heuristische Lösungsverfahren	14
4 Greedy-Algorithmus	17
4.1 Theoretische Grundlagen	17
4.2 Paarweise Randomisierte Greedy-Heuristik	18
4.2.1 Pseudocode	21
5 Tabu Suche	24
5.1 Theoretische Grundlagen	24
5.2 Paarweise randomisierte Greedy-Heuristik mit Tabu Suche	27
5.2.1 Pseudocode	28
6 Genetischer Algorithmus	30
6.1 Theoretische Grundlagen	30
6.1.1 Selektion	33
6.1.2 Rekombination	33
6.1.3 Mutation	34
6.2 Paarweise randomisierte Greedy-Heuristik mit Genetischem Algorithmus	36
6.2.1 Pseudocode	38

6.2.2	Reparaturmechanismus	39
7	Simulated Annealing	41
7.1	Theoretische Grundlagen	41
7.1.1	Abkühlungsverfahren	44
7.2	Paarweise randomisierte Greedy-Heuristik mit Simulated Annealing	45
7.2.1	Pseudocode	46
8	Datenauswertung	48
8.1	Methodik	48
8.1.1	Parameteranpassung	50
8.2	Paarweise randomisierte Greedy-Heuristik	52
8.3	Paarweise randomisierte Greedy-Heuristik mit Tabu Suche	58
8.4	Paarweise randomisierte Greedy-Heuristik mit Genetischem Algorithmus	61
8.5	Paarweise randomisierte Greedy-Heuristik mit Simulated Annealing Verfahren	64
8.6	Performance-Vergleich (Meta-)Heuristiken	67
9	Fazit	69
	Literaturverzeichnis	71
	Abkürzungsverzeichnis	77
	Symbolverzeichnis	78

Abbildungsverzeichnis

5.1	Übersicht generischer Tabu Suche - Eigene Darstellung	26
6.1	Übersicht generischer Genetischer Algorithmus - Eigene Darstellung	32
6.2	Sinlge-Point-Crossover	34
6.3	Two-Point-Crossover	34
7.1	Übersicht generisches Simulated Annealing Verfahren - Eigene Darstellung	42
7.2	Grafische Darstellung der Exponentialfunktion $y = e^x$	43
8.1	Vergleich von Funktionen mit asymptotischen Verhalten und unterschiedlichen Konvergenzgeschwindigkeiten	53
8.2	Funktion des Parameters $a(iterC)$ bei $\bar{a} = 30$ und $b \in [2, 4]$	53

Tabellenverzeichnis

1.1	Laufzeiten exponentieller und polynomieller Algorithmen	2
6.1	Vergleich Terminologie von natürlichen Organismen und Evolutionären Algorithmen - Eigene Darstellung	31
8.1	Übersicht Parameter	51
8.2	PRG - Vortest der Parameter $\bar{a}$ und b bei $k = 50$	55
8.3	PRG - Parameteranpassung des Parameters $a(iterC)$ bei $k = 50$	56
8.4	PRG - Parameteranpassung des Parameters k	56
8.5	Single-PRG und Multi-PRG - Performance-Evaluierung	57
8.6	PRG+TS - Parameteranpassung der Parameter $iterTS$ und p_N bei $t = 10$	59
8.7	PRG+TS - Parameteranpassung des Parameters t	60
8.8	PRG+TS - Performance-Evaluierung	60
8.9	PRG+GA - Parameteranpassung des Parameters $iterGA$	61
8.10	PRG+GA - Parameteranpassung des Parameters pC	62
8.11	PRG+GA - Parameterwerte der Parameter p_m und p_{cr}	63
8.12	PRG+GA - Performance-Evaluierung	63
8.13	PRG+SA - Parameteranpassung der Parameter nb_{acc} und α bei $nb_{trans} = 20.000$. . .	65
8.14	PRG+SA - Parameteranpassung des Parameters nb_{trans}	66
8.15	PRG+SA - Performance-Evaluierung	66
8.16	Performance-Vergleich (Meta-)Heuristiken	67

Algorithmusverzeichnis

3.1	Generischer GRASP	15
4.1	Pseudocode Single-PRG ($a = 1$)	21
4.2	Pseudocode Multi PRG ($a > 1$)	22
5.1	Pseudocode PRG+TS	28
6.1	Pseudocode PRG+GA	38
6.2	Reparaturmechanismus PRG+GA	39
7.1	Pseudocode PRG+SA	46

1. Einleitung

Das 0–1 Rucksackproblem ist ein kombinatorisches Optimierungsproblem und bildet einen simplen Entscheidungsprozess in Form eines mathematischen Modells ab. Daher wurde, wenn auch unbewusst, ein jeder und eine jede bereits im Alltag indirekt mit dem Rucksackproblem konfrontiert. Ein bekanntes Beispiel ist das Einpacken eines Gepäckstücks vor der Unternehmung einer Reise, worauf zudem der anschauliche Name besagten Problems zurückzuführen ist. Angesichts der physischen Eigenschaften eines jeden Gepäckstücks unterliegt ein Rucksack einer Gewichtskapazität, wodurch es nicht möglich ist, diesen uneingeschränkt mit Objekten zu befüllen. Es gilt daher eine Auswahl zu treffen, welche hinsichtlich des nutzenmaximierenden Naturells eines Menschen nicht willkürlich passieren wird.

Um dieses Realproblem mit dem mathematischen Modell in Einklang zu bringen, wird daher allen zu Verfügung stehenden Objekten sowohl ein Gewicht als auch ein Nutzen oder im wirtschaftlichen Kontext ein Profit zugeordnet. Das 0–1 Rucksackproblems zielt darauf ab, den Gesamtnutzen aller eingepackten Objekte zu maximieren, ohne dabei die Gewichtskapazität des Rucksackes zu überschreiten. Beim 0–1 Rucksackproblem handelt es sich demnach um eine ganzzahlige lineare Optimierung, welche darauf abzielt, die Zielfunktion (ZF) zu maximieren.

Eine Vielzahl von Realproblemen kann in Form des Rucksackproblems mathematisch formuliert werden und durch Adaption und Erweiterungen angepasst werden. Eine mögliche Variation entsteht durch das Auftreten von paarweisen Abhängigkeiten der eingepackten Objekte, welche sich entweder positiv oder negativ auf die Zielfunktion des kombinatorischen Optimierungsproblems auswirken können. Diese Spezialform des 0–1 Rucksackproblems wird als 0–1 Quadratisches Rucksackproblem bezeichnet und ist das zentrale Thema dieser Masterarbeit. In seinem Aufbau unterscheidet sich das 0–1 Quadratische Rucksackproblem vom 0–1 Rucksackproblem durch eine quadratische Zielfunktion, unterliegt jedoch denselben Nebenbedingungen. Jene quadratische Zielfunktion ist auf eine multiplikative Verknüpfung zweier binärer Entscheidungsvariablen zurückzuführen und ist die Ursache dafür, dass eine exakte Lösungsfindung des 0–1 Quadratischen Rucksackproblems erheblich komplexer ist als beim 0–1 Rucksackproblem.

Für einen Laien mag die Lösung eines 0–1 Rucksackproblems bei einer geringen Anzahl von verfügbaren Objekten auf den ersten Blick wie ein sehr einfach zu lösendes Problem wirken. Durch Einsatz einer vollständigen Enumeration können theoretisch alle möglichen Kombinationen aus in den Rucksack eingepackten und nicht eingepackten Objekten untersucht werden und dadurch eine Profitmaximale Auswahl der Objekte getroffen werden. Betrachtet man z. B. eine Grundgesamtheit von 3

zur Verfügung stehenden Objekten, so erhält man 8 verschiedene Kombinationen aus eingepackten und nicht eingepackten Objekten ohne Rücksichtnahme auf die Gewichtsbeschränkung. Erhöht man dieses Gedankenspiel auf eine Grundgesamtheit von 4 Objekten, steht man bereits vor der Auswahl von 16 verschiedenen Kombinationen. Es lässt sich demnach erkennen, dass die Möglichkeiten in Abhängigkeit des Eingabeparameters n exponentiell in der Form von 2^n wachsen.

Die Komplexitätstheorie beschäftigt sich mit der Klassifizierung von Rechenproblemen in Abhängigkeit einer Inputdaten-Größe und bewertet die Laufzeit-Performance eines Algorithmus auf Basis der sogenannten $\mathcal{O}$ -Notation. In der Tabelle 1.1 (entnommen aus Meier u. a. [2015] S. 58) kann man erkennen, wie sich die Laufzeiten von polynomiellen und exponentiellen Algorithmen in Abhängigkeit der Eingabegröße n unterschiedlich verhalten. Die Werte beziehen sich auf einen Rechner mit einer Taktfrequenz von 10^{10} Operationen pro Sekunde. Es zeigt sich, dass Rechenprobleme mit einer polynomiellen Laufzeit in einer vertretbaren Zeit lösbar sind, wohingegen Algorithmen mit einer exponentiellen Laufzeit ab einer gewissen Eingabegröße n mit den derzeit existierenden Computern faktisch unlösbar sind.

$f(n)$	$n = 20$	$n = 30$	$n = 40$	$n = 50$	$n = 60$
n^3	$8 \cdot 10^{-7}$ s	$2.7 \cdot 10^{-6}$ s	$6.4 \cdot 10^{-6}$ s	$1.25 \cdot 10^{-5}$ s	$2.16 \cdot 10^{-5}$ s
3^n	0.35 s	5.7 h	38 a	$2 \cdot 10^6$ a	$1.3 \cdot 10^{11}$ a

Tabelle 1.1: Laufzeiten exponentieller und polynomieller Algorithmen

Auch das 0–1 Rucksackproblem weist ein exponentielles Wachstumsverhalten in Abhängigkeit der zur Verfügung stehenden Objekte n auf und zählt daher zur Klasse der sogenannten $\mathcal{NP}$ -schweren Rechenprobleme. Diese Klasse umfasst jene Rechenprobleme für die (bis dato) keine Lösungsalgorithmen mit einer polynomiellen Laufzeit entwickelt worden sind. Für solche Probleme ist es unwahrscheinlich, dass ein Polynomialzeit-Algorithmus existiert, da dieser nicht nur die $\mathcal{NP}$ -Schwere dieses Problems, sondern aller Probleme der $\mathcal{NP}$ -Klasse lösen würde [Kellerer u. a., 2004, S. 14]. Jedoch zählt das 0–1 Rucksackproblem zu den $\mathcal{NP}$ -schweren Problemen im leichten Sinn, da bereits diverse Algorithmen mit einer pseudo-polynomiellen Laufzeit erarbeitet wurden. Der Terminus pseudo-polynomiell bezeichnet jene Algorithmen, die bei einem binären Codierungsschema der ganzzahligen Eingabewerte in die Klasse der $\mathcal{NP}$ -schweren Rechenprobleme fallen, bei einer unären Codierung jedoch in pseudo-polynomieller Laufzeit gelöst werden können. Im Falle des Rucksackproblems kann eine Reduktion auf eine pseudo-polynomielle Laufzeit durch Einsatz einer dynamischen Programmierung [Bellman, 1957] erzielt werden, bei der das Problem in Sub-Probleme unterteilt wird, indem die zur Verfügung stehende Kapazität startend bei 0 sukzessiv um 1 erhöht wird und eine Profit-optimale Auswahl der Objekte für jede Teilkapazität getroffen wird. Dadurch kann das Rucksackproblem optimal in einer Laufzeit von $\mathcal{O}(nC)$ gelöst werden [Leung, 2004, S.9 ff.].

Im Vergleich zum 0–1 Rucksackproblem zählt das 0–1 Quadratische Rucksackproblem zu den $\mathcal{NP}$ -schweren Problemen im strengen Sinn, was durch die Reduktion des Clique Problems bewiesen wurde. Daher ist es unwahrscheinlich, dass ein Algorithmus mit einer pseudo-polynomiellen Laufzeit entwickelt werden kann [Caprara u. a., 1999, S. 2].

Dieser Exkurs in die theoretische Informatik zeigt, dass die Lösungsfindung des Optimalwerts für ein 0–1 Quadratisches Rucksackproblem vor allem bei großen Testinstanzen eine beachtliche Herausforderung darstellt. Daher beschäftigt sich diese Masterarbeit mit der Anpassung und Implementierung bekannter metaheuristischer Verfahren, angewandt auf das 0–1 Quadratische Rucksackproblem. Eine Metaheuristik wie auch eine Heuristik im Allgemeinen zielt darauf ab, Lösungen von hoher Qualität für ein Optimierungsproblem zu finden und führt demnach auch nicht gezwungenermaßen zu der Optimallösung eines Problems. Der Term Metaheuristik wurde erstmals von Glover [1986] definiert und leitet sich aus dem griechischen Wort *heuriskein* ab, welches eine Beschreibung für die Kunst ist, neue Problemlösungsstrategien zu entdecken. Das Präfix *meta* weist auf eine höher gestellte Methodologie hin, welche als Hilfsstrategie beim Design einer geeigneten Heuristik für ein spezifisches Optimierungsproblem dient [Talbi, 2009, S. 1]. Anders ausgedrückt bedeutet das, dass Metaheuristiken nicht nur als mögliche Lösungsmethode eines einzigen Optimierungsproblems herangezogen werden können, sondern eine grobe Struktur vorgeben, welche durch Adaption auf eine Vielzahl von Optimierungsproblemen angewendet werden kann.

Im Speziellen wird in dieser Arbeit die Anwendung einer Tabu Suche, eines Genetischen Algorithmus und eines Simulated Annealing Verfahrens für das 0 – 1 Quadratische Rucksackproblem vorgestellt, deren Performance empirisch getestet und, durch Einsatz einer Datenauswertung werden die erzielten Ergebnisse bewertet und verglichen. Zusätzlich dazu wird vorab ein Greedy-Verfahren präsentiert, welches darauf abzielt, hinreichend gute Startlösungen für die eben genannten metaheuristischen Programme bereitzustellen.

2. Das Rucksackproblem

Wie bereits beschrieben handelt es sich bei dem 0–1 Rucksackproblem um ein ganzzahliges lineares Maximierungsproblem, welches darauf abzielt, den Gesamtnutzen aus einem Set von gewählten Objekten zu maximieren, ohne dabei eine Gewichtsbeschränkung zu verletzen. Im folgenden Kapitel wird eine kurze Darstellung der verschiedenen Abwandlungen des Rucksackproblems geboten und deren mathematische Modelle vorgestellt. Das 0–1 Rucksackproblem wurde bereits im Jahre 1930 von Dantzig [1930] beschrieben und gilt als das simpelste Maximierungsproblem der kombinatorischen Optimierung. Hinsichtlich der einfachen mathematischen Darstellung eines Entscheidungsprozesses erfreut sich das Rucksackproblem eines weiten Spektrums von Anwendungsmöglichkeiten, welche durch Adaptierung einfach auf Realprobleme angewandt werden können. In der englischsprachigen Literatur wird für das 0–1 Rucksackproblem der Terminus *Knapsack Problem* verwendet und daher fortan in dieser Arbeit mit KP abgekürzt.

2.1 0–1 Rucksackproblem

Jedem Objekt i wird hierbei ein Profit p_i und ein Gewicht w_i zugeordnet und die Gewichtsbeschränkung des Rucksacks durch die Variable C abgebildet. Die Bezeichnung 0–1 KP ist auf die Binärvariable x_i zurückzuführen, welche entweder einen Wert von 1 bei Inklusion des Objektes i in den Rucksack oder einen Wert von 0 bei Exklusion annimmt (siehe Nebenbedingung (2.3)). Das Ziel besteht in der Maximierung des Profits und wird durch die Zielfunktion (2.1) abgebildet. Aus der Anzahl der verfügbaren Objekte n wird eine Auswahl getroffen unter der Bedingung (2.2), dass die Kapazität C nicht überschritten wird. Üblicherweise werden die Profite p_i als ganzzahlig und positiv angenommen.

$$\max z = \sum_{i=1}^n p_i x_i \quad (2.1)$$

u.d.B.

$$\sum_{i=1}^n w_i x_i \leq C \quad (2.2)$$

$$x_i \in \{0, 1\}, \quad i = 1, \dots, n \quad (2.3)$$

Eine mögliche Anwendung des 0–1 KPs im Bereich der Betriebswirtschaft ist die Betrachtung einer Kapitalinvestition mit einem Grundkapital von maximal C Euro und n verschiedenen Investitionsmöglichkeiten. Die Variable p_i wäre in diesem Fall der Profit der Investition i , und das Gewicht w_i steht für die individuellen Investitionskosten. Die Optimallösung besteht aus einer Kombination von Investitionsmöglichkeiten, welche unter der Bedingung, dass die Investitionskosten das Budget nicht überschreiten, zu dem maximal realisierbaren totalen Profit führt [Martello u. Toth, 1990, S. 1]. Weitere Realanwendungen des 0–1 KPs liegen im Bereich der Zuschnittprobleme [Gilmore u. Gomory, 1966], Kryptosysteme [Merkle u. Hellman, 1978] und Investitionsplanung [Lorie u. Savage, 1955].

Teilsummenproblem Eine sehr bekannte Variation des 0–1 KPs ist das Teilsummenproblem (engl. Subset Sum Problem), welches in seiner Struktur dem 0–1 KP gleicht, jedoch in diesem Fall der individuelle Nutzen eines Objektes i direkt proportional zu seinem Gewicht ist. Dieses Modell kann z. B. beim Transport von Frachtgut angewandt werden, wo der Profit einer Frachtfirma proportional zum transportierten Gewicht ist. Daher können in diesem Modell die Variablen p_i und w_i gleichgesetzt werden.

$$\max z = \sum_{i=1}^n w_i x_i \quad (2.4)$$

u.d.B.

$$\sum_{i=1}^n w_i x_i \leq C \quad (2.5)$$

$$x_i \in \{0, 1\}, \quad i = 1, \dots, n \quad (2.6)$$

Multidimensionales Rucksackproblem Das Multidimensionale KP (MKP) ist eine Erweiterung des 0–1 KP um m weitere Ressourcen mit den jeweiligen Kapazitäten C_j . Nimmt die Entscheidungsvariable x_i eines Objektes i einen Wert von 1 an, so wird von jeder Ressource j ein Anteil von w_{ij} verbraucht. Bei einem Wert von 1 der Variable m und somit nur einer verfügbaren Ressource reduziert sich das MKP wieder zum klassischen 0–1 KP. Eine mögliche Realanwendung des MKP liegt im Bereich der Investitionsplanung in der Form, dass eine Profit-maximale Auswahl aus n Projekten getätigt wird und dadurch für jedes gewählte Projekt w_{ij} Einheiten der Ressource verbraucht werden [Vasquez u. Hao, 2001]. Weitere Anwendungen liegen im Bereich von Frachtbeladungen [Shih, 1979] und Zuschnittproblemen.

$$\max z = \sum_{i=1}^n p_i x_i \quad (2.7)$$

u.d.B.

$$\sum_{i=1}^n w_{ij}x_i \leq C_j, \quad j = 1, \dots, m \quad (2.8)$$

$$x_i \in \{0, 1\}, \quad i = 1, \dots, n \quad (2.9)$$

Multiple-Rucksackproblem Beim Multiple-KP (M-KP) wird nicht nur die optimale Beladung eines einzelnen Rucksacks betrachtet, sondern die Beladung von einem Set von $M = 1, \dots, m$ zur Verfügung stehenden Rucksäcken. Die individuelle Gewichtsbeschränkung eines jeden Rucksacks m darf hierbei nicht überschritten werden (siehe Nebenbedingung (2.11)). Anwendung findet das M-KP z. B. im Bereich von Beladungsproblemen, wo n Objekte in m Boxen geladen werden müssen und dabei die Kapazitäten C_j der einzelnen Boxen j nicht überschritten werden dürfen [Eilon u. Christofides, 1971].

$$\max z = \sum_{j=1}^m \sum_{i=1}^n p_i x_{ij} \quad (2.10)$$

u.d.B.

$$\sum_{i=1}^n w_i x_{ij} \leq C_j, \quad j = 1, \dots, m \quad (2.11)$$

$$\sum_{j=1}^m x_{ij} \leq 1, \quad i = 1, \dots, n \quad (2.12)$$

$$x_{ij} \in \{0, 1\}, \quad j = 1, \dots, m, \quad i = 1, \dots, n \quad (2.13)$$

Multiple-Choice-Rucksackproblem Beim Multiple-Choice-KP (MCKP) liegen für jedes Objekt i N_i verschiedene Varianten dieses Objektes vor. Die Variable N_i ist demnach abhängig von einem spezifischen Objekt $i \in [n]$. Ziel ist es, den Rucksack dementsprechend so zu packen, dass durch Auswahl einer einzelnen Variante $j \in [N_i]$ eines jeden Objektes i der Gesamtprofit maximiert wird (siehe Nebenbedingung (2.16)). Die binäre Variable x_{ij} bestimmt, ob die j -te Variante des Objektes i in den Rucksack inkludiert wird [Kellerer u. a., 2004, S. 1ff].

Auch beim MCKP findet sich eine Realanwendung im Bereich der Investitionsplanung. Hierbei liegen n Projekt-Kategorien mit jeweils N_i verschiedene Projekte vor. Es gilt daher eine Profit-optimale Auswahl zu treffen, indem innerhalb der Projekt-Kategorien jeweils ein einzelnes Projekt ausgewählt

wird.

Es muss nicht zwingend aus jeder Projekt-Kategorie ein Projekt gewählt werden, jedoch darf pro Kategorie maximal ein Projekt ausgewählt werden, was durch die Nebenbedingung (2.16) sichergestellt wird. Wie bereits bei den anderen Realanwendungen des KPs im Bereich der Investitionsplanung entspricht die Variable p_{ij} dem Profit des j -ten Projekts der Kategorie i und analog dazu verhält sich die Variable w_{ij} . Das verfügbare Budget wird erneut durch die Variable C abgebildet [Nauss, 1978].

$$\max z = \sum_{i=1}^n \sum_{j \in N_i} p_{ij} x_{ij} \quad (2.14)$$

u.d.B.

$$\sum_{i=1}^n \sum_{j \in N_i} w_{ij} x_{ij} \leq C \quad (2.15)$$

$$\sum_{j \in N_i} x_{ij} = 1, \quad i = 1, \dots, n \quad (2.16)$$

$$x_{ij} \in \{0, 1\}, \quad i = 1, \dots, n, \quad j \in N_i \quad (2.17)$$

2.2 0–1 Quadratisches Rucksackproblem

Das 0 – 1 Quadratische Rucksackproblem, fortan als QKP bezeichnet, wurde erstmals von Gallo u. a. [1980] beschrieben und erweitert das 0 – 1 KP um eine zusätzliche Einnahmequelle, bei der nicht nur einem individuellen Objekt, sondern auch einem Objektpaar Profite zugeordnet werden. Daher kann sich eine Kombination zweier in den Rucksack inkludierten Objekte positiv, negativ oder wie im Fall des 0 – 1 KPs gar nicht auf die ZF auswirken. Die Erweiterung der ZF um die bereits genannte zusätzliche Profitquelle verändert das ganzzahlige lineare 0 – 1 KP zu einem ganzzahligen nicht-linearen Maximierungsproblem. Beim QKP wird demnach eine quadratische Zielfunktion (siehe Formel (2.18)) unter der Bedingung einer linearen Kapazitätsbeschränkung maximiert (siehe Nebenbedingung (2.19)). Im mathematischen Modell werden die paarweisen Profite als zweidimensionale Matrix der Länge $[n \times n]$ abgebildet. Analog zur Variable p_i werden auch die paarweisen Profite q_{ij} in der Regel als ganzzahlig und positiv angenommen. Unter der Bedingung, dass die paarweisen Profite q_{ij} für alle $i \neq j$ einen Wert von 0 annehmen, reduziert sich das QKP wieder zum KP. Die in der Datenauswertung verwendeten Testinstanzen weisen eine symmetrische Matrix für die q_{ij} -Werte auf, weswegen im folgenden mathematischen Modell und in den auf das QKP angewandten Algorithmen ausschließlich die obere Dreiecksmatrix berücksichtigt wird. Bei den Testinstanzen nehmen die q_{ij} -Werte für alle Objekte $i = j$ einen Wert von 0 an.

$$\max z = \sum_{i=1}^n p_i x_i + \sum_{i=1}^n \sum_{j=i+1}^n q_{ij} x_i x_j \quad (2.18)$$

u.d.B.

$$\sum_{i=1}^n w_i x_i \leq C \quad (2.19)$$

$$x_i \in \{0, 1\}, \quad i = 1, \dots, n \quad (2.20)$$

Im Kontext der Investitionsplanung kann eine Anwendung des QKPs auftreten, wenn wie bereits beim klassischen 0 – 1 KP aus einer Auswahl von n Investitionsmöglichkeiten eine Profit-maximale Entscheidung getroffen wird, und zusätzlich dazu bei einer Kombination zweier gewählter Investitionen Zusatzprofite bzw. Kostenersparnisse durch Synergieeffekte erzielt werden können. Johnson u. a. [1993] stellen eine Realanwendung des QKPs in der Disziplin der Computertechnik in Form eines Compiler-Design-Problems vor. Die häufigsten Realanwendungen des QKPs befinden sich jedoch im Bereich der Standortplanung, wie z. B. von Flughäfen, Bahnhöfen, Umschlagterminals und Satelliten-Stationen (Rhys [1970], Witzgall [1975]). Das Realproblem der Satelliten-Standortplanung zielt darauf ab, die globale Kommunikationsbandbreite aller ausgewählten Satelliten zu maximieren. Umgelegt auf das QKP stehen n mögliche Standorte der Satelliten mit jeweiligen Installationskosten von w_i zur Verfügung. Das durchschnittliche tägliche Kommunikationsvolumen q_{ij} zwischen den Sa-

telliten soll unter Einhaltung eines Budgets C maximiert werden [Gallo u. a., 1980].

Das mathematische Modell einer nicht-beschränkten Quadratischen 0–1 Programmierung (QP) zeichnet sich durch die gleiche ZF (2.18) wie jene des QKPs aus, besitzt jedoch nur die Binärbedingung (2.20) und keine Kapazitätsbeschränkung (2.19). Demnach ist jedes Set aus Lösungen eines QKPs eine Teilmenge der Lösungen des QPs und kann daher für die Bestimmung von oberen und unteren Schranken diverser Lösungsverfahren (z. B. Branch-and-Bound-Algorithmus) herangezogen werden [Pisinger, 2007, S. 624].

Darüber hinaus ist das QKP eine Verallgemeinerung des graphentheoretischen Clique-Problems unter der Annahme eines vollständigen ungerichteten Graphen mit n Knoten. Jeder Knoten hat ein Gewicht w_i und einen Profit p_i und die Kanten des Graphen stehen für die paarweisen Profite q_{ij} . Das Clique-Problem zielt darauf ab, einen Teilgraphen aus einer vorgegebenen Anzahl von adjazenten Knoten in einem gegebenen ungerichteten Graphen $G = (V, E)$ zu finden. Das Maximum-Clique-Problem ist eine Erweiterung des Clique-Problems und beruht auf der Suche nach einem induzierten, vollständigen Teilgraphen mit der maximalen Anzahl an Knoten in einem gegebenen Graphen $G = (V, E)$. Das Maximum-Clique-Problem kann unter Zuhilfenahme eines QKPs Algorithmus mittels binärer Suche gelöst werden [Caprara u. a., 1999, S. 2]. Jedoch ist das Maximum-Clique-Problem eines der ersten Probleme, welches als $\mathcal{NP}$ -schwer klassifiziert wurde und daher, sofern nicht bewiesen wird, dass " $\mathcal{P} = \mathcal{NP}$ " ist, ein exakter Lösungsalgorithmus eine Berechnungszeit benötigt, welche exponentiell mit der Anzahl der Knoten wächst [Du u. Pardalos, 1999, S. 12]. Da das Maximum-Clique-Problem zu den am schwersten zu lösenden Problemen unter den $\mathcal{NP}$ -schweren Problemen zählt, trifft dies demnach auch auf das QKP zu. Dadurch ist bewiesen, dass eine exakte Lösungsfindung für das QKP bedeutend komplexer ist als beim KP.

3. Literatur-Review

Die simple Struktur des zugrunde liegenden mathematischen Modells auf der einen Seite kombiniert mit der rechnerischen Komplexität des QKPs auf der anderen Seite, weckt die Neugierde vieler Forscher. Trotz der $\mathcal{NP}$ -Schwere des Problems und demnach der rechenintensiven Bestimmung einer Optimallösung wurde bereits eine Vielzahl von exakten Lösungsverfahren erforscht, welche überwiegend auf Branch-and-Bound-Verfahren beruhen und sich untereinander in der Erstellung von oberen und unteren Schranken unterscheiden. Für eine umfangreiche Analyse der exakten Lösungsmethoden des QKPs wird auf die Abhandlung von Pisinger [2007] verwiesen, in der sehr detailliert auf die verschiedenen Lösungsverfahren und im Speziellen die Konzeption von oberen und unteren Schranken eingegangen wird. Jene Schranken werden durch Einsatz von *Upper Planes* [Gallo u. a., 1980], Lagrange-Relaxation [Caprara u. a., 1999], Reformulierung [Caprara u. a., 1999], Linearisierung [Billionnet u. Calmels, 1996], Lagrangian-Decomposition [Billionnet u. Soutif, 2004] und Reduktionstrategien erstellt.

Ein Literatur-Review bezüglich der (meta-)heuristischen Lösungsansätze zeigt, dass diesem Thema bis dato nicht soviel Aufmerksamkeit geschenkt worden ist wie der Erarbeitung von exakten Lösungsverfahren. Die publizierten (meta-)heuristischen Lösungsverfahren des QKPs werden überwiegend für die Erstellung von Startlösungen der exakten Lösungsverfahren herangezogen.

Für die im folgenden Kapitel behandelten exakten sowie auch (meta-)heuristischen Verfahren werden – sofern vorhanden – die Güte und auch die durchschnittlichen Laufzeiten der größten getesteten Instanzengrößen angegeben. Es ist jedoch wichtig, darauf hinzuweisen, dass oftmals kein aussagekräftiger Vergleich der Laufzeiten gezogen werden kann, da diese stark abhängig sind von der Implementierung des Algorithmus und dem für die Testung verwendeten Computer.

3.1 Optimale Lösungsverfahren

Gallo u. a. [1980] definieren in ihrer Abhandlung *Quadratic Knapsack Problems* erstmals das QKP und präsentieren ein exaktes Lösungsverfahren, welches sich einer linearen Relaxierung bedient, mit deren Hilfe die quadratische Zielfunktion $f(x)$ durch sogenannte *Upper Planes* (übersetzt ins Deutsche: obere Ebene) ersetzt wird. Jene *Upper Planes* dienen als obere Schranken und bedienen sich einer linearen Funktion $g(x)$, welche $f(x)$ in allen zulässigen Punkten dominiert.

In der Abhandlung werden Datensätze bis zu einer Größe von $n = 40$ optimal gelöst und es zeigt sich, dass bei den Instanzen mit einer Dichte nahe bei 100 % und nahe bei 0 % die besten Laufzeiten erzielt

werden können. Diese liegen bei der größten Instanzengröße durchschnittlich bei 90 Sekunden. Die Dichte einer Instanz bestimmt den prozentualen Anteil der Profit-Matrix Werte, welche ungleich 0 sind.

Das QKP ist nur eines von mehreren 0 – 1 linearen Optimierungsproblemen, welche sich durch eine quadratische Zielfunktion auszeichnen. Generell erfolgt eine Linearisierung einer nicht-linearen Funktion im Kontext eines Optimierungsproblems durch das Hinzufügen zusätzlicher Variablen und Beschränkungen. Ein allgemeines Linearisierungsverfahren für quadratische Optimierungsprobleme wird von Glover u. Woolsey [1974] vorgestellt und beruht auf dem Hinzufügen von $\binom{n}{2}$ zusätzlichen Binärvariablen, welche stets ein Produkt zweier originaler Binärvariablen repräsentieren. Dadurch wird eine Linearitätsbedingung erreicht und das nunmehr ganzzahlige lineare Problem (ILP) kann durch Einsatz eines sogenannten ILP-Solvers gelöst werden. Eine weitere und verbesserte Linearisierungsmethode wird von Glover [1975] vorgestellt, bei dem eine Linearisierung durch eine Erweiterung des mathematischen Modells des QKPs um nur mehr n zusätzliche Binärvariablen und $4n$ weitere Beschränkungen erzielt werden kann. Diese verbesserte Linearisierungsmethode wurde von diversen Autoren erfolgreich zur Erstellung oberer Schranken exakter Lösungsverfahren des QKP herangezogen, wie z. B. von Billionnet u. Calmels [1996], Billionnet u. a. [1999], Caprara u. a. [1999] und [Pisinger u. a., 2007].

Grainger u. Letchford [2007] nehmen die verbesserte Linearisierungstechnik von Glover [1975] auf und testen in ihrer Abhandlung 4 verschiedene Methoden zur Erstellung von unteren und oberen Schranken. Bei der trivialsten dieser Methoden werden die unteren Schranken auf 0 gesetzt und die oberen Schranken durch $\sum_{i \neq j} q_{ij}x_j$ bestimmt. Eine weitere Methode bezieht sich auf die Abhandlung von Glover [1975], bei der die unteren Schranken auf 0 gesetzt werden und die oberen Schranken durch die Lösung von 0 – 1 KPs ermittelt werden. Die dritte Methode ist das von Adams u. Forrester [2005] publizierte Verfahren, bei welchem die unteren Schranken erneut auf 0 gesetzt werden und die oberen Schranken analog zu Glover [1975] durch die Lösung von mehreren 0 – 1 KPs ermittelt werden. Die vierte und letzte Methode wird von Grainger u. Letchford [2007] neu vorgestellt und verbessert die Linearisierungsmethode dahingehend, dass sowohl die unteren als auch die oberen Schranken durch die Lösung von 0 – 1 KPs erstellt werden. In der Abhandlung werden durch Einsatz eines Branch-and-Bound-Verfahrens Instanzen bis zu einer Größe von $n = 120$ optimal gelöst, und es werden durch die geringeren Abstände oberer und unterer Schranken bei der neu publizierten Methode weniger Branch-and-Bound-Knoten und dadurch auch eine geringere Laufzeit zur Auffindung der Optimallösung benötigt. Diese liegt bei einer Instanzengröße von $n = 120$ bei durchschnittlich 357 Sekunden.

Caprara u. a. [1999] stellen in ihrer Abhandlung *Exact Solution of the Quadratic Knapsack Problem* ein exaktes Lösungsverfahren in Form eines Branch-and-Bound-Algorithmus vor. Die oberen Schranken werden durch Einsatz einer Lagrange-Relaxierung ermittelt, welche durch mehrere (kontinuierliche) KP gelöst wird. Bei der Bestimmung von oberen Schranken bei einem Branch-and-Bound-Verfahren steht man üblicherweise vor einem Trade-Off zwischen der Güte der oberen Schranke und dem benötigten Rechenaufwand. Je enger die obere Schranke gesetzt wird, desto weniger Branch-and-Bound-Knoten werden benötigt und desto schneller kann das Problem optimal gelöst werden. Die Berechnung einer oberen Schranke nahe am Optimalwert ist jedoch sehr rechen- und zeitintensiv. Daher setzen Caprara u. a. [1999] bei der Bestimmung der oberen Schranken nicht wie bis dato üblich auf eine lineare oder semidefinite Programmierung, sondern auf eine schnellere (duale) Heuristik für eine Lagrange-Relaxierung. Im Vergleich zu den zum Zeitpunkt der Publikation anderen erforschten exakten Lösungsmethoden des QKPs ist das Verfahren von Caprara u. a. [1999] in der Lage, Instanzen bis zu einer Größe von $n = 400$ optimal zu lösen. Auch wenn die generierten oberen Schranken von der Optimallösung weiter weg liegen als bei den zum Zeitpunkt der Publikation bereits erforschten Branch-and-Bound-Verfahren, zeigt ihre Abhandlung, dass es sinnvoller ist, marginal schlechtere obere Schranken zu berechnen, wenn sich das positiv auf die Rechendauer auswirkt. Eine komplexere und effizientere Methode zur Berechnung von oberen Schranken benötigt mehr Rechenzeit und führt oftmals dazu, dass ein Lösungsverfahren bei großen Instanzen schnell an seine Grenzen stößt. Darüber hinaus ist der Algorithmus auch dazu in der Lage, das Maximum-Clique-Problem für Instanzen überschaubarer Größe zu berechnen.

Wie bereits im Kapitel 1 beschrieben, existiert für das 0–1 KP ein pseudo-polynomieller Algorithmus auf Basis einer dynamischen Programmierung, welcher in der Lage ist, das $\mathcal{NP}$ -schwere Problem in einer Laufzeit von $\mathcal{O}(nC)$ optimal zu lösen. Die Autoren Fomeni u. Letchford [2014] stellen eine Abwandlung des eben genannten Algorithmus vor und präsentieren dadurch eine hoch-effiziente Heuristik in Form einer dynamischen Programmierung, kombiniert mit einer simplen lokalen Suche. Durch Einsatz der abgeänderten dynamischen Programmierung des 0–1 KPs können für das QKP Ergebnisse erzielt werden, welche nur einen Bruchteil eines Prozents von der Optimallösung abweichen. Diese bereits sehr guten Startlösungen werden als untere Schranken herangezogen, und durch anschließende Anwendung einer lokalen Suche wird für alle getesteten Instanzen die Optimallösung erzielt. Der Algorithmus weist jedoch eine Laufzeit von $\mathcal{O}(n^2C)$ und einen Speicher von $\mathcal{O}(nC)$ auf und ist demnach für Datensätze mit einer sehr großen Anzahl von verfügbaren Objekten n ungeeignet. Für die Testung des Algorithmus werden Instanzen bis zu einer Größe von $n = 380$ verwendet, welche in einer Laufzeit von durchschnittlich unter 10 Sekunden berechnet werden können.

Aufgrund der $\mathcal{NP}$ -Schwere des QKPs stoßen exakte Lösungsverfahren, wie z. B. ein Branch-and-Bound-Verfahren, bei großen Testinstanzen sehr schnell an ihre Grenzen und sind nicht mehr dazu in der Lage, eine Optimallösung in einer vertretbaren Rechendauer zu bestimmen. Dieses Problems nehmen sich Pisinger u. a. [2007] an und stellen einen Optimallösungs-Algorithmus vor, welcher auf einer

sogenannten aggressiven Reduktion der Instanzengröße basiert. Für die Erstellung von oberen und unteren Schranken wird auf eine Abfolge verschiedener bereits publizierter Methoden zurückgegriffen. Zum einen wird eine verbesserte Version des von Caprara u. a. [1999] vorgestellten Verfahrens auf Basis von *Upper Planes* und Reformulierung herangezogen, und zum anderen wird auch die Lagrange-Decomposition von Billionnet u. Soutif [2004] verwendet. Die Reduzierung der Inputgröße erfolgt auf Basis einer impliziten Enumeration, durch welche jene Teile des Branch-and-Bound-Baumes reduziert werden, welche entweder zu unzulässigen Lösungen oder zu keiner Verbesserung führen. Um eine möglichst gute untere Schranke für das Problem zu generieren, werden auf ein Subproblem mit einer reduzierten Inputgröße diverse heuristische Verfahren angewandt, wie z. B. eine Kombination aus dem Greedy-Verfahren von Billionnet u. Calmels [1996] mit anschließender Verbesserung durch eine Tabu Suche. Die Methoden zur Erstellung von oberen und unteren Schranken werden mehrmals wiederholt, und anschließend wird auf die Datenstruktur mit einer reduzierten Inputgröße das exakte Branch-and-Bound-Verfahren von Caprara u. a. [1999] angewandt. Es zeigt sich, dass Instanzen mit einem Umfang von bis zu 1500 Objekten auf eine Größe von 100 Objekten reduziert werden können, welche im Anschluss ohne Probleme durch Einsatz eines Branch-and-Bound-Algorithmus berechnet werden können. Darüber hinaus erzielt der Algorithmus auch sehr gute Ergebnisse für Instanzen mit einer geringen Dichte der Profitmatrix, was oftmals bei der Modellierung von Graphenproblemen in Form eines QKPs auftreten kann. Die durchschnittliche Laufzeit bei der größten Instanzengröße des gesamten Verfahrens – aggressive Reduktion von n und Branch-and-Bound-Verfahren – benötigt bei einer Dichte von 100 % ungefähr 12700 Sekunden und demnach rund 3.5 Stunden.

3.2 (Meta-)heuristische Lösungsverfahren

Die von Gallo u. a. [1980] publizierte Methode zur Berechnung von einer *Upper Plane* eines 0–1 KPs ist auch stets eine zulässige Lösung für das QKP und kann daher als untere Schranke herangezogen werden. Die Autoren stellen eine Heuristik vor, in der die Lösung einer solchen unteren Schranke durch Einsatz einer lokalen Suche verbessert wird. Bei dieser Verbesserungsheuristik, welche als "*fill-up-and-exchange*" bezeichnet wird, werden iterativ Objekte der Sublösung hinzugefügt oder ein bereits inkludiertes Objekt mit einem Objekt außerhalb des Rucksacks getauscht.

Chaillou u. a. [1989] stellen in ihrer Abhandlung eine Heuristik auf Basis eines Greedy-Verfahrens vor, bei welchem im ersten Schritt alle verfügbaren Objekte der Sublösung hinzugefügt werden. Es werden im Anschluss iterativ jene Objekte aus dem Rucksack wieder entfernt, durch deren Ausschluss das kleinstmögliche Verhältnis aus Profitverlust und Gewicht realisiert wird. Dieser Prozess wird so lange wiederholt, bis die Einhaltung der Gewichtsbeschränkung wieder erfüllt ist.

Die von Billionnet u. Calmels [1996] publizierte Heuristik ist eine Kombination beider oben genannter Heuristiken und verfährt nach dem Prinzip, dass durch Einsatz der Methode von Chaillou u. a. [1989] Startlösungen erstellt werden, welche im Anschluss durch die lokale Suche "*fill-up-and-exchange*" von Gallo u. a. [1980] verbessert werden. Jene hybride Heuristik wird aufgrund ihrer guten Performance in der Literatur des QKPs oftmals herangezogen, um Startlösungen für diverse exakte Lösungsverfahren zu generieren.

Glover u. a. [2002] stellen eine Tabu Suche vor, welche darauf basiert, dass das QKP in einem ersten Schritt als unbeschränkte binäre quadratische Programmierung (QP) reformuliert wird. Diese Reformulierung dient als Startlösung und wird dann in einem zweiten Schritt durch Einsatz einer Tabu Suche verbessert. Die Datenauswertung zeigt, dass für die getesteten Instanzen, welche eine Eingabegröße bis zu $n = 500$ aufweisen, sehr gute bzw. optimale Ergebnisse erzielt werden können. Bei einer Größe von $n = 500$ benötigt das Programm hierfür durchschnittlich 240 Sekunden. Jedoch liegen nur für Instanzen bis zu einer Größe von $n = 30$ die Optimallösungen vor. In der von Glover u. a. [2002] publizierten Abhandlung werden zum Zeitpunkt der Veröffentlichung erstmals für Instanzen größer als $n = 100$ gute Ergebnisse in einer vertretbaren Laufzeit generiert.

Julstrom [2005] präsentiert in seiner Abhandlung mit dem Titel *Greedy, Genetic, and Greedy Genetic Algorithms for the QKP* (meta-)heuristische Lösungsansätze für das QKP in Form von zwei Greedy-Verfahren, einem genetischen Algorithmus und zusätzlich einer Kombination aus beiden Metaheuristiken. Die Greedy-Verfahren werden im Kapitel 4 genauer beschrieben und ein Vergleich zu der in dieser Arbeit publizierten Greedy Heuristik gezogen. Bei den genetischen Algorithmen werden ausschließlich Chromosomen gebildet, welche die Kapazitätsbeschränkung nicht verletzen, wodurch weder der Einsatz von Reparatur- noch von Strafmechanismen nötig ist. Die Auswertung der QKP

Instanzen zeigt, dass der einfache genetische Algorithmus gute, jedoch auch stark variierende Ergebnisse liefert. Durchschnittlich weichen die gefundenen Zielfunktionswerte nicht mehr als 0.5 % vom Optimum ab. Es zeigt sich, dass durch Kombination des Greedy-Algorithmus mit dem Genetischen Algorithmus Ergebnisse erzielt werden können, welche öfters den Optimalwert erreichen als durch Einsatz des reinen Genetischen Algorithmus. Diese Methoden wurden jedoch nur an kleinen Testinstanzen mit einer Größe von maximal 200 Objekten getestet.

Die nach dem derzeitigen Forschungsstand als am effizientesten angesehene metaheuristische Lösungsanwendung für das QKP wird von Yang u. a. [2013] vorgestellt und trägt den Namen *Greedy Randomized Adaptive Search Procedure*. Es setzt sich, wie die Benennung bereits suggeriert, aus einem Greedy-Verfahren als Startlösungsheuristik und einer simplen Short-Term-Memory Tabu Suche als Verbesserungsheuristik zusammen. Die vorgestellte Metaheuristik wird mit der Bezeichnung GRASP+T abgekürzt und besteht aus drei Hauptschritten (siehe Algorithmus 3.1). In der Konstruktionsphase werden zulässige Startlösungen mittels einer adaptiven und iterierenden Methode erstellt, indem in jeder Iteration ein zufällig ausgewähltes Element aus einer eingeschränkten Kandidatenliste gewählt und der Teillösung hinzugefügt wird. Die Auswahl der Kandidatenliste basiert auf einer Greedy-Nutzenfunktion, welche den Zusatznutzen abbildet, der durch Inklusion eines Objektes mit Bezugnahme auf die bereits gefundene Teillösung generiert wird. In der zweiten Phase werden die gefundenen zulässigen Startlösungen durch Einsatz einer lokalen Suche optimiert, solange bis für die zu untersuchende Nachbarschaft ein lokales Optimum gefunden wird. Der oben beschriebene Prozess wird für eine festgelegte Anzahl von zu untersuchenden Startlösungen wiederholt und die ZF-Wert optimierendste Lösung aller Startlösungen gespeichert. Mit dem GRASP+T Verfahren können für Instanzen mit einem großen Eingabeparameter ($n = 2000$) Optimalwert-nahe Lösungen gefunden werden. Da jedoch für die getesteten Instanzen dieser Größe keine Optimallösungen vorliegen, wird als Referenzwert die von Caprara u. a. [1999] vorgestellte Methode zur Berechnung einer oberen Schranke $\hat{U}_{CPT}^2$, herangezogen, welche bei Instanzen der Größe $n = 200$ durchschnittlich 1.3 % vom Optimalwert abweicht und somit als die am Besten operierende Methode zur Berechnung von oberen Schranken für das QKP bekannt ist. Die prozentuale Abweichung der gefundenen Lösung durch Einsatz der GRASP+T Heuristik mit der bereits genannten oberen Schranke zeigt, dass die Ergebnisse durchschnittlich 1.4 % von den oberen Schranken abweichen und mit einer durchschnittlichen Rechenzeit von 288.39 CPU Sekunden Ergebnisse von hoher Qualität in einer kurzen Rechendauer auch für große Instanzen berechnet werden können.

Algorithm 3.1 Generischer GRASP

- 1: Einlesen der Instanz
 - 2: **while** Terminierungskriterium nicht erfüllt **do**
 - 3: Konstruktionsphase
 - 4: Lokale Suche
 - 5: Aktualisierung der besten Lösung
 - 6: **end while**
 - 7: Ausgabe der besten Lösung
-

Ein iterativer Semi-Greedy Algorithmus wird von Alva u. Benavides [2018] vorgestellt, welcher vergleichbare Ergebnisse wie der oben genannte GRASP+T liefert, jedoch einfacher zu implementieren ist und zusätzlich dazu schnellere Laufzeiten aufweist. Auch bei diesem Algorithmus werden in der Konstruktionsphase Startlösungen durch den Einsatz einer Greedy Konstruktionsheuristik erstellt. Als Greedy-Funktion wird die Prioritätsregel f_2 von Yang u. a. [2013] herangezogen, und im Anschluss an die Erstellung einer Startlösung wird eine lokale Suche ausgeübt. In der sogenannten Semi-Greedy Perturbation Phase werden willkürlich d Objekte aus der Teillösung entfernt und die freigewordene Kapazität iterativ und auf Zufallsbasis wieder mit Objekten aufgefüllt. Die Bezeichnung Semi-Greedy rührt daher, dass nicht gezwungenermaßen jenes Objekt mit dem besten Greedy-Funktionswert der Teillösung hinzugefügt wird, sondern ein Objekt zufällig aus der Menge aller Objekte gewählt wird, deren Zusatznutzen einen prozentualen Anteil des maximalen Zusatznutzens überschreitet. Dieser Prozess wird so lange wiederholt, bis die Kapazität aufgebraucht ist. Für die Testung des iterativen Semi-Greedy Algorithmus werden dieselben Instanzen sowie auch dieselben Testabläufe wie bei der Metaheuristik von Yang u. a. [2013] angewandt. Dies ermöglicht einen Vergleich beider Algorithmen und es zeigt sich, dass sehr ähnliche Ergebnisse generiert werden können, jedoch der iterative Semi-Greedy Algorithmus mit einer Laufzeit von durchschnittlich 94 Sekunden eine deutlich bessere Laufzeit-Performance aufweist.

4. Greedy-Algorithmus

4.1 Theoretische Grundlagen

Ein Greedy-Algorithmus zeichnet sich dadurch aus, dass in einer iterativen Abfolge von Schritten auf Basis einer sogenannten Greedy-Funktion eine Entscheidung aus mehreren Möglichkeiten getroffen wird. Es gibt keine allgemeine Greedy-Funktion, da diese für ein spezifisches Problem konzipiert werden muss, jedoch beruht es auf der allgemeinen Logik, dass in jedem Schritt die vorteilhafteste Handlungsmöglichkeit getätigt wird. Die treffende Bezeichnung dieses Algorithmus ergibt sich aus der Bedeutung von *greedy*, welches dem deutschen Wort gierig entspricht und suggeriert, dass stets die vorteilhafteste Entscheidung getroffen wird. Diese heuristische Lösungsmethode weist üblicherweise eine schnelle Rechenzeit auf, führt jedoch nicht gezwungenermaßen zur Optimallösung eines Optimierungsproblems. Darüber hinaus kann das zwingende Hinzufügen der nächstbesten Lösung darin resultieren, dass sich der Algorithmus nur auf ein Teilgebiet des Suchraums beschränkt und sich demnach die Lösungssuche in einem lokalen Optimum verfängt.

Bekannte Beispiele für die Anwendung eines Greedy-Algorithmus im Bereich der Graphentheorie sind z. B. das Verfahren von Kruskal zur Bestimmung eines minimalen Spannbaums und Dijkstra's Algorithmus des kürzesten Pfades bei einem gegebenen Startknoten. Beide Algorithmen liefern stets eine Optimallösung. Aus dem Bereich der Kombinatorik ist ein bekanntes Beispiel Chvátal's Greedy-Heuristik für das Mengenüberdeckungsproblem [Cormen u. a., 2001, S. 370].

In der Forschung werden für diverse metaheuristische Lösungsansätze des QKPs Greedy Algorithmen in einer ersten Konstruktionsphase eingesetzt, welche darauf abzielen, hinreichend gute Startlösungen zu erstellen. Zwei simple Greedy-Funktionen werden von Julstrom [2005] vorgestellt und als absoluter und relativer Greedy-Algorithmus betitelt. Beim absoluten Greedy-Algorithmus wird für alle Objekte das Verhältnis d_i aus dem Gesamtprofit (individuelle Profite p_i und paarweise Profite q_{ij}) und dem Gewicht w_i bestimmt. Der Lösungsvektor der Entscheidungsvariable entsteht dadurch, dass die Verhältnisse $d_i \forall i \in [n]$ absteigend sortiert und jene Objekte mit den besten Verhältnissen aus Profit und Gewicht iterativ der Sublösung S hinzugefügt werden, so lange bis die Gewichtskapazität aufgebraucht ist.

$$d_i = \frac{p_i + \sum_{l \neq i} q_{il}}{w_i}, \quad i = 1, \dots, n \quad (4.1)$$

Beim relativen Greedy-Algorithmus hingegen erfolgt die iterative Neuberechnung der Greedy-Funktion jedes Mal bevor ein Objekt der Sublösung S hinzugefügt wird. Hierbei berechnet sich der Gesamtprofit erneut aus dem individuellen Profit p_i eines Objektes, jedoch werden nur noch die paarweisen Profite q_{ij} dem Gesamtnutzen hinzugerechnet, welche durch die Interdependenzen mit den bereits inkludierten Objekten l der Sublösung S auftreten. Die Greedy-Funktion r_i des relativen Greedy-Algorithmus ist aussagekräftiger als jene beim absoluten Greedy-Algorithmus, da sich die Greedy-Funktion iterativ an die sich verändernde Sublösung anpasst und nur tatsächlich realisierbare Profite berücksichtigt werden.

$$r_i = \frac{p_i + \sum_{l \in S} q_{il}}{w_i}, \quad i = 1, \dots, n \quad (4.2)$$

4.2 Paarweise Randomisierte Greedy-Heuristik

Die im Zuge des Verfassens dieser Masterarbeit entwickelten Metaheuristiken bedienen sich als Startlösungsheuristik eines Greedy-Algorithmus. Im folgenden Kapitel wird dieser Greedy-Heuristik - bezeichnet als Paarweiser Randomisierter Greedy-Algorithmus (PRG) - vorgestellt. Die zugrunde liegende Greedy-Funktion des PRG ähnelt in seiner Logik dem relativen Greedy-Algorithmus von Julstrom [2005], unterscheidet sich jedoch maßgeblich darin, dass in einer Iteration nicht nur ein einziges zu inkludierendes Objekt, sondern stets ein zu inkludierendes Objektpaar betrachtet wird. Aufgrund dessen wird die vorgestellte Heuristik auch als *Paarweiser* Randomisierter Greedy bezeichnet, da im Gegensatz zum relativen Greedy von Julstrom der realisierbare Profit bei Inklusion einer Kombination zweier Objekte anstatt eines einzelnen Objektes berechnet wird.

Die Menge S sei wie zuvor eine Teillösung der bereits eingepackten Objekte ($x_l = 1 \quad \forall \quad l \in S$). Das Verhältnis aus zu gewinnenden Profiten und benötigtem Gewicht wird für jede mögliche Kombination aus gewählten und nicht gewählten Objekten berechnet und die Ergebnisse werden in der Greedy-Matrix $g_{ij} \quad \forall \quad i \in [n], \quad j \in [n] \quad (i \neq j)$ gespeichert. Hierbei können 4 verschiedene Kombinationen aus eingepackten und nicht eingepackten Objekten entstehen, wobei jedoch sinnigerweise nur 3 Kombinationen betrachtet werden. In den Formeln (4.3) bis (4.6) werden die Berechnungen der möglichen Objektpaar-Kombinationen abgebildet. Beim ersten Fall befinden sich beide Objekte des betrachteten Objektpaares außerhalb des Rucksacks. Für das Verhältnis aus Profiten und Gewichten werden die individuellen Profite beider Objekte, der paarweise Profit des Objektpaares und die Zusatzprofite, welche durch Interdependenzen mit den inkludierten Objekten l der Sublösung auftreten, summiert. In den Fällen 2a und 2b wird der Umstand betrachtet, dass bereits eines der beiden Objekte des zu betrachtenden Objektpaares in der Sublösung inkludiert ist. Dadurch reduziert sich die Formel aus dem ersten Fall (4.3) und ist ident mit dem relativen Greedy von Julstrom [2005]. Der paarweise Profit des Objektpaares q_{ij} muss in diesen beiden Fällen nicht extra hinzugerechnet werden, da er bereits im zweiten Summanden des Dividenden der Formeln (4.5) und (4.4) inkludiert ist. Der dritte Fall

(siehe Formel (4.6)) bildet die Situation ab, in der beide Objekte aus dem zu betrachtenden Objektpaar bereits in den Rucksack inkludiert sind und es daher nicht mehr möglich ist, einen Zusatznutzen zu generieren. Die g_{ij} -Matrix weist eine symmetrische Struktur auf ($g_{ij} = g_{ji}$), weshalb in den folgenden Berechnungen nur die obere Dreiecksmatrix berücksichtigt wird.

Fall 1: $x_i = 0$ und $x_j = 0$

$$g_{ij} = \frac{p_i + p_j + q_{ij} + \sum_{l \in S} (q_{il} + q_{jl})}{w_i + w_j} \quad (4.3)$$

Fall 2a: $x_i = 1$ und $x_j = 0$

$$g_{ij} = \frac{p_j + \sum_{l \in S} q_{jl}}{w_j} \quad (4.4)$$

Fall 2b: $x_i = 0$ und $x_j = 1$

$$g_{ij} = \frac{p_i + \sum_{l \in S} q_{il}}{w_i} \quad (4.5)$$

Fall 3: $x_i = 1$ und $x_j = 1$

$$g_{ij} = 0 \quad (4.6)$$

Durch multiplikative Verknüpfung der Binärvariablen x_i und x_j können die drei verschiedenen Fälle in einer einzelnen Formel (4.7) zusammengefasst und im Zuge dessen der Programmcode vereinfacht werden. Die Logik der Formel beruht darauf, dass jeder Term der Formel mit dem konträren Wert der dazugehörigen Binärvariable multipliziert wird. Tritt z. B. der erste Fall ein, dass beide Objekte noch nicht in der Sublösung S inkludiert sind, so sind auch beide Binärvariablen x_i und x_j auf 0 gesetzt, und demnach bleiben alle Terme der Formel bestehen. Anders verhält es sich bei den Fällen 2a und 2b, welche beschreiben, dass nur eines der beiden Objekte noch nicht in S inkludiert ist. Hier werden alle Terme, welche sich auf das bereits inkludierte Objekt beziehen, mit 0 multipliziert und werden demnach nicht mehr für das Verhältnis aus Profiten und Gewichten berücksichtigt.

Wie man anhand der Formel (4.7) erkennen kann, können im Vergleich zum relativen Algorithmus von Julstrom [2005] in einem iterativen Schritt mehrere auftretende Interdependenzen von zu inkludierenden Objekten mit den Objekten der Sublösung betrachtet werden.

$$g_{ij} = \frac{p_i(1 - x_i) + p_j(1 - x_j) + \sum_{l \in S} (q_{il}(1 - x_i) + q_{jl}(1 - x_j)) + q_{ij}(1 - x_i)(1 - x_j)}{w_i(1 - x_i) + w_j(1 - x_j)} \quad (4.7)$$

Die PRG-Heuristik verfährt nach dem Prinzip, dass die Greedy-Matrix g_{ij} in jeder Iteration zunächst berechnet und anschließend absteigend sortiert wird und das Objektpaar mit dem nächsthöchsten g_{ij} -Wert in den Rucksack inkludiert wird. Dieser iterative Prozess wird so lange wiederholt, bis die Gewichtskapazität C ausgelastet ist. Wenn der Fall aus den Formeln (4.4) oder (4.5) auftritt, welcher abbildet, dass nur eine der beiden Binärvariablen den Wert von 0 annimmt, so wird nur jenes Objekt der Lösung hinzugefügt, welches sich noch nicht im Rucksack befindet. Durch das zwingende Hinzufügen der nächstbesten Objektpaarkombination läuft man Gefahr, dass die Lösungssuche in einem lokalen Optimum stagniert. Daher wird das Auswahlverfahren randomisiert, indem nicht standardmäßig das Objektpaar mit dem besten g_{ij} -Wert ausgewählt wird, sondern aus den Objektpaaren mit den k -besten g_{ij} -Werten eine Zufallsauswahl getroffen wird. Nimmt demnach k einen Wert von 1 an, so wird wie im Falle eines klassischen Greedy-Algorithmus das Objektpaar mit dem besten Verhältnis aus Profiten und Gewichten in die Teillösung inkludiert. Die optimalen Werte für den Parameter k werden im Kapitel 8 im Zuge einer Parameteranpassung bestimmt.

Ein weiterer Parameter $a \forall s \in [a]$ bestimmt die Anzahl der zu generierenden unterschiedlichen Startlösungen und wird dahingehend eingesetzt, dass der PRG Algorithmus (4.1) so oft wiederholt wird, bis a unterschiedliche Startlösungen gefunden werden. Die binären Lösungsvektoren der Startlösungen werden in der Variable $\omega_{i,s}$ mit der Dimension $[n \times a]$ gespeichert.

4.2.1 Pseudocode

Der Single-PRG Algorithmus beschreibt den einfachen Fall, dass der Parameter a einen Wert von 1 annimmt und somit nur eine einzelne Startlösung erstellt wird. Die bereits beschriebene iterative Berechnung der g_{ij} -Werte aller möglichen Objektpaarkombinationen und anschließende Festlegung des nächstbesten Schrittes (Inklusion eines Objektpaares in den Rucksack) erfolgt durch zufällige Auswahl eines der k -profitabelsten Objektpaare. Dieser iterative Prozess wird so lange wiederholt, bis die Kapazität aufgebraucht ist. Ergebnis dieses Algorithmus ist der Lösungsvektor $x_i \forall i \in n$.

Algorithm 4.1 Pseudocode Single-PRG ($a = 1$)

```

1: while  $C \geq 0$  do
2:   for  $i = 1$  to  $n$  do
3:     for  $j = i + 1$  to  $n$  do
4:       if  $i \neq j$  then
5:         if  $C - w_i(1 - x_i) - w_j(1 - x_j) \geq 0$  then
6:           Berechne  $g_{ij}$  ▷ siehe Formel (4.7)
7:         else
8:            $g_{i,j} \leftarrow 0$ ; ▷ Einhaltung von NB (2.19)
9:         end if
10:      end if
11:    end for
12:  end for
13:  Sortiere Elemente der  $g_{i,j}$ -Matrix absteigend
14:  for  $f = 1$  to  $k$  do
15:     $g_{bestf} \leftarrow 0$ 
16:    Speichere die  $k$ -besten  $g_{i,j}$ -Werte in  $g_{bestf}$ 
17:    for  $i = 1$  to  $n$  do
18:      for  $j = 1$  to  $n$  do
19:         $ind_{bestf,1} \leftarrow i$  ▷ Speichere die Objektindizes der  $k$ -besten  $g_{i,j}$ -Werte
20:         $ind_{bestf,2} \leftarrow j$ 
21:      end for
22:    end for
23:  end for
24:   $R \leftarrow$  Gleichverteilte Zufallszahl im Intervall  $[1, k]$ 
25:   $i_{next} \leftarrow ind_{bestR,1}$ 
26:   $j_{next} \leftarrow ind_{bestR,2}$ 
27:   $C \leftarrow (C - w_i(1 - x_{i_{next}}) - w_j(1 - x_{j_{next}}))$ 
28:   $x_{i_{next}} = 1$  ▷ Füge die Objekte dem Rucksack hinzu
29:   $x_{j_{next}} = 1$ 
30: end while

```

In Zeile 5 des Pseudocodes 4.1 wird sichergestellt, dass nur jene Objektpaare der Teillösung hinzugefügt werden können, durch deren Inklusion die Kapazitätsbeschränkung nicht verletzt wird (siehe Nebenbedingung (2.19)). In Zeile 14 bis 23 werden in dem eindimensionalen Vektor g_{best} die k -besten g_{ij} -Werte gespeichert. Darüber hinaus werden die Indizes der zugehörigen Objektpaarkombinationen der k -besten g_{ij} Werte in die Matrix ind_{best} mit der Dimension $[k \times 2]$ aufgenommen. Im Anschluss

wird in Zeile 24 eine gleichverteilte Zufallszahl R aus dem Intervall $[1, k]$ ermittelt, welche die nächste Objektpaar-Inklusion bestimmt. Das gezogene Objektpaar wird dann in Zeile 27 bis 29 der Sublösung hinzugefügt und die verbleibende Kapazität angepasst. Dieser iterative Prozess wird so lange wiederholt, bis keine Restkapazität mehr übrig ist (siehe Zeile 1).

Der Multi-PRG-Algorithmus (siehe Pseudocode 4.2) zielt darauf ab, a voneinander unabhängige Startlösungen zu erstellen. Die Lösungsvektoren der ermittelten einzigartigen Startlösungen werden in die Variable $\omega_{i,s}$ ($\forall i \in [n], s \in [a]$) gespeichert. Hierfür wird der Single-PRG Algorithmus iterativ ausgeführt und dessen Ergebnisvektor x_i der $\omega_{i,s}$ -Matrix hinzugefügt, wenn sich der Lösungsvektor x_i einer Iteration von allen bereits gefundenen Startlösungen unterscheidet. Die Überprüfung auf Einzigartigkeit der gefundenen Lösung wird im Anschluss nach einem einmaligen Durchführen des Single-PRG ausgeübt. Das eben beschriebene Verfahren wird so oft wiederholt, bis a voneinander unabhängige Lösungen erstellt worden sind.

Algorithm 4.2 Pseudocode Multi PRG ($a > 1$)

```

1: while  $common_s \leq 10a$  do
2:   for  $iter_s = 1$  to  $a$  do
3:     Bestimme Lösungsvektor  $x_i \forall i \in [n]$  ▷ Durch Einsatz von Single-PRG Alg. 4.1
4:     for  $s = 1$  to  $a$  do
5:        $unique_s \leftarrow false$ 
6:       for  $i = 1$  to  $n$  do
7:         if  $x_i \neq \omega_{i,s}$  then ▷ Überprüfe auf Einzigartigkeit der gefundenen Lösung
8:            $unique_s \leftarrow true$ 
9:            $i = n$  ▷ Verlasse for-Schleife  $i$ 
10:        end if
11:      end for
12:    end for
13:     $all_{unique} = true$ 
14:    for  $s = 1$  to  $a$  do
15:      if  $unique_s = false$  then
16:         $all_{unique} \leftarrow false$ 
17:         $s = a$  ▷ Verlasse for-Schleife  $s$ 
18:      end if
19:    end for
20:    if  $all_{unique} \leftarrow true$  then ▷  $x_i$  ist einzigartig
21:      for  $i = 1$  to  $n$  do
22:         $\omega_{i,s} \leftarrow x_i$  ▷ Füge gefundene Lösung den Startlösungen hinzu
23:         $common_s = 0$ 
24:      end for
25:    else ▷  $x_i$  gleicht einer zuvor gefundenen Lösung
26:       $iter_s = iter_s - 1$  ▷ Iteration  $iter_s$  wiederholen
27:       $common_s = common_s + 1$ 
28:    end if
29:  end for
30: end while

```

In Zeile 27 zählt die Variable $common_s$ die Iterationen, in denen keine eindeutige Lösung generiert werden kann. Ohne Terminierungskriterium könnte es passieren, dass sich der Algorithmus in einer Endlosschleife verfängt. Daher wird der Multi-PRG Algorithmus in Abhängigkeit des Parameters a nach $10a$ aufeinanderfolgenden Iterationen, in denen keine einzigartige Lösung gefunden worden ist, abgebrochen. Für die Überprüfung auf Einzigartigkeit werden zwei neue Boolesche Variable eingeführt. Der eindimensionale Boolesche Vektor $unique_s \forall s \in [a]$ überprüft in Zeile 4 bis 12, ob sich der gefundene Lösungsvektor x_i der aktuellen Iteration von allen bereits gefundenen einzigartigen Startlösungen $\omega_{i,s}$ unterscheidet. In Zeile 5 wird die Boolesche Variable $unique_s$ vor jeder Überprüfung auf Einzigartigkeit eines Lösungsvektors s standardmäßig auf *falsch* gesetzt. In Zeile 8 wird jener Wert auf *wahr* umgeändert, sobald sich x_i und $\omega_{i,s}$ an der Stelle s in einem einzigen Objekt i unterscheiden. Im nächsten Schritt (Zeile 13 bis 19) wird überprüft, ob die Variable $unique_s$ für alle untersuchten Startlösungen s auf *wahr* gesetzt wurde. Die Variable all_{unique} wird in Zeile 13 standardmäßig auf *wahr* gesetzt. Nur wenn sich x_i von den Lösungsvektoren aller anderen Startlösungen unterscheidet, bleibt der Wert der zweiten Boolesche Variable all_{unique} unverändert auf *wahr*. Somit wird die gefundene Lösung als einzigartig deklariert und in der Zeile 22 der Lösungsvektor x_i der Matrix $\omega_{i,s}$ hinzugefügt. Unterscheidet sich der Lösungsvektor der aktuellen Iteration $iter_s$ jedoch nicht von den bereits gefundenen Startlösungen, so wird die Anzahl der aktuellen Iterationen in der Zeile 26 um 1 verringert und somit die Suche nach einer einzigartigen Startlösung an der Stelle s erneut gestartet.

5. Tabu Suche

5.1 Theoretische Grundlagen

Das Prinzip einer Tabu Suche, fortan als TS bezeichnet, wurde von Glover [1986] erstmals vorgestellt und seit damals laufend weiterentwickelt. In vielen Anwendungsgebieten liefert der Einsatz einer Tabu Suche effiziente Ergebnisse, welche sich oftmals nahe am Optimum befinden, und erfreut sich daher großer Beliebtheit als Lösungsansatz von großen kombinatorischen Optimierungsproblemen. Die Grundkomponenten einer TS Heuristik sind der aktuelle Suchbereich, die zu untersuchenden Nachbarschaftslösungen und eine Tabu Liste (TL), welche bereits vorgenommene Nachbarschaftsschritte speichert und für eine gewisse Anzahl von Perioden die Umkehrung dieser Schritte verhindert. Dies ist ein iterativer Suchvorgang, welcher bei einem Ausgangspunkt startet, sukzessiv lokale Veränderungen vornimmt und daher auch als lokale Suche (LS) bezeichnet wird. Die Grundidee eines TS Algorithmus kann beschrieben werden als der Einsatz von einer LS, welche vorübergehend ZF-Wert verschlechternde Schritte zulässt, um zu unterbinden, dass die Lösungssuche in einem lokalen Optimum stagniert. Anders ausgedrückt wird durch den Einsatz der LS ein lokales Optimum verlassen und durch zusätzliche Verwendung einer TL wird verhindert, dass der Algorithmus wieder zu diesem lokalen Optimum zurückfindet. Der Parameter t bestimmt die Anzahl der Perioden bzw. Iterationen, die ein Objekt auf der TL verweilt.

In der Literatur werden verschiedenste Mechanismen erforscht, welche darauf abzielen, die LS einer Tabu Suche in unerforschte Suchräume zu dirigieren sowie die oftmals rechenintensive Struktur einer Nachbarschaftssuche performanter zu gestalten. Gendreau u. Potvin [2010] stellen in dem Kapitel *Tabu Search* des Buches *Handbook of Metaheuristics* eine Schritt für Schritt Anleitung für die problemspezifische Anpassung einer generischen Tabu Suche vor und thematisieren folgende Erweiterungen, welche darauf abzielen, dass sich die Lösungssuche nicht in einem lokalen Optimum verfängt.

Aspirationskriterium Wie bereits erklärt, ist das Prinzip einer TL, dass Nachbarschaftsschritte dann unterbunden werden, wenn sie sich auf der TL befinden. Wenn jedoch dadurch attraktive Nachbarschaftsschritte nicht berücksichtigt werden, kann sich das negativ auf die Lösungsfindung auswirken. Hier kommt ein sogenanntes Aspirationskriterium zum Tragen, welches einen Tabu-Schritt dennoch zulässt, unter der Bedingung, dass jener Schritt zu dem zwischenzeitlich besten Ergebnis einer Lösungssuche führt. Der Grundgedanke eines Aspirationskriteriums ist jener, dass Tabu-Schritte immer

dann zugelassen werden können, wenn keine Gefahr besteht, dass sich die Suche in einem lokalen Optimum verfängt [Gendreau u. Potvin, 2010, S. 42].

Terminierungskriterium Eine weitere Möglichkeit zur Verbesserung einer TS ist die sinnvolle Auswahl des Terminierungskriteriums einer iterativen Lösungssuche. Üblicherweise gibt es bei einer TS drei verschiedene Arten von Terminierungskriterien [Gendreau u. Potvin, 2010, S. 44]:

- nach einer fest vorgegebenen Anzahl von Iterationen
- nach einer bereits im Vorhinein fixierten Anzahl von Iterationen, in denen keine Verbesserung des ZF-Wertes stattgefunden hat
- wenn ein im Vorhinein festgelegter Mindestwert des ZF-Wertes erreicht wurde

Reduzierte Nachbarschaft Die Auswahl einer gut konstituierten Nachbarschaftssuche ist ein grundlegendes Element einer effizienten Tabu Suche. Die Untersuchung einer großen Nachbarschaft hat den Vorteil, dass der Suchraum intensiv erforscht wird, jedoch wirkt sich das negativ auf die Rechendauer des Algorithmus aus. Um dem entgegenzuwirken, kann einerseits eine probabilistische TS und andererseits eine Kandidatenliste eingesetzt werden. Bei der probabilistischen TS wird die Nachbarschaft reduziert, indem anstatt einer vollständigen Erforschung der Nachbarschaft zufällig ausgewählte Nachbarschaftsschritte untersucht werden. Diese Methode wirkt sich nicht nur positiv auf die Rechendauer aus, sondern verringert auch das Risiko einer Stagnation in einem lokalen Optimum.

Auch bei einer Kandidatenliste wird mit einer verringerten Nachbarschaftsstruktur gearbeitet, wobei die Auswahl jedoch nicht auf zufälliger Basis, sondern auf strategischen Auswahlkriterien erfolgt [Gendreau u. Potvin, 2010, S. 44].

Intensivierung und Diversifizierung In der Literatur werden zwei Hauptstrategien vorgestellt, bei denen einerseits nahe liegende Lösungspunkte intensiv untersucht werden (Intensivierung) und andererseits eine Suche in bisher unerforschte Lösungsräume angestrebt wird (Diversifizierung) [Gendreau u. Potvin, 2010]. Eine Möglichkeit, diese zwei Strategien auszuloten, ist über die Steuerung des Parameters t . Ein kleiner Wert für t führt zu einer kürzeren Tabu Liste und dementsprechend zu einer größeren möglichen Nachbarschaft und intensiviert daher die Punkte, die nahe an der Optimallösung liegen. Eine lange Tabu Liste hingegen verkleinert die zu untersuchende Nachbarschaft und führt eher dazu, dass lokale Optima verlassen werden und unerforschte Suchräume erforscht werden [Taha, 2011, S. 395].

In der Abbildung 5.1 wird schematisch gezeigt, wie der Algorithmus einer generischen TS verfährt.

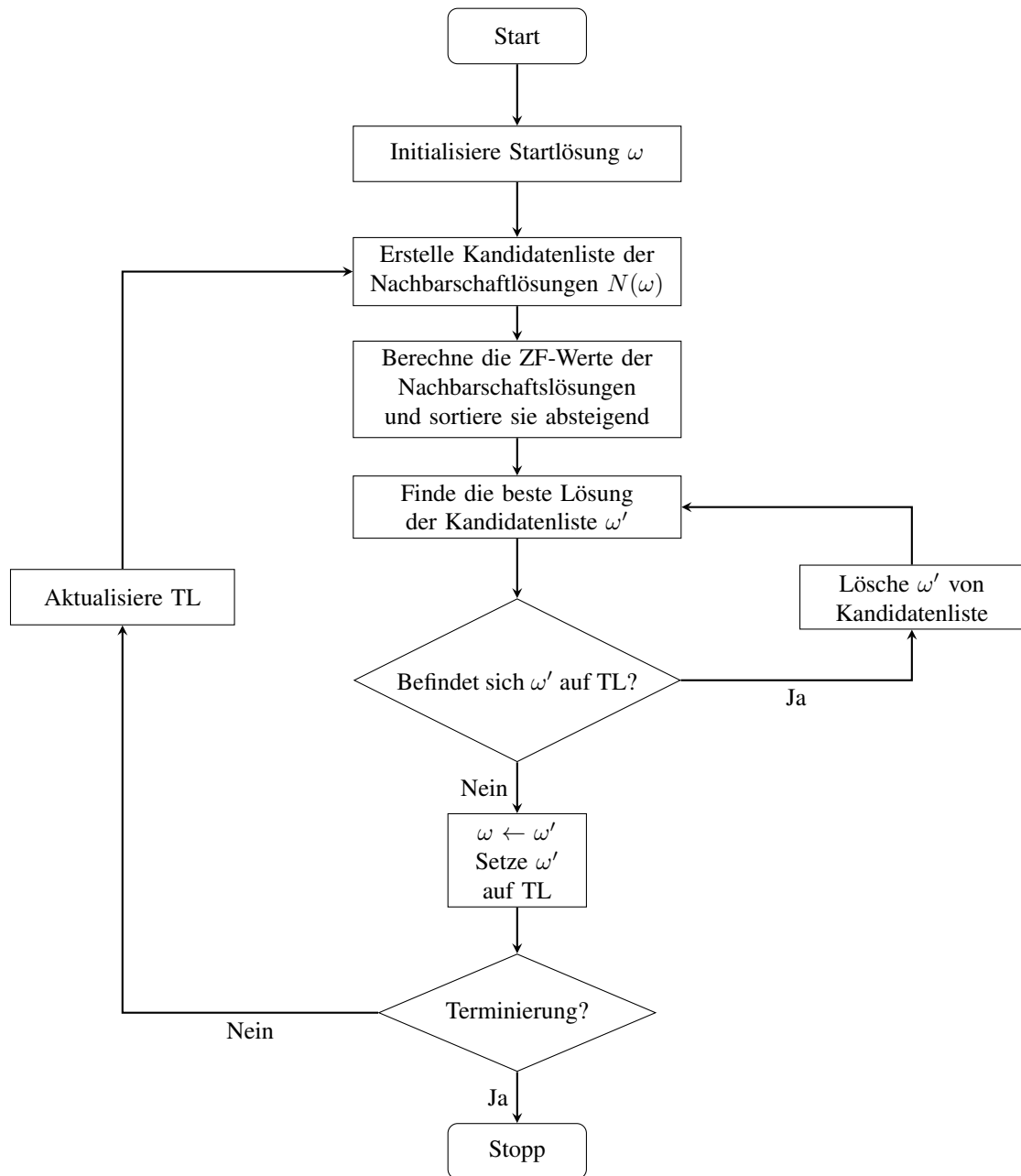


Abbildung 5.1: Übersicht generischer Tabu Suche - Eigene Darstellung

5.2 Paarweise randomisierte Greedy-Heuristik mit Tabu Suche

Die im folgenden Kapitel vorgestellte Metaheuristik verbessert die durch den PRG Algorithmus (siehe 4.1) generierten Startlösungen $\omega_{i,s}$ durch Einsatz einer TS und wird fortan als PRG+TS bezeichnet. Die Nachbarschaftssuche basiert auf der Überprüfung eines sukzessiven Tausches zweier ausgewählter Objekte und einer Bewertung der Auswirkung auf den ZF-Wert. Im Speziellen wird ein Objekt auf Zufallsbasis ausgewählt, welches bereits in der Sublösung inkludiert ist, und der vorteilhafteste Tausch von mehreren zufällig ausgewählten Objekten, welche sich nicht im Rucksack befinden, wird untersucht. Daher wird in diesem Algorithmus keine vollständige Nachbarschaftssuche vorgenommen, sondern eine probabilistische TS durchgeführt. Dadurch kann die Laufzeit optimiert werden und darüber hinaus durch die Zufallskomponente bisher unerforschte Suchräume erreicht werden. Da der Tausch zweier Objekte sowohl zu zulässigen als auch zu nicht zulässigen Lösungen führen kann, werden nur jene Nachbarschaftsschritte berücksichtigt, welche nicht die Nebenbedingung des QKPs verletzen (siehe Nebenbedingung (2.19)). Ein Tausch zweier Objekte erfolgt jedoch nur dann, wenn es in einer Verbesserung der zwischenzeitlich besten Lösung resultiert oder zumindest die zwischenzeitlich beste Lösung erreicht wird. Die TS wird für $iterTS$ Iterationen wiederholt, oder anders ausgedrückt wird das Entfernen und somit der vorteilhafteste Tausch für $iterTS$ inkludierten Objekten überprüft. Der Parameter p_N ist ein Prozentsatz und bestimmt, wie viele zurzeit exkludierten Objekte für die Überprüfung des vorteilhaftesten Tausches mit einem inkludierten Objekt berücksichtigt werden. Die Anzahl der zu untersuchenden Tausche wird aus der Multiplikation von dem Parameter p_N mit der Variable n . Wenn ein ZF-Wert verbessernder Tausch vorgenommen wird, so werden beide Objekte für die Dauer von t Perioden auf die TL gesetzt und daher ein Rücktausch dieser Objekte vorübergehend verhindert. Erst wenn ein Objekt nach t Perioden wieder von der TL genommen wird, kann es für einen möglichen Rücktausch berücksichtigt werden. Die Parameter t (Länge der TL), p_N (Nachbarschaftsgröße) und $iterTS$ (Terminierungskriterium) sind Eingabeparameter, deren optimale Werte im Zuge einer Parameteranpassung im Kapitel 8.3 bestimmt werden.

Die oben angeführte Kombination aus einer randomisierten Greedy-Heuristik und anschließender Verbesserungsheuristik in Form einer LS bzw. TS wird in der Literatur auch als GRASP (*Greedy Randomized Adaptive Search*) bezeichnet. In der Konstruktionsphase eines GRASP werden verschiedene Startlösungen erstellt, indem durch Einsatz einer eingegrenzten Kandidatenliste nicht zwangsläufig die nächstbeste Option der Teillösung hinzugefügt wird, sondern auf Zufallsbasis eine Auswahl der k -besten Optionen getätigt wird. Wie bereits in Kapitel 4 beschrieben verfährt auch der PRG-Algorithmus nach diesem Schema. In einem zweiten Schritt eines GRASP wird eine lokale Suche durchgeführt und die Startlösungen - wenn möglich - verbessert. Die Bildung der Nachbarschaft ist hierbei essenziell, und der Einsatz einer Kandidatenliste, welche darauf abzielt eine Nachbarschaft einzugrenzen, kann sich erheblich auf die Performance der Heuristik auswirken [Resende u. Ribeiro, 2010]. Es lässt sich demnach festhalten, dass der PRG+TS in seiner Grundstruktur analog zu einem GRASP verfährt.

5.2.1 Pseudocode

Für die Verbesserungsheuristik PRG+TS wird ein Set aus a Startlösungen mit dem Lösungsvektor $\omega_{i,s} \forall i \in [n], s \in [a]$ und den dazugehörigen ZF-Werten z_s benötigt. Für die Bestimmung des bestmöglichen Tausches zweier Objekte wird die Variable z' eingeführt, welche den durch einen spezifischen Tausch realisierbaren ZF-Wert speichert, und ergänzend eine zweite Variable z_{BS} , welche den maximalen ZF-Wert aller untersuchten Tausche einer Iteration bestimmt.

Algorithm 5.1 Pseudocode PRG+TS

Require: Set an Startlösungen $\omega_{i,s} \forall i = 1, \dots, n, s = 1, \dots, a$, ZF-Wert der Startlösung z_s ($s = 1, \dots, a$), Parameterwert für Parameter *iterTS* (Terminierungskriterium), p_N (Parameter zur Bestimmung des Umfangs der Nachbarschaftssuche) und t (Länge Tabu Liste)

```

1: for  $s = 1$  to  $a$  do
2:    $z_{BS} \leftarrow z_s$ 
3:   while  $iterTS \geq 0$  do
4:      $rand_{in} \leftarrow$  Zufallsauswahl eines inkludierten Objekts, welches nicht auf TL steht
5:      $improv \leftarrow false$ 
6:      $N_{size} = n p_N$ 
7:     while  $N_{size} \geq 0$  do
8:        $rand_{ex} \leftarrow$  Zufallsauswahl eines exkludierten Objekts, welches nicht auf TL steht
9:       if Falls Tausch von  $rand_{in}$  mit  $rand_{ex}$  zulässig ist then
10:         $N_{size} = N_{size} - 1$ 
11:        Tausche Objekte  $rand_{in}$  und  $rand_{ex}$ 
12:        Berechne neuen ZF-Wert  $z'$ 
13:        if  $z' \geq z_{BS}$  then
14:           $z_{BS} \leftarrow z'$ 
15:          Speichere den vorteilhaftesten Tausch
16:           $improv \leftarrow true$ 
17:        end if
18:        Rücktausch von Objekten  $rand_{in}$  und  $rand_{ex}$ 
19:      end if
20:    end while
21:    if  $improv = true$  then
22:      Unternimm den vorteilhaftesten Tausch
23:      Setze getauschte Objekte auf TL
24:    end if
25:    Entferne Objekte von TL wenn TL-Verweildauer =  $t$ 
26:    Erhöhe Anzahl Perioden für alle Objekte auf TL um 1
27:     $iterTS = iterTS - 1$ 
28:  end while
29: end for

```

In der Zeile 3 beginnt die iterative TS, welche nach einer Anzahl von *iterTS* Durchgängen terminiert wird. Auf Zufallsbasis wird ein Objekt $rand_{in}$, welches bereits Teil der Lösung ist, unter der Bedingung ausgewählt, dass sich das Objekt zum Zeitpunkt der Auswahl nicht auf der TL befindet (siehe Zeile 4). Für die Umsetzung des Algorithmus – jedoch nicht in diesem Pseudocode dargestellt – ist

ein Mechanismus notwendig, der den Fall abfängt, dass kein Objekt gefunden werden kann, welches die soeben genannten Bedingungen erfüllt. Die Variable N_{size} bestimmt den Umfang der Nachbarschaftssuche und wird ermittelt durch die Multiplikation der Anzahl an zur Verfügung stehenden Objekten n mit dem Parameter p_N . Dadurch wird sichergestellt, dass für größere Instanzen auch eine umfangreichere Nachbarschaftssuche vorgenommen wird. Im nächsten Schritt wird ein Objekt $rand_{ex}$ auf Zufallsbasis aus all jenen Objekten ausgewählt, welche nicht in den Rucksack inkludiert sind und sich nicht auf der TL befinden (siehe Zeile 8). Die Bewertung eines potenziellen Objekt-Tausches erfolgt in den Zeilen 13 bis 17 und durch Zuhilfenahme der Booleschen Variable *improv* wird überprüft, ob eine Verbesserung stattgefunden hat oder nicht. Da vorerst nur potenzielle Tausche überprüft werden, wird in der Zeile 18 der Lösungsvektor $\omega_{i,s}$ an den Stellen $rand_{in}$ und $rand_{ex}$ wieder auf die ursprünglichen Werte zurückgesetzt. Dieser Prozess wird für N_{size} zufällig ausgewählte exkludierte Objekte wiederholt und somit der vorteilhafteste Tausch für $rand_{in}$ bestimmt. Im Falle einer Verbesserung werden in den Zeilen 21 bis 24 die Objekte $rand_{in}$ und $rand_{ex}$ tatsächlich ausgetauscht und für eine Dauer von t Perioden auf die TL gesetzt. Eine TL-Periode kann in dieser Nachbarschaftsstruktur mit einem Durchgang der *iterTS*-Schleife gleichgesetzt werden. Die TL wird in den Zeilen 25 bis 26 aktualisiert.

6. Genetischer Algorithmus

6.1 Theoretische Grundlagen

Die im Zuge dieser Arbeit entwickelte Anwendung eines genetischen Algorithmus (GA) zählt zu der Klasse der evolutionären Algorithmen (EA), welche von der Funktionsweise natürlicher Lebewesen inspiriert sind und in einer Vielzahl von Disziplinen als metaheuristischer Lösungsansatz für Realprobleme angewandt werden. Seit den frühen 70er Jahren des vergangenen Jahrhunderts existiert die Grundlagenforschung Genetischer Algorithmen, die von Holland [1975] geprägt und anschließend von Goldberg [1989] erweitert wurde. Unter Einsatz genetischer Operatoren wird durch Rekombination zweier Elternteile $PA1_i$ und $PA2_i$ eine neue Generation von Individuen erstellt und dadurch der biologische Evolutionsprozess nachgeahmt. Ein GA verläuft sehr stark zufallsbasiert und steht daher auf den ersten Blick im Widerspruch zum zielgerichteten mathematischen Charakter von Optimierungsproblemen. Diese Diskrepanz hält jedoch bei genauerer Betrachtung nicht stand, da sich auch in der Natur diejenigen Individuen durchsetzen, die sich ganz nach Darwin's Evolutionstheorie [Darwin, 1859] *Survival of the fittest* möglichst gut an die Umwelt anpassen können. Jene *fitten* Individuen erzeugen durchschnittlich mehr Nachkommen, was auf eine bessere Lebenserwartung zurückzuführen ist und daher rührt, dass sich gutes Genmaterial langfristig durchsetzt und im Zuge dessen eine Population optimiert wird [Heistermann, 1994, S. 11f]. Bei der Implementierung einer jeden Metaheuristik muss der Algorithmus an ein spezifisches Problem angepasst werden, wobei jedoch üblicherweise die Grundstruktur eines jeden GA folgende Hauptkomponenten aufweist [Jansen, 2013, S. 7].

Fitnessfunktion Ein evolutionärer Algorithmus untersucht einen Teilbereich eines Suchraums S und bewertet die zu untersuchenden Objekte aufgrund der sogenannten Fitnessfunktion f . Dieser Terminus ist von den EA geprägt und stellt in vielen Fällen das Äquivalent zu der Zielfunktion f' des zugrunde liegenden Optimierungsproblems dar. Die Fitnessfunktion f und die Zielfunktion f' müssen jedoch nicht gezwungenermaßen ident sein.

Population Der zu untersuchende Teilbereich des Suchraums wird als Population P bezeichnet und besteht aus einer selbst gewählten Anzahl von Individuen. Im Kontext der Mengenlehre ist die Population P eine Teilmenge des Suchraums S ($P \subseteq S$) und kann darüber hinaus die Eigenschaft einer Multimenge¹ aufweisen. Da dem GA ein iterativer Prozess innewohnt und sich daher die Population von Generation zu Generation verändert, wird die Variable P_t mit einem Index t versehen, mithilfe dessen man auf die t -te Generation rückschließen kann. Der iterative Prozess des GA wird mit der Population P_0 initialisiert und es wird in jeder Generation für alle Individuen x die Fitnessfunktion $f(x)$ berechnet und gespeichert [Jansen, 2013, S. 7]. Aus der Population werden in jeder Iteration jeweils zwei Individuen $PA1_i$ und $PA2_i$ ausgewählt, welche als Elternteile für die Reproduktion der Nachkommen eingesetzt werden. Die Auswahl kann aufgrund der Fitnessfunktion oder auch auf Zufallsbasis erfolgen.

Chromosom/Gen Die evolutionären Algorithmen bedienen sich der Terminologie der Genetik, weswegen auch Chromosomen und Gene zwei wichtige Hauptbestandteile dieser Algorithmen sind. In einem biologischen Organismus speichert ein Chromosom die Erbgutinformationen in Form von Genen. Eines oder mehrere Chromosomen werden benötigt, um den gesamten Organismus zu spezifizieren, welcher als Phänotyp bezeichnet wird. Ein Set aus Chromosomen wird als Genotyp bezeichnet. Aufgabe eines Gens ist es, eine spezifische Eigenschaft eines Organismus zu codieren. Als Allel werden die verschiedenen Variationen bezeichnet, die ein Gen innerhalb einer Gattung von Individuen annehmen kann [Gen u. Cheng, 1997, S. 7]. Bei den evolutionären Algorithmen besteht eine Population aus mehreren Individuen bzw. im engeren Sinn aus mehreren Chromosomen. Jede zulässige Lösung kann ein Chromosom sein. Die Chromosomen wiederum bestehen aus Genen, welche nach einer an das Problem angepassten Gen-Codierung definiert sind [Taha, 2011, S. 405f.].

Natürliche Organismen	Evolutionäre Algorithmen
Gen	Position eines Strings
Allel	Ausprägung einer bestimmten Position
Chromosom	String
Fitness	Zielfunktionswert

Tabelle 6.1: Vergleich Terminologie von natürlichen Organismen und Evolutionären Algorithmen - Eigene Darstellung

¹Ein Begriff aus der Mengenlehre, der beschreibt, dass die Reihenfolge der Elemente irrelevant aber die Häufigkeit signifikant ist. Anders ausgedrückt bedeutet das, dass ein und dasselbe Element einer Menge mehrfach auftreten kann.

In der Abbildung 6.1 wird schematisch dargestellt, wie der Algorithmus eines generischen GA verfährt.

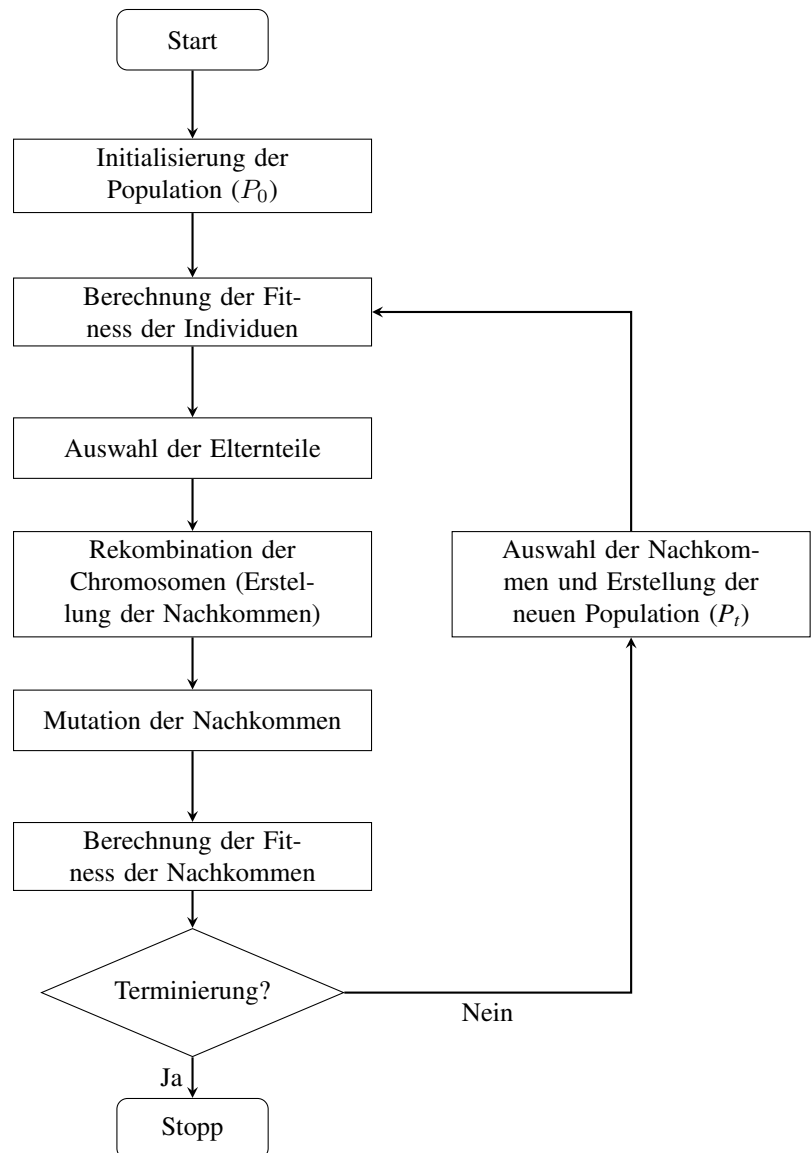


Abbildung 6.1: Übersicht generischer Genetischer Algorithmus - Eigene Darstellung

6.1.1 Selektion

Die Selektion von Individuen in einem GA erfolgt in der Regel an zwei Stellen des Algorithmus. Zum einen werden mithilfe einer geeigneten Methode zwei Individuen aus der aktuellen Population ausgewählt, welche für die Rekombination herangezogen werden. Des Weiteren werden aus der aktuellen Population Individuen gewählt, welche durch die Nachkommen ersetzt werden und somit eine neue Population bilden. Sowohl der Selektionsmechanismus der Elternteile als auch die Ersetzungsstrategie der derzeitigen Population wirken sich maßgeblich auf die Performance eines GA aus. In der Literatur wurden bereits diverse Selektionsmechanismen erforscht, und im folgenden Kapitel wird ein kleiner Einblick in die geläufigsten Methoden geboten.

Roulette-Wheel-Selektion Bei der Roulette Wheel-Selektion bekommt jedes Individuum einen Platz auf einem gedanklichen Roulette-Rad. Je besser die Fitness eines Individuums im Vergleich zur Fitness der anderen Individuen ist, desto größer ist der Bereich, welcher von dem Individuum auf dem Roulette-Rad eingenommen wird. Wird das gedankliche Roulette-Rad nun gedreht, so wird ein Individuum auf Zufallsbasis ausgewählt, wobei jedoch die Wahrscheinlichkeit einer Auswahl in Abhängigkeit zur Fitness-Funktion steht [Blickle u. Thiele, 1995].

Stochastic-Universal-Sampling-Methode Die Stochastic-Universal-Sampling-Methode ist eine Adaption der Roulette Wheel-Selektion, welche jedoch darauf abzielt, das Risiko von frühzeitiger Konvergenz zu vermindern. Bei der *Roulette Wheel* Selektion gibt es einen fixierten Punkt, mithilfe dessen ein Individuum zufällig gewählt wird. Demnach ist ersichtlich, dass für die Auswahl zweier Elternteile dieser Prozess auch zweimal wiederholt werden muss. Im Gegensatz dazu gibt es bei der Stochastic-Universal-Sampling-Methode zwei fixierte Punkte, wodurch beide Elternteile in einem Durchgang gewählt werden können [Goldberg u. Deb, 1991].

Tournament-Selektion Bei der Tournament-Selektion wird von der Grundgesamtheit der Population eine festgelegte Anzahl von Individuen zufällig ausgewählt. Unter diesen ausgewählten Individuen erfolgt die finale Selektion eines Elternteils auf Basis der Fitnessfunktion [Goldberg u. Deb, 1991].

6.1.2 Rekombination

Die Rekombination der Elternteile erfolgt auf Basis eines geeigneten Rekombinationsoperators, in der englischsprachigen Literatur als Crossover-Methode bekannt. Die einfachste Crossover-Methode ist die sogenannte Single-Point-Crossover-Methode, bei der durch Einsatz eines Zufallsgenerators eine Gen-Position ausgewählt wird, und das erste Kind $C1_i$ die Allele der Gene des ersten Elternteils $PA1_i$ bis zu diesem spezifischen Gen und die Allele des zweiten Elternteils $PA2_i$ ab diesem Gen übernimmt. Dasselbe gilt vice versa für das zweite Kind $C2_i$ (siehe Abbildung 6.2). Ein anderer bekannter Rekombinationsoperator ist das Two-Point bzw. Multi-Point-Crossover-Verfahren. Dieses erfolgt nach demselben Schema wie die Single-Point-Crossover-Methode, jedoch mit dem Unterschied, dass

hier zwei bzw. mehrere Austauschpunkte cp gewählt werden (siehe Abbildung 6.3) [Affenzeller u. a., 2009, S. 7f]. Sowohl die Single-Point-Crossover-Methode als auch die Two-Point-Crossover-Methode werden in dieser Arbeit auf den entwickelten Genetischen Algorithmus angewandt. Eine weitere sehr gebräuchliche Methode, welche jedoch nicht anhand des vorgestellten GA getestet wird, ist die sogenannte Uniform-Crossover-Methode. Bei dieser Methode werden alle Allele an beide Kinder vererbt, welche sowohl beim $PA1_i$ als auch beim $PA2_i$ ident sind. Die restlichen, noch nicht bestimmten Allele der Kinder werden durch eine Verteilungsfunktion ermittelt.

Ob ein zu erstellendes Individuum einer Rekombination unterliegt, wird über die sogenannte Crossover-Wahrscheinlichkeit p_{cr} gesteuert. Bei einer Crossover-Wahrscheinlichkeit von 0 bleibt ein aktueller Lösungsvektor unverändert und wird direkt an die nächste Population weiter gegeben [Laoufi u. a., 2006, S.394]. Die in der Literatur erprobten Werte für die Crossover-Rate sind meistens recht hoch und liegen üblicherweise in einem Bereich von p_{cr} (0.5-1) [Hesser u. Männer, 1991].

In den Abbildungen 6.2 und 6.3 werden die beiden angewandten Methoden noch einmal grafisch veranschaulicht. Die Crossover Punkte nehmen hierbei bei dem Single Point Crossover Verfahren einen Wert von $cp = 3$ und bei dem Two-Point Crossover-Verfahren die Werte $cp_1 = 3$ und $cp_2 = 5$ an.

PA₁	1	0	1	0	0	1
PA₂	0	0	0	1	1	1
C₁	1	0	0	1	1	1
C₂	0	0	1	0	0	1

Abbildung 6.2: Single-Point-Crossover

PA₁	1	0	1	0	0	1
PA₂	0	0	0	1	1	1
C₁	1	0	0	1	0	1
C₂	0	0	1	0	1	1

Abbildung 6.3: Two-Point-Crossover

6.1.3 Mutation

Mutation ist eine weitere grundlegende Komponente der EA, welche sich an das Verhalten natürlicher Organismen anlehnt. Dieser Prozess verfährt ähnlich wie sein Pendant aus der Biologie und sorgt dafür, dass sich der Wert des Allels eines Nachkommen mit einer gewissen Wahrscheinlichkeit p_m verändert. Eine Mutation eines Allels findet jedoch selten statt, da die Mutationsrate üblicherweise in einem Bereich von p_m (0.001-0.05) liegt [Hesser u. Männer, 1991]. Ziel einer Mutation ist es, noch unerforschtes Genmaterial zu untersuchen und Genmaterial, welches im Laufe der Suche verloren gegangen ist, wieder herzustellen. Ein weiterer positiver Effekt, welcher beim Einsatz von Mutation auftreten kann, ist das Verhindern von frühzeitiger Konvergenz der Lösungen. [Laoufi u. a., 2006, S.395]. Konvergenz tritt dann auf, wenn in einer Population die Individuen für die Gene dieselben Ausprägungen aufweisen. Wenn dieses Phänomen zu früh im Suchverlauf eines genetischen Algorithmus auftritt, dann spricht man von frühzeitiger Konvergenz, und die Lösungssuche konvergiert zu einem suboptimalen Ergebnis.

Bei einer binären Gen-Codierung führt Mutation zu einem Umschlagen in den konträren Wert (Bit-Flip-Mutation), wohingegen bei einer numerischen Gen-Codierung auf einer Zufallsbasis ein Wert innerhalb eines Intervalls ausgewählt wird. Eine zufällige Mutation eines Gens muss jedoch auf Durch-

föhrbarkeit geprüft werden, da dies zu einer nicht zulässigen Lösung föhren kann. Weiters können dadurch idente Lösungen entstehen, wodurch bei bestimmten Optimierungsproblemen, wie z. B. dem Routing-Problem, Komplikationen auftreten können [Affenzeller u. a., 2009, S. 9].

Die Festlegung der Parameter p_m und p_{cr} sind maßgeblich für die Performance eines Genetischen Algorithmus. In der Literatur wird eine Vielzahl von Methoden für die Festlegung sinnvoller Parameterwerte vorgestellt. Grefenstette [1986] z. B. betrachtet die Festlegung dieser Parameter als eigenes Optimierungsproblem und stellt einen Ansatz vor, in dem dieses Problem durch Einsatz eines zweiten Genetischen Algorithmus gelöst wird und dadurch die optimalen Werte für p_m und p_{cr} festgelegt werden. Dieser Ansatz ist jedoch sehr rechenintensiv und wirkt sich daher negativ auf die Laufzeit aus. Hohe Werte für p_m und p_{cr} erweitern den zu betrachtenden Suchraum, wohingegen niedrige Werte dazu föhren, dass attraktive Lösungen intensiv untersucht werden.

Im Rahmen dieser Arbeit werden die Crossover-Rate und die Mutations-Rate nach dem Erstellen einer neuen Population an das Maß der Konvergenz dieser Population angepasst (siehe Algorithmus 8.4).

6.2 Paarweise randomisierte Greedy-Heuristik mit Genetischem Algorithmus

Die PRG+GA Heuristik setzt sich zusammen aus einem Set von Startlösungen, generiert durch die bereits im Kapitel 4.2 vorgestellte Multi-PRG Heuristik und einer anschließenden Verbesserungsheuristik in Form eines Genetischen Algorithmus. Wie bereits erwähnt, bildet den Ausgangspunkt eines GA eine Population aus Individuen, aus der in jeder Iteration zwei Elternteile gewählt werden. Die anfängliche Population P_0 wird durch Einsatz der Multi-PRG-Heuristik erstellt (siehe Algorithmus 4.2), und dementsprechend bestimmt der bereits bekannte Parameter a die Größe der Population in allen Iterationen. Im Falle des PRG+GA erfolgt die Bewertung der Fitness auf Basis des Zielfunktionswertes (siehe Formel (2.18)), und als Chromosom wird der Lösungsvektor des Optimierungsproblems herangezogen, wodurch die Gene eine binäre Gen-Codierung aufweisen.

Der eingesetzte Selektionsoperator beruht auf einer Auswahl hinsichtlich der Fitnessfunktion, kombiniert mit einer zufälligen Auswahl. Der erste Elternteil $PA1_i$ wird stets auf Basis der besten Fitness aller Individuen der aktuellen Population gewählt. Die restlichen Individuen werden aufsteigend nach ihrer Fitness sortiert, und aus den ersten $0.5a$ Individuen mit schlechtester Fitness wird der zweite Elternteil $PA2_i$ auf Zufallsbasis auserkoren. Die Chromosomen der Elternteile $PA1_i$ und $PA2_i$ werden jeweils in einem Vektor der Länge n dargestellt. Jedes Element aus diesem Vektor zeigt an, ob ein Objekt im Rucksack enthalten ist oder nicht. Im Falle einer Inklusion eines Objektes nimmt das Allel eines Chromosoms den Wert 1 und bei Ausschluss des Objektes einen Wert von 0 an.

Als Crossover-Methode werden sowohl die bereits vorgestellte Single-Point-Crossover-Methode als auch die Two-Point-Crossover-Methode getestet. Anders als bei einem generischen GA werden beim PRG+GA nicht standardmäßig zwei Nachkommen pro Iteration erstellt, sondern der Parameter nb_C eingeführt, welcher die Anzahl der zu erzeugenden Nachkommen steuert. Gespeichert werden die Chromosomen der Nachkommen in der zweidimensionalen $C_{i,d}$ Matrix, welche die Dimension $[n \times nb_C]$ aufweist. Über die Crossover-Wahrscheinlichkeit p_{cr} wird gesteuert, wie viele der Nachkommen prozentual durch eine Crossover-Methode erstellt werden. Bei einer Crossover-Wahrscheinlichkeit von 100 % werden alle Nachkommen in dieser Iteration durch Rekombination generiert, wohingegen bei einer Crossover-Wahrscheinlichkeit von 0 % die Chromosomen aller Nachkommen von Individuen der aktuellen Generation übernommen werden.

Als Mutationsmethode der Chromosomen wird aufgrund der binären Gen-Codierung die Bit-Flip-Methode mit einer Mutationswahrscheinlichkeit von p_m eingesetzt. Es werden demnach alle Allele der Nachkommen mit einer Wahrscheinlichkeit von p_m in den gegenteiligen Wert umgeklappt.

Sowohl der Einsatz beider genannter Crossover-Methoden als auch das zufällige Mutieren der Allele führt zu zulässigen, aber auch nicht zulässigen Lösungen. Daher wird ein Reparaturmechanismus eingesetzt, welcher die Einhaltung der Gewichtskapazität der Nachkommen sicherstellt. Das Konzept und der Algorithmus des Reparaturmechanismus werden im Kapitel 6.2.2 erklärt.

Als Ersetzungsstrategie der Individuen der aktuellen Population durch die erstellten Nachkommen

werden die Individuen zuerst aufsteigend nach ihrer Fitness sortiert und anschließend die schlechtesten nb_C Individuen durch die rekombinierten Nachkommen ersetzt.

Maßgeblich für die Implementierung eines erfolgreichen GA ist ein Balanceakt zwischen der Diversität einer Population, welche darauf abzielt, nicht nur einen Teilbereich sondern den gesamten Suchraum zu untersuchen, und der Notwendigkeit, Zielfunktionswert-optimierende Individuen auszuwählen. Eine mangelnde Diversität ist eine der Hauptursachen für eine frühzeitige Konvergenz. Gesteuert wird die Konvergenz der Population über die Parameterwahl von p_{cr} und p_m . In jeder Generation wird das Maß an Konvergenz durch den prozentualen Anteil identer Lösungen bestimmt und die Werte von p_m und p_{cr} adaptiv an die aktuelle Konvergenz angepasst. Je stärker eine Generation konvergiert, desto höher werden die Werte für p_m und p_{cr} gewählt mit dem Ziel, die Lösungssuche in noch unerforschte Suchräume zu dirigieren. Die Bestimmung der Werte von p_m und p_{cr} wird im Kapitel 8.4 dargestellt.

Die PRG+GA Heuristik wird zum einen durch das Erreichen von $iterGA$ Generationen und zum anderen durch eine Stagnation der Lösungssuche terminiert. Hierfür werden durch Einsatz einer Laufvariable die aufeinanderfolgenden Generationen gezählt, in denen keine Verbesserung des ZF-Wertes erzielt werden konnte, und die Lösungssuche wird nach Erreichen von $0.5iterGA$ Generationen dieser Art terminiert. Das zweite Terminierungskriterium zielt darauf ab, dass der Wert für den Parameter $iterGA$ für jene Instanzen, in denen tatsächlich eine Verbesserung durch den PRG+GA erzielt wird, hinreichend hoch angesetzt werden kann. Datensätze, bei denen die Startlösungheuristik bereits sehr gute bzw. optimale Lösungen gefunden hat und somit keine Verbesserung mehr erfolgt, sollen durch das zweite Terminierungskriterium frühzeitig abgebrochen werden.

6.2.1 Pseudocode

Bevor der iterative Prozess des PRG+GA startet, wird die erste Population P_0 initialisiert, indem durch Einsatz der Multi-PRG Heuristik a Startlösungen mit dem Lösungsvektor $\omega_{i,s} \forall i \in [n], s \in [a]$ erstellt werden. Demnach bestimmt der Parameter a die konstante Größe der Population in jeder Generation.

Algorithm 6.1 Pseudocode PRG+GA

Require: Parameterwert für $iterGA$ (Terminierungskriterium), a (Größe der Population) und nb_C (Anzahl der zu erstellenden Nachkommen)

```

1: Initialisiere Population  $P_0$  mit  $a$  unabhängigen Individuen durch Einsatz der Multi-PRG Heuristik
   (siehe 4.2)
2: while  $t \leq iterGA$  OR  $cnt_{improv} \leq 0.5iterGA$  do
3:   Berechne Fitness  $f$  aller Individuen  $\omega_{i,s}$ 
4:   Sortiere Individuen der aktuellen Population  $P_t$  aufsteigend nach Fitness
5:    $PA1_i \leftarrow$  Individuum mit bester Fitness
6:    $PA2_i \leftarrow$  Zufallsauswahl aus den ersten  $0.5a$  Individuen mit schlechtester Fitness
7:   for  $d = 0$  to  $nb_C$  do ▷ Rekombination
8:     Setze alle Elemente  $i$  von  $C_{i,d}$  auf 0
9:      $cp \leftarrow$  Gleichverteilte Zufallszahl  $R_1 \in [1, n]$ 
10:    if  $R_2 \in [0, 1] \leq p_{cr}$  then
11:      for  $i = 0$  to  $cp$  do
12:         $C_{i,d} = PA1_i$ 
13:      end for
14:      for  $i = cp$  to  $n$  do
15:         $C_{i,d} = PA2_i$ 
16:      end for
17:    end if
18:  end for
19:  Mutation der Nachkommen mit einer WK von  $p_m$  ▷ Bit-Flip Mutation
20:  Reparaturmechanismus falls Lösungen von Nachkommen unzulässig sind ▷ Alg. 6.2
21:  Ersetze die  $nb_C$  schlechtesten Individuen der aktuellen Population  $P_t$ 
22:  Aktualisiere  $p_m$  und  $p_{cr}$  in Abhängigkeit der Konvergenz von  $P_t$  ▷ siehe Tabelle 8.11
23:  if Falls Verbesserung von ZF-Wert erreicht wurde then
24:     $cnt_{improv} = 0$ 
25:  else
26:     $cnt_{improv} = cnt_{improv} + 1$ 
27:  end if
28:   $t = t + 1$ 
29: end while

```

In Zeile 3 bis 6 werden alle notwendigen Schritte für den bereits beschriebenen Selektionsmechanismus der Elternteile ausgeführt. Die Rekombination beider Elternteile erfolgt in Zeile 7 bis 18 auf Basis einer Single-Point-Crossover-Methode. Hierfür wird in Zeile 9 für die Erstellung eines jeden Nachkommen $C_{i,d} \forall d \in [nb_C]$ auf Zufallsbasis im Intervall $[1, n]$ ein Crossover-Punkt cp ausgesucht. Eine Rekombination beider Elternteile erfolgt jedoch nur dann, wenn eine gleichverteilte und rationale Zufallszahl R_2 mit zwei Nachkommastellen im Intervall $[0,1]$ kleiner oder gleich dem aktuellen

Wert der Crossover-Wahrscheinlichkeit p_{cr} ist (siehe Zeile 10). Die optimalen Werte der Parameter nb_C und a werden im Zuge einer Parameteranpassung ermittelt. Da jedoch die Ersetzungsstrategie in jeder Iteration auf dem Prinzip erfolgt, dass die nb_C Individuen mit der schlechtesten Fitness ersetzt werden, darf der Parameter nb_C nicht größer als a sein. In der Zeile 26 werden durch Einsatz der Variable cnt_{improv} alle aufeinanderfolgenden Generationen ohne Verbesserung des ZF-Wertes gezählt und nach Erreichen von $0.5iterGA$ Generationen dieser Art die Lösungssuche terminiert.

6.2.2 Reparaturmechanismus

Die Erstellung der Nachkommen $C_{i,d}$ weist durch den Prozess der Rekombination und der Mutation zwei Randomisierungskomponenten auf, wodurch der Algorithmus sowohl zu zulässigen als auch zu nicht zulässigen Chromosomen führen kann. Daher ist hier der Einsatz eines geeigneten Reparaturmechanismus sinnvoll und führt zu einer erheblichen Verbesserung der PRG+GA Heuristik. Im folgenden Kapitel wird anhand eines Pseudocodes der eingesetzte Reparaturmechanismus vorgestellt, welcher dann zum Tragen kommt, wenn die Gewichtssumme der jeweiligen Nachkommen die Kapazität C überschreitet. Der Algorithmus verfährt nach dem Prinzip, dass in jeder Iteration für alle Objekte, die sich bereits im Rucksack befinden, der minimale Profitverlust berechnet wird, welcher durch Exklusion eines Objektes aus dem Rucksack entsteht. Das Objekt, welchem der geringste Profitverlust zugeordnet ist, wird aus der Sublösung entfernt. Dieser iterative Prozess wird solange wiederholt, bis die Einhaltung der Gewichtsbeschränkung gewährleistet ist.

Algorithm 6.2 Reparaturmechanismus PRG+GA

```

1: for  $d = 0$  to  $nb_C$  do
2:   while Gewicht von  $C_{i,d} \geq C$  do
3:      $z_{BR} \leftarrow 0$ 
4:     for Jedes Objekt  $i$  von  $C_{i,d}$  welches in den Rucksack inkludiert ist do
5:       Entferne Objekt  $i$  aus Rucksack
6:       Berechne neuen ZF-Wert  $z_R$ 
7:       Füge Objekt  $i$  dem Rucksack wieder hinzu
8:       if  $z_R \geq z_{BR}$  then
9:          $z_{BR} \leftarrow z_R$ 
10:         $ind_{BR} \leftarrow i$ 
11:      end if
12:    end for
13:     $C_{ind_{BR},d} \leftarrow 0$ 
14:  end while
15: end for

```

In Zeile 4 bis 7 des Algorithmus 6.2 werden sukzessiv die in den Rucksack inkludierten Objekte eines Nachkommen $C_{i,d}$ betrachtet und zwischenzeitlich das betrachtete Objekt aus der Lösung entfernt. Der durch die Exklusion verringerte ZF-Wert z_R wird berechnet und anschließend das betrachtete Objekt der Lösung wieder hinzugefügt. In Zeile 8 bis 11 wird der vorerst höchste z_R -Wert bestimmt und in die Variable z_{BR} gespeichert. Dieser Prozess wird für alle in den Rucksack inkludierten Objekte des Nachkommen wiederholt und der Index ind_{BR} bestimmt jenes Objekt, dessen Exklusion den geringsten Profitverlust verursacht. In Zeile 13 wird das Objekt mit dem Index ind_{BR} aus der Lösung des Nachkommen d endgültig entfernt. Die sukzessive Entfernung von Objekten wird so lange wiederholt, bis die Gewichtsbeschränkung nicht mehr verletzt wird. Die *for*-Schleife d in der Zeile 1 stellt sicher, dass dieser Prozess für alle Nachkommen, deren Lösungsvektoren die Gewichtsbeschränkung verletzen, wiederholt wird.

7. Simulated Annealing

7.1 Theoretische Grundlagen

Auf Grundlage des von Metropolis u. a. [1953] veröffentlichten Metropolis-Algorithmus erkannten die Autoren Kirkpatrick u. a. [1983] und V., Černý [1985] unabhängig voneinander den Zusammenhang zwischen der Minimierung einer Kostenfunktion eines kombinatorischen Optimierungsproblems und dem thermodynamischen Abkühlungsprozess eines Metalls, welches dabei einen energiearmen Zustand nahe am Optimalwert erreicht. Metropolis u. a. [1953] entwickelten eine effiziente Simulation, welche die Evolution eines Metalls hin zu einem thermischen Equilibrium beschreibt. Durch einen Austausch von Kosten mit Energie im Metropolis-Algorithmus und durch eine sequenziellen Verringerung der Temperaturwerte erhielten Kirkpatrick u. a. [1983] einen kombinatorischen Optimierungsalgorithmus, welchen sie als Simulated Annealing (SA) bezeichneten [Laarhoven u. Aarts, 1987, Preface].

Ein SA Algorithmus zielt darauf ab, einem lokalen Optimum zu entfliehen, indem bei einer lokalen Suche aufgrund einer Wahrscheinlichkeitsfunktion ein ZF-Wert verschlechternder Schritt zeitweise erlaubt wird. Ein Schritt, welcher den ZF-Wert verbessert, wird hingegen stets erlaubt. Betrachtet man ein Minimierungsproblem mit einer aktuellen Lösung ω und einer Nachbarschaftslösung ω' , so liefert die Nachbarschaftslösung ein schlechteres Ergebnis unter der Bedingung, dass $z(\omega) < z(\omega')$. Die Nachbarschaftslösung ω' wird jedoch trotz Verschlechterung angenommen, wenn für eine gleichverteilte Zufallszahl R im Intervall $[0,1]$ die Bedingung $R \leq e^{\frac{-|z(\omega')-z(\omega)|}{T_i}}$ erfüllt wird [Taha, 2011, S. 399f]. Da die Differenz der Ausgangslösung ω und der Nachbarschaftslösung ω' in Betragsstriche gesetzt ist und das resultierende Ergebnis als negativ angesehen wird, kann diese Formel sowohl für ein Minimierungsproblem als auch für ein Maximierungsproblem angewendet werden.

Der Parameter T_i steht für die sequenzielle Temperaturverringern und wird aufgrund einer gewissen Wahrscheinlichkeitsfunktion abhängig vom Parameter nb_{acc} , welcher die Anzahl der Akzeptanzperioden beschreibt, verringert. Es wird also bei jedem Nachbarschaftsschritt mit der Formel (7.1) die Differenz des vorherigen und des aktuellen ZF-Wertes berechnet. Anschließend wird mit der Formel (7.2) bestimmt, ob ein Nachbarschaftsschritt trotz Verschlechterung der ZF zugelassen wird.

$$\delta = z[\omega'] - z[\omega] \quad (7.1)$$

$$R \leq e^{\frac{-|\delta|}{T_t}} \quad (7.2)$$

In der Abbildung 7.1 wird eben beschriebene Methodik eines generischen Simulated Annealing Verfahrens anschaulich anhand eines Flussdiagramms aufgezeigt.

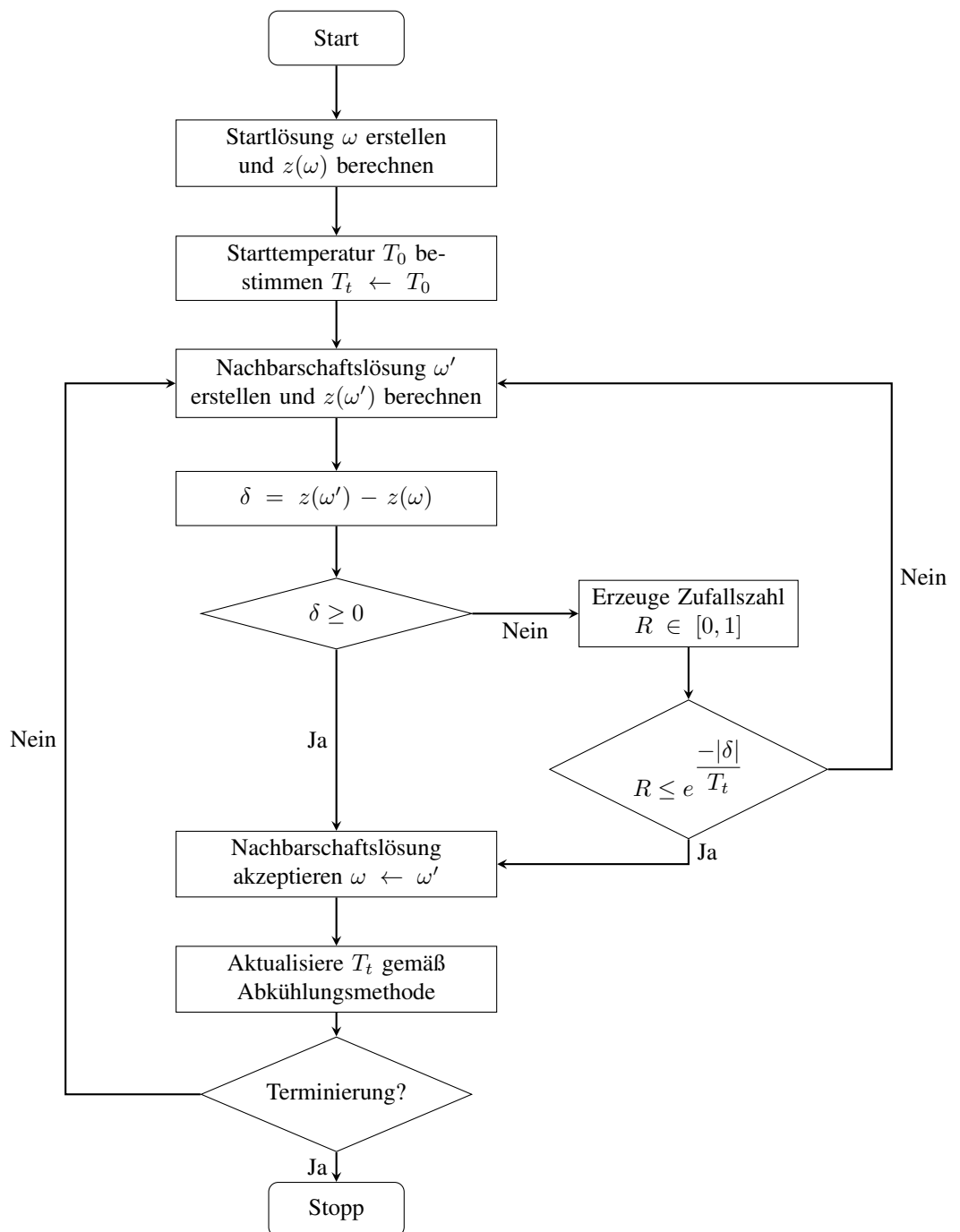


Abbildung 7.1: Übersicht generisches Simulated Annealing Verfahren - Eigene Darstellung

Eine SA-Heuristik kann demnach als eine auf einer Wahrscheinlichkeitsfunktion basierende LS angesehen werden. Wie man in der Abbildung 7.2 erkennen kann, steigen die Funktionswerte der Exponentialfunktion im negativen Bereich der x-Skala mit größer werdendem x. Bei einem x-Wert von 0 erreicht die Funktion im negativen Bereich ihren Maximalwert bei 1, und dementsprechend wird eine Nachbarschaftslösung mit einer Wahrscheinlichkeit von 100 % angenommen. Dieses Verhalten ist kohärent mit dem Grundgedanken der SA Heuristik, da der Quotient der Exponentialfunktion, in der der Formel (7.2) nur dann 0 sein kann, wenn die Nachbarschaftslösung denselben ZF-Wert wie die Ausgangslösung aufweist.

Mit einem kleiner werdenden x-Wert nimmt auch die Wahrscheinlichkeit ab, dass eine ZF-Wert verschlechternde Lösung angenommen wird. Daher wird in der SA Heuristik der Wert für den Parameter T_i sukzessiv verringert, wodurch der Quotient der Exponentialfunktion steigt und infolgedessen der x-Wert sinkt. Aufgrund dessen ist es auch wahrscheinlicher, dass zu Beginn des iterativen Suchprozesses des SA Algorithmus eine Nachbarschaftslösung, welche den ZF-Wert verschlechtert, angenommen wird.

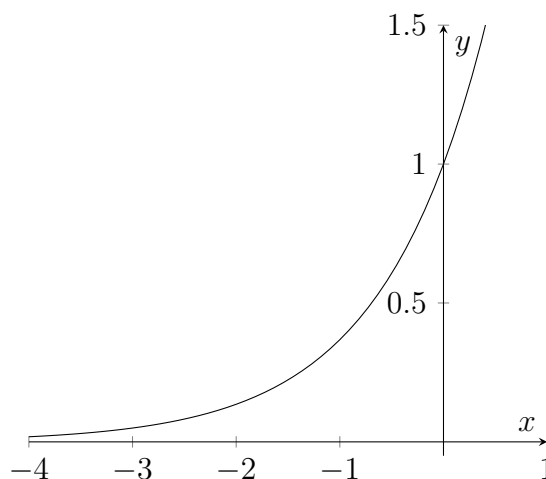


Abbildung 7.2: Grafische Darstellung der Exponentialfunktion $y = e^x$

7.1.1 Abkühlungsverfahren

Die Gestaltung einer sinnvoll gewählten Abkühlungsmethode ist ein essenzieller Schritt bei der Implementierung eines SA Verfahrens, welches auf ein spezifisches Optimierungsproblem angewandt wird. Hierbei wird in einem ersten Schritt die Anfangstemperatur T_0 bestimmt und darauffolgend nach einem gewählten Schema der Parameter T_t sukzessiv reduziert. Durch eine Verringerung des Parameters T_t wird es kontinuierlich unwahrscheinlicher, dass ein Schritt, welcher den ZF-Wert verschlechtert, angenommen wird. Es werden in der Literatur diverse Methoden zur Berechnung der Anfangstemperatur vorgestellt, wobei sich die von Johnson u. a. [1989] vorgestellte Methode besonders großer Beliebtheit erfreut. Im folgenden Kapitel werden die Methode von Johnson u. a. [1989] zur Berechnung der Starttemperatur sowie drei verschiedene Abkühlungsmethoden des Kontrollparameters T_t präsentiert.

Bestimmung der Anfangstemperatur T_0 Die sukzessive Verringerung der Variable T_t startet in der ersten Iteration eines SA Verfahrens mit der Ausgangstemperatur T_0 . Die Ausgangstemperatur muss ausreichend hoch angesetzt werden, damit die lokale Suche am Anfang eines SA Algorithmus einer randomisierten lokalen Suche entspricht und es wahrscheinlicher ist, ein lokales Optimum zu verlassen. Wird der Kontrollparameter T_0 jedoch zu hoch gewählt, so werden oftmals eine Vielzahl von unnötigen randomisierten Nachbarschaftsschritten getätigt, bis der ZF-Wert kontinuierlich optimiert wird. Dies kann sich negativ auf die Rechenzeit des Algorithmus auswirken. Darüber hinaus kann es einen negativen Effekt auf die Qualität der Lösung haben, da ein SA Algorithmus oftmals durch eine festgelegte Anzahl von Iterationen terminiert wird. Bei der von Johnson u. a. [1989] vorgestellten Methode wird in einem Testdurchlauf die durchschnittliche Starttemperatur für eine im Vorhinein festgesetzte Akzeptanzwahrscheinlichkeit x_0 berechnet. Die Akzeptanzwahrscheinlichkeit liegt üblicherweise in einem Bereich von $0.50 < x_0 < 0.95$ [Lee u. Lee, 2014, S. 505]. Die Variable δ ist in diesem Fall die durchschnittliche Differenz der untersuchten Nachbarschaftsschritte des Testlaufs.

$$T_0 = \frac{-|\delta|}{\ln x_0} \quad (7.3)$$

Logarithmische Abkühlungsmethode Geman u. Geman [1984] präsentieren eine logarithmische Abkühlungsmethode, wobei die Temperatur T_t in der Iteration t durch Einsatz einer positiven Konstante λ , welche unabhängig von t ist, berechnet wird. Hajek [1988] beweist, dass die logarithmische Abkühlungsmethode bei einer unlimitierten Rechenzeit stets zu einem globalen Minimum führt, unter der Bedingung, dass die Variable λ größer oder gleich dem maximalen Energieunterschied aller möglichen Nachbarschaftsschritte ist. Bei diesem Verfahren verringert sich die Temperatur jedoch äußerst langsam, wodurch der Rechenaufwand stark beeinträchtigt wird und daher diese Methode vorwiegend von theoretischer und weniger von praxisrelevanter Bedeutung ist.

$$T_t = \frac{\lambda}{\log(1+t)} \quad (7.4)$$

Geometrische Abkühlungsmethode Bei dieser Methode wird die Temperatur T_t durch eine Multiplikation der Starttemperatur mit dem Term α^t berechnet. Die Dimension des Parameters α liegt in einem Intervall von $(0,1)$, und die Abkühlung verläuft umso langsamer, je höher der Wert für α angesetzt wird [Kirkpatrick u. a., 1983].

$$T_t = T_0 \alpha^t \quad (7.5)$$

Lineare Abkühlungsmethode Kirkpatrick u. a. [1983] stellen eine weitere bekannte Abkühlungsmethode vor, bei der die lineare und sukzessive Verringerung der Temperatur auf Basis eines Faktors η erfolgt. Der Parameter η kann hierbei den Wert einer beliebigen reellen, positiven Zahl annehmen, wobei jedoch im Gegensatz zur geometrischen Abkühlungsmethode ein hoher Parameterwert zu einer schnelleren Abkühlung führt.

$$T_t = T_0 - \eta t \quad (7.6)$$

7.2 Paarweise randomisierte Greedy-Heuristik mit Simulated Annealing

Die PRG+SA ist die dritte und letzte Verbesserungsheuristik, welche im Zuge dieser Masterarbeit entwickelt und vorgestellt wird. Wie bereits im vorangegangenen Kapitel beschrieben, basiert ein SA Verfahren auf einer sukzessiven Temperaturverringerung. Hierfür werden drei verschiedene Parameter benötigt, deren optimale Werte unter Zuhilfenahme einer Parameteranpassung im Kapitel 8.5 ermittelt werden. Diese Parameter sind die Starttemperatur T_0 , der Temperaturverringerungsfaktor α der geometrischen Abkühlungsmethode und die Terminierungstemperatur T_{BOUND} .

Ein zu untersuchender Nachbarschaftsschritt erfolgt auf Zufallsbasis, indem der Tausch zweier zufällig ausgewählter Objekte vorgenommen wird. Hierfür wird unter Zuhilfenahme einer *while*-Schleife so lange unter den zur Verfügung stehenden Objekten gewählt, bis ein Objekt, welches bereits Teil der Lösung ist, und ein Objekt außerhalb des Rucksacks gezogen werden. Für den reibungslosen Ablauf des Algorithmus ist ein Mechanismus notwendig, welcher die *while*-Schleife wieder verlässt, für den Fall, dass keine Kombination eines auszutauschenden Objektpaars gefunden werden kann, welche diese Bedingung erfüllt. Im Sinne einer übersichtlichen Darstellung wird auf die Abbildung dieses Mechanismus im Pseudocode verzichtet. Die Kapazitätsbeschränkung wird bei Auswahl der Objekte nicht berücksichtigt, weshalb dieser Auswahlmechanismus zu unzulässigen Lösungsvektoren führen kann. Daher ist der Einsatz eines Reparaturmechanismus nötig, welcher analog zu jenem aus der PRG+GA Heuristik (siehe Algorithmus 6.2) verfährt.

Der Parameter nb_{trans} bestimmt, wie viele Nachbarschaftsschritte maximal im Durchlauf einer vollständigen Nachbarschaftssuche untersucht werden, wobei diese LS unterbrochen wird, sobald eine Temperaturverringerung stattgefunden hat. Eine Temperaturverringerung erfolgt nach nb_{acc} erreichten Akzeptanzperioden, welche sich durch einen Nachbarschaftsschritt auszeichnen, der aufgrund des Metropolis-Kriteriums zugelassen wird. Demnach muss sichergestellt werden, dass der Parameterwert von nb_{acc} nicht größer ist als jener von nb_{trans} , da sonst keine Temperaturverringerung statt-

finden kann.

Die PRG+SA Heuristik kann auf zwei verschiedene Arten terminiert werden. Zum einen wird der iterative Suchvorgang abgebrochen, wenn die aktuelle Temperatur die Terminierungstemperatur unterschreitet, und zum anderen auf Basis einer vorgegebenen Anzahl von aufeinanderfolgenden vollständigen Nachbarschaftsuchen, in denen keine Temperaturverringerung mehr stattgefunden hat.

7.2.1 Pseudocode

Das PRG+SA Verfahren benötigt wie zuvor der PRG+TS und der PRG+GA ein Set an Startlösungen, welche in dem Vektor $\omega_{i,s} \forall i \in [n], s \in [a]$ gespeichert sind. Darüber hinaus werden die Parameterwerte der Parameter T_0 , α , T_{BOUND} , nb_{trans} und nb_{acc} benötigt.

Algorithm 7.1 Pseudocode PRG+SA

```

1: while  $T_t \geq T_{BOUND}$  do
2:    $temp\_change \leftarrow \text{false}$ 
3:   while  $iter_{trans} \leq nb_{trans}$  do
4:     Erstelle zufällige Nachbarschaftslösung  $\omega'$ 
5:     Falls  $\omega'$  unzulässig setze Reparaturmechanismus ein ▷ siehe Kapitel 6.2.2
6:      $\delta = z(\omega) - z(\omega')$ 
7:      $p_{acc} = e^{\frac{-|\delta|}{T_t}}$ 
8:      $R \leftarrow$  Gleichverteilte Zufallszahl im Intervall  $[0, 1]$ 
9:     if  $\delta \geq 0$  OR  $R \leq p_{acc}$  then
10:       $\omega \leftarrow \omega'$ 
11:       $cnt_{acc} = cnt_{acc} + 1$  ▷ Erhöhe Anzahl erreichter Akzeptanzperioden
12:    end if
13:    if  $cnt_{acc} = nb_{acc}$  then
14:       $t = t + 1$ 
15:       $T_t = T_0 \alpha^t$ 
16:       $change_t \leftarrow \text{true}$ 
17:       $cnt_{temp} = 0$ 
18:       $cnt_{acc} = 0$ 
19:       $iter_{trans} = nb_{trans}$  ▷ Verlasse while-Schleife der 3. Zeile
20:    end if
21:     $iter_{trans} = iter_{trans} + 1$ 
22:  end while
23:   $iter_{trans} = 1$ 
24:  if  $change_t = \text{false}$  then
25:     $cnt_{temp} = cnt_{temp} + 1$ 
26:  end if
27:  if  $cnt_{temp} = 5$  then
28:     $T_t = T_{BOUND}$  ▷ Terminierung der PRG+SA Heuristik
29:  end if
30: end while

```

In Zeile 1 des Pseudocodes 7.1 wird das Terminierungskriterium auf Basis der sukzessiven Temperaturverringering definiert. Die PRG+SA Heuristik wird so lange fortgeführt, bis das aktuelle Temperaturlevel einen vordefinierten Wert T_{BOUND} unterschreitet. Für jedes Temperaturlevel T_t wird eine Anzahl von nb_{trans} zufälligen Nachbarschaftsschritten getätigt, und diese werden im Falle einer unzulässigen Lösung repariert (siehe Zeile 5). Der ZF-Wert der neu gefundenen Lösung wird berechnet und in Zeile 6 die Differenz der ZF-Werte der neu gefundenen und der aktuellen Lösung ermittelt. Des Weiteren wird in Zeile 7 bis 8 die Akzeptanzwahrscheinlichkeit p_{acc} berechnet und eine gleichverteilte Zufallszahl R im Intervall $[0, 1]$ bestimmt. Die neu gefundene Lösung wird im Falle einer Verbesserung des ZF-Wertes stets angenommen und im Falle eines negativen Wertes von δ unter Erfüllung des Metropolisikriteriums (Zeile 7) trotz Verschlechterung des ZF-Wertes angenommen. In beiden Fällen wird die Zählvariable cnt_{acc} um 1 erhöht und die Temperatur bei Erreichung einer fixen Anzahl von Akzeptanzperioden nb_{acc} verringert (Zeile 13 bis 20). Tritt dies ein, so wird das aktuelle Temperaturlevel gemäß der geometrischen Abkühlungsmethode reduziert, die *while*-Schleife aus der Zeile 3 frühzeitig verlassen, und es werden erneut nb_{trans} zufällige Nachbarschaftsschritte für das neue Temperaturlevel T_t untersucht. In Zeile 23 wird die Laufvariable $iter_{trans}$ auf 1 gesetzt, damit die *while*-Schleife aus Zeile 3 erneut startet. In den Zeilen 24 bis 29 wird das zweite Terminierungskriterium bestimmt, welches sicherstellt, dass die PRG+SA Heuristik nach 5 aufeinanderfolgenden randomisierten LS, in denen keine Temperaturreduzierung stattgefunden hat, abgebrochen wird.

8. Datenauswertung

Im folgenden Kapitel wird ein Einblick in die angewandte Methodik der Datenauswertung, den verwendeten Testinstanzen, ein Überblick der Kontrollparameter sowie die Evaluierung der ermittelten Testergebnisse geboten. Die in dieser Masterarbeit vorgestellten Algorithmen wurden in C# implementiert und in einem Microsoft Visual Studio 2017 Compiler auf einem Computer mit einem Intel(R) Core(TM) i5-4300 Prozessor CPU @ 1.90GHz 2.50 GHz und 8 GB RAM-Speicher ausgewertet. Auf dem Computer läuft das Microsoft Windows 10 Pro Betriebssystem.

8.1 Methodik

Im Zuge der Datenauswertung werden sowohl die erreichten ZF-Werte als auch die durchschnittlichen Laufzeiten (LZ) der in dieser Arbeit vorgestellten Metaheuristiken bestimmt und deren Performances evaluiert und verglichen. Um die statistische Aussagekraft der erzielten Ergebnisse zu bekräftigen, werden die Metaheuristiken für eine bestimmte Anzahl von randomisierten Läufen wiederholt. Die Performance-Evaluierung erfolgt auf Basis eines geometrischen Mittelwerts der Verhältnisse aus erreichtem ZF-Wert und einem Referenzwert für alle getesteten Instanzen gleicher Inputgröße. Liegt für die getesteten Instanzen eine exakte Lösung vor, so wird als Referenzwert die Optimallösung herangezogen, wodurch eine akkurate Performance-Evaluierung getroffen werden kann.

Die in dieser Arbeit zur Testung herangezogenen schweren Instanzen werden nach der Methodik von Schauer [2016] bestimmt, welcher sich in seiner Abhandlung mit der Aussagekraft der in der Literatur oftmals verwendeten QKP-Instanzen beschäftigt. Wie man anhand der Literaturrecherche (siehe Kapitel 3) erkennen kann, existieren bereits eine Vielzahl von exakten sowie auch (meta-)heuristischen Lösungsverfahren, welche optimale bzw. Lösungen von sehr hoher Qualität erzielen können. Jedoch hat Schauer [2016] bewiesen, dass auch bei Einsatz einer simplen Heuristik für eine große Klasse von Testinstanzen (asymptotisch) sehr gute Ergebnisse generiert werden können. In diese Klasse fallen auch jene Instanzen, welche nach der sehr bekannten und in der Literatur oftmals verwendeten Methode von Gallo u. a. [1980] erstellt werden. Bei dieser Methode wird zuerst durch Einsatz einer Bernoulli-Verteilung mit einem Dichteparameter δ bestimmt, ob quadratische Profite q_{ij} in dieser Instanz vorliegen oder nicht. Wenn quadratische Profite auftreten, werden im nächsten Schritt die ganzzahligen q_{ij} -Werte durch Einsatz einer gleichverteilten Zufallszahl im Intervall $[0,100]$ bestimmt. Die linearen Profite p_i sind stets vorhanden und werden anhand von einer unabhängigen und gleichverteilten Zufallsvariablen definiert. Die Gewichte w_i und die Kapazität C werden durch Einsatz

einer Gleichverteilung aus dem Intervall $[1, 50]$ bzw. aus dem Intervall $[50, \sum w_i]$ zufällig ausgewählt. In der Abhandlung von Schauer [2016] wird eine neue Methode zur Bestimmung von Instanzen vorgestellt, welche darauf beruht, dass ein Clique der Größe $\lfloor \sqrt{n} \rfloor$ für einen Graphen $G(n, 1/2)$ gefunden werden soll. Darüber hinaus liefert Schauer [2016] eine empirische Evidenz dafür, dass es schwerer ist für jene *hidden clique* Instanzen die Optimallösung zu bestimmen. Die Kapazität C des Rucksacks ermittelt sich durch $\lfloor \sqrt{n} \rfloor$, die Gewichte w_i eines jeden Knoten sind 1, die linearen Profite p_i sind stets 0 und die quadratischen Profite q_{ij} nehmen dann einen Wert von 1 an, wenn zwei Knoten mit einer Kante verbunden sind. Die Testinstanzen dieser Arbeit werden in schwere (SI) und leichte Instanzen (LI) eingeteilt, wobei die 40 Datensätze der schwer zu lösenden Testinstanzen nach eben beschriebener Clique-Methode bestimmt werden und für jene Datensätze mit einer großen Wahrscheinlichkeit die Optimallösungen vorliegen. Diese Optimallösungen werden durch die Berechnung von $\frac{1}{2}(\lfloor \sqrt{n} \rfloor)(\lfloor \sqrt{n} \rfloor - 1)$ ermittelt.

Für die 560 Datensätze der leichten Testinstanzen liegt kein Optimalwert vor, weswegen als Referenzwert für die Berechnung der geometrischen Mittelwerte die Ergebnisse der von Schauer [2016] vorgestellten OG-Heuristik herangezogen werden. Die OG-Heuristik ist eine Adaption des von Julstrom [2005] vorgestellten Genetischen Algorithmus mit einer maximalen Laufzeit von 1400 Sekunden. Des Weiteren werden die Instanzen hinsichtlich der Inputgröße n gegliedert, wodurch vier weitere Klassen ($n = 200$, $n = 400$, $n = 600$ und $n = 800$) entstehen. Der Parameter L bestimmt die Anzahl der randomisierten Läufe.

In der Formel (8.1) kann man die allgemeine Berechnung des geometrischen Mittelwerts eines Testlaufs mit M verschiedenen Instanzen gleicher Inputgröße n und einer Wiederholung von L randomisierten Läufen erkennen.

E ... Anzahl der getesteten Instanzen gleicher Inputgröße ($e = 1, \dots, E$)

L ... Anzahl der randomisierten Läufe der Instanzen gleicher Inputgröße ($l = 1, \dots, L$)

z_{r_e} ... Referenzlösung der Instanz e

$z_{l,e}$... Erreichter Zielfunktionswert für die Instanz e beim randomisierten Lauf l

$$\bar{x}_{geom} = 2 - \sqrt[L \cdot E]{\prod_{l=1}^L \prod_{e=1}^E (2 - \frac{z_{l,e}}{z_{r_e}})} \quad (8.1)$$

Im Speziellen wird hier auf einen sogenannten *shifted* geometrischen Mittelwert zurückgegriffen. Dieser zeichnet sich dadurch aus, dass zuerst die Produkte des geometrischen Mittelwerts unter der Wurzel von einer positiven Konstante abgezogen werden, und im Anschluss an die Berechnung der Wurzel das Ergebnis erneut von der positiven Konstante subtrahiert wird. Im Falle der Formel (8.1) handelt es sich demnach um einen um 2 geshifteten geometrischen Mittelwert. Das *shiften* führt dazu, dass im Vergleich zu einem arithmetischen Mittelwert das Ergebnis nicht durch hohe Ausreißer und im Vergleich zu einem reinen geometrischen Mittelwert das Ergebnis nicht durch niedrige Ausreißer stark verzerrt wird.

8.1.1 Parameteranpassung

Ein essenzieller Schritt bei der Anwendung einer Metaheuristik für die Lösung eines Realproblems ist die Festlegung sinnvoll gewählter Parameter und Parameterwerte. Das Vornehmen einer Parameteranpassung ist nicht nur sehr zeitintensiv, sondern benötigt oftmals die Expertise eines erfahrenen Algorithmus-Experten. In der Literatur gibt es eine Vielzahl von Erfahrungswerten bezüglich sinnvoll gewählter Parameter der bekannten metaheuristischen Lösungsverfahren. Jedoch kann dies nur als grobe Vorgabe herangezogen werden, da die optimalen Parameterwerte stark von einem spezifischen Realproblem abhängig sind und daher die Qualität einer Metaheuristik nur durch eine eigens durchgeführte Parameteranpassung maximiert werden kann [Scheibenpflug u. a., 2012]. Der Einfluss von Parametern beschränkt sich jedoch nicht nur auf die Güte, sondern auch maßgeblich auf die Laufzeit der Algorithmen und muss daher auch unter dem Gesichtspunkt einer Laufzeitoptimierung festgelegt werden.

Bei der Parameteranpassung wird zwischen zwei verschiedenen Strategien unterschieden. Eine Möglichkeit ist die *offline* Parameteranpassung, bei der die zu testenden Parameterwerte a priori festgelegt werden und meistens nur die Werte eines einzigen Parameters auf einmal bestimmt werden. Im Gegensatz dazu steht die *online* Parameteranpassung, bei welcher ein oder mehrere Parameter während der Ausführung einer Metaheuristik dynamisch, adaptiv oder selbstanpassend bestimmt werden [Talbi, 2009, S. 54]. Eine online Anpassung ist jedoch nur für numerische Parameter (z. B. Mutationswahrscheinlichkeit p_m) und nicht für symbolische Parameter (z. B. Crossover-Methode) sinnvoll. Aufgrund des benötigten Fachwissens und des umfangreichen Zeitaufwands gibt es in der Forschungswelt der Metaheuristiken bereits diverse Bestrebungen einer verbesserten und automatisierten Parameteranpassung. So stellen z. B. Nadi u. Khader [2011] einen genetischen Algorithmus vor, bei dem auf die Parameter p_m und p_{cr} verzichtet wird und stattdessen die Allele auf Basis einer Wahrscheinlichkeitsfunktion verändert werden. Diese berücksichtigt, in welchem Ausmaß die Veränderung den ZF-Wert beeinflusst. Smit u. Eiben [2009] stellen einen weiteren Ansatz der automatisierten Parameteranpassung vor, bei der die Parameteranpassung eines EA als eigenständiges Optimierungsproblem betrachtet wird und die Parameter durch Einsatz von Parameter-Kontrollalgorithmen festgelegt werden.

Im Rahmen dieser Masterarbeit wird vorwiegend eine *offline* Anpassung der Parameter vorgenommen. Lediglich die Mutationswahrscheinlichkeit p_m und die Crossover-Wahrscheinlichkeit p_{cr} werden im Zuge einer *online* Parameteranpassung in jeder Generation an das Maß der Konvergenz der Population angepasst. Bei der offline Parameteranpassung werden die Werte der Parameter innerhalb eines sinnvoll gewählten Intervalls getestet, wobei die oberen Schranken der Intervalle meist so gesetzt werden, dass eine maximale Laufzeit nicht überschritten wird. Hierbei werden entweder unter der Fixierung der übrigen Parameter einzelne Parameter getestet oder die optimale Kombination zweier Parameter unter Zuhilfenahme einer sogenannten Grid-Search ermittelt. Darüber hinaus werden diverse Parameter durch eine Funktion in Abhängigkeit des Inputparameters n ermittelt.

Für die Testung der Parameter werden jeweils 4 aussagekräftige Instanzen unterschiedlicher Instanzengröße der LI und SI ausgewählt. Dadurch liegt ein Parameter-Testbed mit insgesamt 8 Datensätzen mit jeweils einer schweren und einer leichten Instanz pro Instanzengröße vor. In der Tabelle 8.1 wird ein Überblick der bereits vorgestellten numerischen Parameter und deren Dimensionen geboten.

Parameter	Definition	Dimension	Programm
k	Zufallsauswahl aus den k -besten Werten der Greedy Matrix g_{ij}	$k \in \mathbb{N}$, $1 \leq k \leq \frac{n(n-1)}{2}$	PRG
a	Anzahl der zu generierenden Startlösungen	$a \in \mathbb{N}$	PRG
t	Länge der Tabu Liste	$t \in \mathbb{N}$	PRG+TS
p_N	Prozentsatz zur Bestimmung des Umfangs der Nachbarschaftssuche	$p_N \in \mathbb{R}^+$, $0 < p_N \leq 1$	PRG+TS
N_{size}	Umfang der Nachbarschaftssuche als ganzzahligen prozentualen Anteil von n	$N_{size} \in \mathbb{N}$	PRG+TS
$iterTS$	Anzahl der Iterationen von PRG+TS (Terminierungskriterium)	$iterTS \in \mathbb{N}$	PRG+TS
$iterGA$	Anzahl der Iterationen von PRG+GA (Terminierungskriterium)	$iterGA \in \mathbb{N}$	PRG+GA
p_C	Prozentsatz zur Bestimmung der Anzahl von Nachkommen pro Generation	$p_C \in \mathbb{R}^+$, $0 < p_C \leq 1$	PRG+GA
nb_C	Konstante Anzahl von Nachkommen pro Generation als ganzzahligen prozentualen Anteil von n	$nb_C \in \mathbb{N}$, $nb_C \leq a$	PRG+GA
p_{cr}	Crossover-Wahrscheinlichkeit	$p_{cr} \in \mathbb{R}^+$, $0 < p_{cr} \leq 1$	PRG+GA
p_m	Mutationswahrscheinlichkeit	$p_m \in \mathbb{R}^+$, $0 < p_m \leq 1$	PRG+GA
T_0	Starttemperatur von PRG+SA	$T_0 \in \mathbb{R}^+$	PRG+SA
T_{BOUND}	Terminierungstemperatur von PRG+SA	$T_{BOUND} \in \mathbb{R}^+$	PRG+SA
α	Temperaturverringerungsfaktor bei der geometrischen Abkühlungsmethode	$\alpha \in \mathbb{R}^+$, $0 < \alpha < 1$	PRG+SA
nb_{acc}	Anzahl an erfolgten Akzeptanzperioden bei denen eine Temperaturverringerung stattfindet	$nb_{acc} \in \mathbb{N}$, $nb_{acc} \leq nb_{trans}$	PRG+SA
nb_{trans}	Anzahl der zu untersuchenden Nachbarschaftsschritten	$nb_{trans} \in \mathbb{N}$	PRG+SA

Tabelle 8.1: Übersicht Parameter

8.2 Paarweise randomisierte Greedy-Heuristik

Für die Performance-Evaluierung der PRG Startlösungsheuristik müssen in einem ersten Schritt die optimalen Werte der Parameter k und a festgelegt werden. Ein Vortest der Parameter hat gezeigt, dass sowohl durch die Erhöhung des Parameters k als auch des Parameters a eine Verbesserung der ZF-Werte erzielt werden kann. Ein zu hoher Wert für a wirkt sich jedoch negativ auf die Rechendauer aus und wird daher vor allem unter Einbezug dessen, dass der PRG+SA und der PRG+TS Algorithmus auf alle gefundenen Startlösungen angewandt wird, bei den Vortests nach oben hin mit einem Wert von 30 begrenzt.

Die Parameter k und a beeinflussen sich gegenseitig maßgeblich, weswegen auch die Kombinationen sinnvoll gewählter Werte beider Parameter untersucht werden. Bei einem Wert von $k = 1$ kann der Parameter a nie größer als 1 sein, da keine Randomisierungskomponente vorliegt und demnach nur eine eindeutige Lösung gefunden werden kann.

Die Laufzeit, die benötigt wird, um eine einzige Startlösung einer Instanz zu bestimmen, wird als Indikator bei der Auswahl der a zu erstellenden Startlösungen herangezogen. Hierfür wird eine neue Variable $iterC \in \mathbb{N} \setminus \{0\}$ eingeführt, welche die maximalen Iterationen angibt, die bei einem sukzessiven Hinzufügen der Objekte gemäß der Reihenfolge nach absteigenden Gewichten durchlaufen werden, bis die Kapazität aufgebraucht ist. Ein hoher Wert von $iterC$ wirkt sich dementsprechend negativ auf die Rechendauer des Programms aus.

Der Parameter a wird als Funktion in Abhängigkeit der Variable $iterC$ definiert und der Funktionswert durch den Parameter $\bar{a}$ nach oben hin beschränkt. Eine Funktion des Parameters a muss die Bedingung erfüllen, dass die Variable $iterC$ nach oben hin unbeschränkt ist und eine Obergrenze an maximal zu erstellenden Startlösungen $\bar{a}$ definiert werden kann. Daher liegt hier die Anwendung einer Funktion mit einem asymptotischen Verhalten nahe. Eine simple Funktion der Form $f(x) = \frac{\bar{a}}{x}$ würde diese Bedingungen erfüllen, wie man jedoch in der Abbildung 8.1 erkennen kann, konvergiert diese Funktion sehr schnell gegen 0. Die Konvergenzgeschwindigkeit von $f(x)$ kann jedoch verlangsamt werden, indem der Divisor der Funktion durch eine Wurzelfunktion ersetzt wird. Darüber hinaus verlangsamt ein Erhöhen des Wurzelexponenten b erneut die Konvergenzgeschwindigkeit.

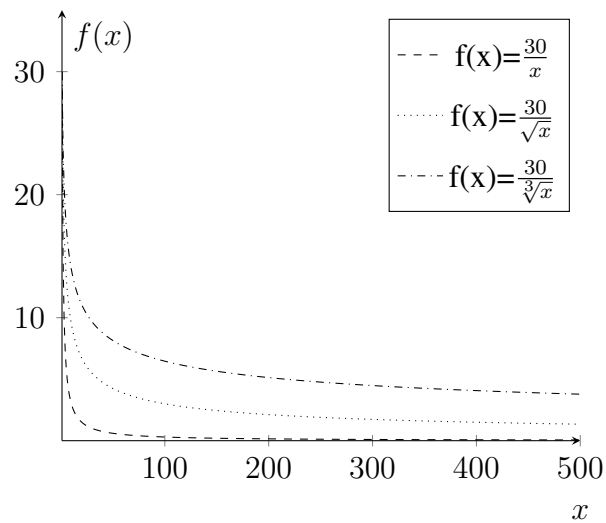


Abbildung 8.1: Vergleich von Funktionen mit asymptotischen Verhalten und unterschiedlichen Konvergenzgeschwindigkeiten

Die Berechnung des Parameters a in Abhängigkeit der Variable $iterC$ wird in der Funktion (8.2) dargestellt. Der Abbildung 8.2 kann man die Verläufe der Funktion a in Abhängigkeit von $iterC$ bei unterschiedlichen Wurzelexponenten b entnehmen.

$$a(iterC) = \frac{\bar{a}}{\sqrt[b]{iterC}} \quad (8.2)$$

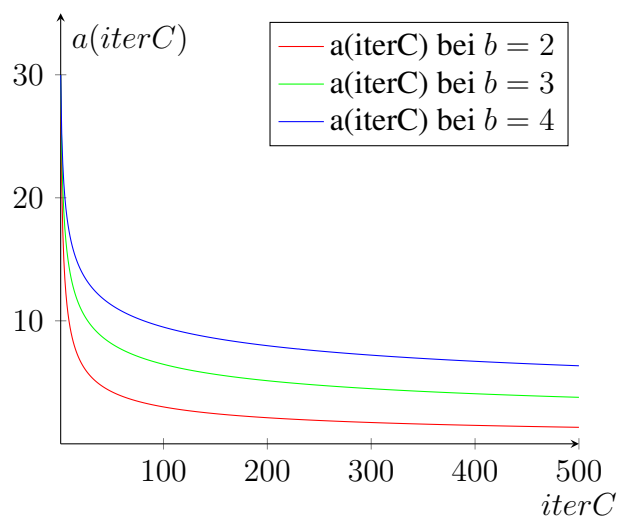


Abbildung 8.2: Funktion des Parameters $a(iterC)$ bei $\bar{a} = 30$ und $b \in [2, 4]$

Sowohl der Parameter a als auch die Parameter $\bar{a}$ und b sind offline angepasste Parameter. Jedoch werden die optimalen Parameterwerte der Parameter $\bar{a}$ und b a priori durch Einsatz einer Grid-Search bestimmt und der Parameterwert für a für eine spezifische Instanz vor Durchlauf der PRG Heuristik mit der Formel (8.2) ermittelt. Anhand der Testinstanzen des Parameter-Testbeds wird vorab ein Vortest vorgenommen, welcher darauf abzielt, die Intervalle der zu testenden Werte der finalen Parameteranpassung zu definieren. Bei diesem Vortest wird der Parameter $\bar{a}$ in einem Wertebereich von $1 \leq \bar{a} \leq 30$ und der Parameter b in einem Wertebereich von $5 \leq b \leq 20$ getestet. Der Parameter k wird vorübergehend auf einen Wert von 50 fixiert, um für den Vortest und der finalen Parameteranpassung von a eine ausreichende Randomisierungskomponente zu gewährleisten. Die Ergebnisse werden in der Tabelle 8.2 dargestellt und eine Performance-Evaluierung auf Basis des geometrischen Mittelwerts der erzielten ZF-Werten $\bar{x}_{geom}$, der durchschnittlich erstellten Startlösungen $\emptyset a$ und der durchschnittlichen Laufzeiten LZ getroffen. In dem Fall, dass der Parameter $\bar{a}$ einen Wert von 1 annimmt und somit nur eine einzige Startlösung erstellt wird, hat der Parameter b keinen Einfluss auf das Ergebnis der Funktion $a(iterC)$. Daher werden die Ergebnisse der Zeile ($\bar{a} = 1$) nur in einer einzigen Spalte der Tabellen dargestellt. Je höher die Werte für $\bar{a}$ und b gesetzt werden, desto höher ist die Anzahl der maximal zu erstellenden Startlösungen. An dieser Stelle wird erneut darauf hingewiesen, dass der Parameter a nur eine Obergrenze ist, da die iterative Suche des Multi-PRG Algorithmus nach 10a aufeinanderfolgenden Iterationen in denen keine einzigartige Startlösung erstellt werden kann, abgebrochen wird (siehe Kapitel 4.2). Den Ergebnissen der Tabelle 8.2 kann man entnehmen, dass das Erhöhen der Parameterwerte von $\bar{a}$ und b eine stark positive Auswirkung auf die erzielten ZF-Werte hat. Im Vergleich zu dem einfachen Fall, dass nur eine Startlösung generiert wird, können die Ergebnisse bei einer Festlegung der Parameterwerte von $\bar{a} = 30$ und $b = 20$ für alle Instanzengrößen um 4-5 Prozentpunkte verbessert werden. Betrachtet man die PRG-Heuristik demnach als eine alleinstehende Heuristik, macht es Sinn, die Parameterwerte so festzulegen, dass durchschnittlich mindestens 25 Startlösungen erstellt werden. Im Zuge dieser Masterarbeit wird die vorgestellte PRG-Heuristik jedoch als Startlösungsheuristik verwendet, weshalb der Parameter b für die finale Parameteranpassung nach oben hin mit einem Wert von 5 beschränkt wird. Hierbei liegen die durchschnittlich maximal erstellten Startlösungen nach aufsteigenden Instanzengrößen des Parameter-Testbeds bei 15, 13.5, 20.5 und 14.5. Dadurch dass der festgelegte Wert von a nicht zu hoch angesetzt wird, können in den folgenden Parameteranpassungen der Metaheuristiken unter Einbezug der gesamten Laufzeit aller verbesserten Startlösungen z. B. die Iterations-Parameter $iterTS$ und $iterGA$ höher angesetzt werden.

n	Parameter	$\bar{x}_{geom}$	LZ[s]	$\varnothing a$	$\bar{x}_{geom}$	LZ[s]	$\varnothing a$	$\bar{x}_{geom}$	LZ[s]	$\varnothing a$	$\bar{x}_{geom}$	LZ[s]	$\varnothing a$
		$b = 5$			$b = 10$			$b = 15$			$b = 20$		
200	$\bar{a} = 1$	-	-	-	-	-	-	-	-	-	92.10 %	1.39	1
	$\bar{a} = 5$	92.68 %	2.65	2.5	93.22 %	5.34	4	93.39 %	3.98	4.5	93.91 %	5.80	4.5
	$\bar{a} = 10$	93.39 %	6.35	5	94.08 %	8.87	7.5	94.08 %	10.12	8.5	94.26 %	9.52	8.5
	$\bar{a} = 15$	93.91 %	8.07	7.5	93.39 %	10.95	11	93.56 %	12.48	12.5	94.08 %	12.57	13
	$\bar{a} = 20$	94.79 %	9.59	10	94.08 %	13.18	14.5	94.08 %	14.43	16	94.08 %	14.76	17
	$\bar{a} = 25$	93.91 %	11.61	12.5	94.08 %	14.81	18	93.91 %	15.79	20	93.91 %	16.39	21
	$\bar{a} = 30$	94.61 %	12.56	15	93.91 %	16.41	21.5	94.43 %	19.03	24.5	96.59 %	24.38	25.2
400	$\bar{a} = 1$	-	-	-	-	-	-	-	-	-	91.58 %	13.41	1
	$\bar{a} = 5$	92.88 %	19.81	2.5	92.39 %	19.07	3.5	92.64 %	23.64	4.5	92.72 %	26.17	4.5
	$\bar{a} = 10$	92.64 %	24.88	5	92.89 %	36.09	7	92.56 %	34.87	8	92.64 %	40.22	8.5
	$\bar{a} = 15$	94.25 %	27.68	7	93.22 %	43.83	10.5	93.21 %	52.80	12	94.79 %	56.86	12.5
	$\bar{a} = 20$	92.97 %	36.47	9	93.98 %	56.11	13.5	94.30 %	63.93	15.5	93.38 %	71.07	17
	$\bar{a} = 25$	93.46 %	43.37	11.5	93.71 %	65.91	17	93.79 %	72.40	19.5	94.21 %	75.11	20.5
	$\bar{a} = 30$	93.05 %	44.94	13.5	93.21 %	73.77	20.5	93.71 %	83.67	23	95.64 %	114.94	24.2
600	$\bar{a} = 1$	-	-	-	-	-	-	-	-	-	90.37 %	4.77	1
	$\bar{a} = 5$	90.82 %	17.99	4	92.10 %	13.66	4.5	91.60 %	14.26	5	91.74 %	14.87	5
	$\bar{a} = 10$	92.04 %	28.60	7.5	91.53 %	26.85	8.5	91.71 %	26.41	9.5	92.23 %	29.05	9.5
	$\bar{a} = 15$	91.93 %	36.21	10.5	92.49 %	35.95	12.5	92.38 %	44.00	13.5	92.16 %	43.71	14
	$\bar{a} = 20$	92.10 %	49.91	14	92.44 %	54.63	16.5	92.32 %	55.75	18	92.21 %	59.28	18.5
	$\bar{a} = 25$	91.93 %	60.32	17.5	92.10 %	61.84	21	92.10 %	62.30	22.5	92.32 %	67.81	23
	$\bar{a} = 30$	91.93 %	60.32	20.5	92.32 %	67.01	24.5	92.44 %	73.34	26.5	95.28 %	80.73	27.8
800	$\bar{a} = 1$	-	-	-	-	-	-	-	-	-	89.81 %	12.45	1
	$\bar{a} = 5$	90.66 %	25.19	3	91.21 %	28.02	4	90.95 %	31.64	4.5	90.84 %	37.53	5
	$\bar{a} = 10$	91.18 %	37.15	5.5	90.65 %	49.95	7.5	90.84 %	54.54	8.5	90.78 %	59.31	9
	$\bar{a} = 15$	90.95 %	46.74	7.5	90.92 %	65.19	10.5	90.94 %	79.29	12.5	90.87 %	80.20	13
	$\bar{a} = 20$	91.56 %	60.86	10	91.13 %	83.05	14	90.95 %	96.96	16	90.90 %	106.39	17
	$\bar{a} = 25$	90.66 %	69.75	12	91.25 %	97.48	17.5	91.74 %	107.11	20	91.94 %	130.43	21.5
	$\bar{a} = 30$	91.06 %	76.63	14.5	91.36 %	114.03	21	91.89 %	152.22	24	94.75 %	133.25	25.5

Tabelle 8.2: PRG - Vortest der Parameter $\bar{a}$ und b bei $k = 50$

Bei der finalen Parameteranpassung von a werden die Parameter $\bar{a}$ und b erneut getestet und überprüft, ob ohne Verlust der Qualität der Ergebnisse der Parameter b kleiner als 5 angesetzt werden kann (siehe Tabelle 8.3). Es zeigt sich, dass für alle Instanzengrößen gute Ergebnisse bei einer Kombination von $\bar{a} = 20$ und $b = 4$ erzielt werden können und demnach die Parameter fortan auf diese Werte festgesetzt werden.

n	Parameter	$\bar{x}_{geom}$	LZ[s]	$\varnothing a$	$\bar{x}_{geom}$	LZ[s]	$\varnothing a$	$\bar{x}_{geom}$	LZ[s]	$\varnothing a$	$\bar{x}_{geom}$	LZ[s]	$\varnothing a$
		$b = 2$			$b = 3$			$b = 4$			$b = 5$		
200	$\bar{a} = 1$	-	-	-	-	-	-	-	-	-	92.98 %	0.20	1
	$\bar{a} = 5$	94.47 %	0.21	1.5	94.94 %	0.45	2.5	94.42 %	0.44	2.5	94.42 %	0.47	2.5
	$\bar{a} = 10$	94.51 %	0.59	2.5	94.94 %	0.86	4	95.37 %	1.14	5	94.64 %	1.13	5
	$\bar{a} = 15$	94.93 %	0.39	3.5	95.06 %	1.41	5.5	94.85 %	1.63	6.5	94.96 %	2.18	7.5
	$\bar{a} = 20$	94.52 %	1.02	4.5	95.17 %	1.69	7	95.38 %	2.47	9	95.38 %	2.88	10
	$\bar{a} = 25$	94.67 %	0.51	5	95.06 %	2.10	8.5	95.27 %	2.93	10.5	95.59 %	3.47	12.5
	$\bar{a} = 30$	95.16 %	1.36	6.5	94.75 %	2.46	10	95.17 %	3.08	13	95.37 %	3.57	15
400	$\bar{a} = 1$	-	-	-	-	-	-	-	-	-	92.25 %	1.53	1
	$\bar{a} = 5$	93.39 %	1.57	1.5	93.38 %	1.68	1.5	93.43 %	3.60	2.5	93.13 %	3.98	2.5
	$\bar{a} = 10$	93.03 %	1.60	2	93.08 %	4.40	3	93.28 %	5.52	4	93.23 %	5.91	5
	$\bar{a} = 15$	93.43 %	1.61	2.5	93.87 %	5.39	4.5	93.68 %	7.13	6	93.33 %	7.67	7
	$\bar{a} = 20$	93.03 %	4.47	3.5	93.72 %	6.76	6	93.68 %	7.28	7.5	93.48 %	9.34	9
	$\bar{a} = 25$	93.43 %	4.47	4	93.72 %	6.18	7	93.63 %	10.93	9.5	93.33 %	10.21	11.5
	$\bar{a} = 30$	93.58 %	4.14	4.5	93.62 %	7.30	8.5	93.82 %	9.64	11.5	93.73 %	12.35	13.5
600	$\bar{a} = 1$	-	-	-	-	-	-	-	-	-	91.52 %	0.90	1
	$\bar{a} = 5$	92.19 %	2.64	2.5	91.99 %	2.90	3	92.09 %	3.33	3.5	92.12 %	4.65	4
	$\bar{a} = 10$	92.19 %	4.19	4.5	91.95 %	6.03	5.5	91.89 %	5.92	6.5	92.19 %	6.34	7.5
	$\bar{a} = 15$	92.30 %	5.26	6.5	92.39 %	7.20	8.5	92.43 %	8.71	9.5	92.22 %	8.58	10.5
	$\bar{a} = 20$	92.46 %	6.50	8.5	92.29 %	9.01	10.5	92.22 %	9.98	13	92.12 %	10.68	14
	$\bar{a} = 25$	92.29 %	8.10	10.5	92.43 %	9.35	13.5	92.53 %	12.47	15.5	92.49 %	11.19	17.5
	$\bar{a} = 30$	92.46 %	8.78	12.5	92.63 %	10.47	16	92.73 %	11.69	18.5	92.83 %	13.14	20.5
800	$\bar{a} = 1$	-	-	-	-	-	-	-	-	-	90.59 %	2.11	1
	$\bar{a} = 5$	90.07 %	2.46	1	90.56 %	4.00	2	90.82 %	4.76	2.5	90.80 %	6.55	3
	$\bar{a} = 10$	90.77 %	4.26	2	90.84 %	6.80	3.5	91.39 %	7.70	4.5	91.18 %	9.05	5.5
	$\bar{a} = 15$	90.59 %	4.57	2.5	91.44 %	7.70	4.5	91.14 %	12.98	6.5	91.13 %	13.67	7.5
	$\bar{a} = 20$	91.04 %	6.24	3.5	91.31 %	11.03	6	91.20 %	13.28	8	91.21 %	15.79	10
	$\bar{a} = 25$	91.09 %	6.77	4	91.25 %	11.53	7.5	91.29 %	17.81	10	91.26 %	17.33	12
	$\bar{a} = 30$	91.13 %	7.80	5	91.25 %	12.57	8.5	91.16 %	17.99	12.5	91.24 %	21.23	14.5

Tabelle 8.3: PRG - Parameteranpassung des Parameters $a(iterC)$ bei $k = 50$

Der Tabelle 8.4 kann man die Ergebnisse der offline Parameteranpassung des Parameters k entnehmen. Es werden die Werte des Parameters k in einem Wertebereich von $1 \leq k \leq 100$ in Kombination mit den bereits festgelegten Parameterwerten des Parameters a getestet. Die farblich markierten Zellen der Tabelle bilden die optimalen Parameterwerte ab. Es zeigt sich, dass eine Erhöhung des Parameters k zu einer Performanceverbesserung führt, jedoch ein zu stark zufallsbasiertes Auswahlverfahren den gegenteiligen Effekt erzielt. Daher wird der Parameter k für alle Instanzengrößen auf einen Wert von 10 fixiert, da hier die maximalen geometrischen Mittelwerte erzielt werden konnten.

	$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]
	$n = 200$		$n = 400$		$n = 600$		$n = 800$	
$k = 1$	93.91 %	0.08	92.64 %	0.68	91.82 %	0.54	89.78 %	1.42
$k = 10$	95.30 %	2.86	93.73 %	8.45	92.51 %	9.76	91.30 %	14.25
$k = 20$	94.93 %	3.49	93.51 %	11.40	92.50 %	12.54	91.21 %	19.14
$k = 30$	94.43 %	4.10	93.66 %	14.24	92.43 %	15.12	91.29 %	24.56
$k = 40$	94.20 %	5.12	93.17 %	19.04	92.21 %	24.63	90.93 %	29.01
$k = 50$	93.91 %	5.29	93.01 %	19.75	92.24 %	28.44	90.92 %	32.98
$k = 60$	94.02 %	5.94	93.01 %	22.83	92.22 %	31.83	90.80 %	37.56
$k = 70$	93.44 %	6.68	92.79 %	25.55	92.05 %	30.04	90.81 %	43.18
$k = 80$	93.60 %	7.47	92.71 %	29.91	91.61 %	28.95	90.72 %	48.44
$k = 90$	93.11 %	8.06	92.59 %	30.86	91.54 %	29.69	90.75 %	51.24
$k = 100$	93.01 %	8.49	92.48 %	32.73	91.31 %	31.87	90.47 %	54.96

Tabelle 8.4: PRG - Parameteranpassung des Parameters k

Mit den festgelegten Werten der Parameter werden die leichten (LI) und schweren Testinstanzen (SI) getestet und die Ergebnisse in der Tabelle 8.5 zusammengefasst. Es lässt sich erkennen, dass das Erstellen mehrerer Startlösungen (Multi-PRG) kombiniert mit einer Randomisierungskomponente ($k > 1$) einen positiven Effekt auf die Performance der PRG-Heuristik hat. Eine Testung der LI ergibt, dass die Ergebnisse der PRG-Heuristik sehr nahe an der Referenzheuristik liegen. Aus der Testung der SI kann man herauslesen, dass die Ergebnisse für die kleinste Instanzengröße von $n = 200$ bereits durchschnittlich 91 % der Optimallösung ausmacht. Die Güte der Ergebnisse sinkt jedoch mit steigender Inputgröße n und liegt bei der größten Instanzengröße nur mehr bei durchschnittlich 82 %.

Instanzen	Programm	$\bar{x}_{\text{geom}}$	LZ[s]	$\bar{x}_{\text{geom}}$	LZ[s]	$\bar{x}_{\text{geom}}$	LZ[s]	$\bar{x}_{\text{geom}}$	LZ[s]
		$n = 200$		$n = 400$		$n = 600$		$n = 800$	
LI	Single-PRG	98.27 %	0.29	98.76 %	1.78	99.00 %	4.51	99.15 %	10.55
	Multi-PRG	99.04 %	2.00	99.30 %	11.47	99.38 %	32.19	99.46 %	78.46
SI	Single-PRG	88.32 %	0.03	84.16 %	0.15	82.68 %	0.54	80.69 %	1.50
	Multi-PRG	91.28 %	0.36	87.06 %	2.43	84.46 %	5.68	82.45 %	9.98

Tabelle 8.5: Single-PRG und Multi-PRG - Performance-Evaluierung

8.3 Paarweise randomisierte Greedy-Heuristik mit Tabu Suche

Wie bereits bei der Auswertung der PRG-Heuristik werden auch bei der PRG+TS Verbesserungsheuristik in einem ersten Schritt die optimalen Werte der Parameter anhand der Testinstanzen des Parameter-Testbeds und anschließend die finale Datenauswertung für alle Testinstanzen vorgenommen. Die zu bestimmenden Parameter sind die Anzahl der gesamten Iterationen der TS $iterTS$, die Größe der Nachbarschaftssuche p_N und die Länge der Tabu Liste t .

Trotz der probabilistischen TS der PRG+TS Heuristik und demnach des Vorliegens einer reduzierten Nachbarschaftssuche haben erste Vortests der Parameter gezeigt, dass das Programm im Vergleich zu den anderen Heuristiken dieser Masterarbeit sehr rechenaufwendig ist. Die gesamte Rechendauer der PRG+TS Verbesserungsheuristik wird nicht nur durch den Durchlauf einer einzigen TS bestimmt, sondern auch maßgeblich durch die Anzahl der Startlösungen, auf welche die TS jeweils angewandt wird. Daher wird im Gegensatz zu den anderen Programmen in dieser Arbeit der Parameter $\bar{a}$ bei der PRG+TS Heuristik auf einen Wert von 10 reduziert. Dadurch können die Iterationen der TS höher angesetzt werden und das Programm dennoch in einer vertretbaren Laufzeit berechnet werden.

In der Tabelle 8.6 werden die Ergebnisse des Parametertestings der Parameter $iterTS$ und p_N dargestellt. Da sich beide Parameter maßgeblich auf die Laufzeit des Algorithmus auswirken und den Umfang der Nachbarschaftssuche beeinflussen, werden diese beiden Parameter in Kombination getestet. Wie bereits in Kapitel 5.2.1 dargestellt, werden in jeder Iteration der TS N_{size} Nachbarschaftsschritte untersucht und die Gesamtanzahl der Iterationen durch $iterTS$ bestimmt. Die Variable N_{size} wird aus der Multiplikation von n mit dem Parameter p_N bestimmt. Daher werden in einem Durchlauf der gesamten TS höchstens $N_{size} \cdot iterTS$ Nachbarschaftsschritte untersucht.

Der ursprüngliche Gedanke bei der Implementierung einer reduzierten Nachbarschaft war jener eine vollständige Nachbarschaftssuche, um mindestens die Hälfte zu reduzieren und dadurch die Iterationen der TS höher ansetzen zu können. Dementsprechend werden für den Parameter p_N die Werte 0.25 und 0.5 geprüft. Die Werte des Parameters $iterTS$ werden in 100er-Schritten überprüft und mit dem Wert 1500 nach oben hin beschränkt. Für die Berechnung dieser Parameterkombinationen wird vorerst der Parameter t auf 10 festgelegt, dessen optimale Werte dann in einer eigenen Parameteranpassung bestimmt werden. Bei dieser Kombination aus zu testenden Parameterwerten und durch die oberen Schranken der Parameter befindet sich die durchschnittliche Laufzeit für die größte Instanzengruppe bei ungefähr 1200 Sekunden und somit 20 Minuten.

Eine Evaluierung der Ergebnisse aus Tabelle 8.6 zeigt, dass durch das Erhöhen des Parameters $iterTS$ bei allen Instanzengrößen eine Verbesserung der Ergebnisse erzielt werden kann. Bei einem Wert des Parameters p_N von 0.5 können die Ergebnisse jedoch im Vergleich zu einem Wert von 0.25 nicht maßgeblich verbessert werden, obwohl die Laufzeit durchschnittlich doppelt so groß ist. Daher wird für den Parameter p_N der Wert von 0.25 für alle Instanzengrößen fixiert. Für den Parameter $iterTS$ wird der Wert 1300 ausgewählt, da hier durchschnittlich sehr gute Ergebnisse erzielt werden können, und die maximale Laufzeit bei der Kombination beider Parameter bei ungefähr 466 Sekunden und somit

maximal 8 Minuten pro Instanz liegt. Ein geometrischer Durchschnitt von über 100 % bei der Instanzengröße von $n = 200$ deutet darauf hin, dass die erzielten Ergebnisse der leichten Testinstanzen die Ergebnisse der Vergleichsheuristik übertreffen.

n	Parameter	$\bar{x}_{\text{geom}}$	LZ[s]	$\bar{x}_{\text{geom}}$	LZ[s]
		$p_N = 0.25$		$p_N = 0.5$	
200	$iterTS = 100$	98.88 %	1.07	99.43 %	1.84
	$iterTS = 300$	101.18 %	3.14	101.10 %	6.33
	$iterTS = 500$	101.21 %	4.18	100.49 %	9.40
	$iterTS = 700$	101.44 %	5.91	100.53 %	12.33
	$iterTS = 900$	100.42 %	8.32	100.52 %	17.44
	$iterTS = 1100$	100.21 %	8.57	100.85 %	16.42
	$iterTS = 1300$	100.34 %	12.71	101.26 %	26.85
	$iterTS = 1500$	100.48 %	11.76	103.28 %	32.37
400	$iterTS = 100$	93.46 %	4.38	93.46 %	9.65
	$iterTS = 300$	93.21 %	13.46	96.04 %	26.82
	$iterTS = 500$	93.71 %	22.07	94.04 %	42.89
	$iterTS = 700$	93.96 %	30.28	95.88 %	60.45
	$iterTS = 900$	94.45 %	41.31	96.20 %	76.56
	$iterTS = 1100$	94.37 %	47.04	93.71 %	94.32
	$iterTS = 1300$	93.71 %	57.20	94.54 %	117.83
	$iterTS = 1500$	94.62 %	69.67	94.20 %	139.92
600	$iterTS = 100$	92.60 %	8.52	92.44 %	16.13
	$iterTS = 300$	92.77 %	23.71	92.66 %	47.63
	$iterTS = 500$	92.44 %	40.34	92.78 %	79.67
	$iterTS = 700$	92.55 %	55.81	92.38 %	111.74
	$iterTS = 900$	93.06 %	72.91	93.28 %	143.54
	$iterTS = 1100$	92.83 %	87.50	93.00 %	174.20
	$iterTS = 1300$	95.30 %	102.65	93.17 %	203.29
	$iterTS = 1500$	93.06 %	127.98	93.11 %	276.92
800	$iterTS = 100$	90.87 %	39.62	91.38 %	81.93
	$iterTS = 300$	96.34 %	108.49	91.52 %	220.06
	$iterTS = 500$	91.66 %	180.58	91.67 %	364.22
	$iterTS = 700$	91.73 %	254.53	91.63 %	520.46
	$iterTS = 900$	91.80 %	332.41	92.12 %	657.11
	$iterTS = 1100$	94.59 %	394.15	91.57 %	785.57
	$iterTS = 1300$	94.51 %	466.06	91.91 %	937.20
	$iterTS = 1500$	94.65 %	582.12	91.94 %	1212.83

Tabelle 8.6: PRG+TS - Parameteranpassung der Parameter $iterTS$ und p_N bei $t = 10$

Ein Vortest des Parameters t hat gezeigt, dass bei einer Testung des Parametertestbeds kein eindeutiges Trendverhalten aus den Ergebnissen herausgelesen werden kann. Daher wird für die Testung des Parameters t das Parameter-Testbed um 4 weitere Datensätze der LI und 4 weitere Datensätze der SI ergänzt und besteht demnach aus 16 Datensätzen. Der Parameter wird in 10er-Schritten getestet und nach oben hin mit einem Wert von 100 beschränkt. Darüber hinaus hat der Vortest gezeigt, dass sich ein Wert von über 100 nicht mehr positiv auf die Ergebnisse auswirkt. Zusätzlich wird auch eine rein lokale Suche überprüft, indem der Wert von t auf 0 festgelegt wird.

Die Daten der Tabelle 8.7 zeigen einen Trend dahingehend, dass durch eine Erhöhung von t die erzielten ZF-Werte marginal verbessert werden können, bei einem zu hohen Wert von t sich die Ergebnisse jedoch wieder verschlechtern. Bei einem Wert von $t = 50$ werden für alle Instanzengrößen gute

Ergebnisse erzielt und darüber hinaus die Laufzeit reduziert, da im Vergleich zu einem niedrigeren Parameterwert seltener Nachbarschaftsschritte vorgenommen werden.

	$\bar{x}_{\text{geom}}$	LZ[s]	$\bar{x}_{\text{geom}}$	LZ[s]	$\bar{x}_{\text{geom}}$	LZ[s]	$\bar{x}_{\text{geom}}$	LZ[s]
	$n = 200$		$n = 400$		$n = 600$		$n = 800$	
$t = 0$	100.82 %	15.23	94.67 %	72.78	93.19 %	136.17	91.78 %	543.03
$t = 5$	100.40 %	11.49	94.82 %	65.06	93.39 %	136.97	91.77 %	501.21
$t = 10$	100.12 %	12.07	94.70 %	64.24	94.36 %	136.35	92.01 %	544.25
$t = 20$	100.50 %	10.72	95.48 %	38.60	94.54 %	123.67	92.90 %	505.93
$t = 30$	100.54 %	10.14	95.50 %	44.23	93.34 %	70.93	93.09 %	346.14
$t = 40$	100.25 %	8.23	94.93 %	46.15	93.55 %	73.10	93.84 %	391.01
$t = 50$	100.05 %	8.49	96.61 %	41.24	93.77 %	67.99	92.04 %	331.18
$t = 60$	99.98 %	6.60	93.86 %	35.36	93.23 %	47.46	93.68 %	310.70
$t = 70$	100.40 %	5.32	94.70 %	37.47	93.53 %	53.12	92.75 %	357.36
$t = 80$	100.65 %	5.15	95.04 %	41.40	94.86 %	58.81	91.80 %	226.59
$t = 90$	100.37 %	4.74	94.12 %	43.50	93.76 %	56.65	92.21 %	225.91
$t = 100$	100.59 %	4.23	94.67 %	39.01	93.15 %	54.37	91.81 %	200.60

Tabelle 8.7: PRG+TS - Parameteranpassung des Parameters t

In der Tabelle 8.8 werden die Ergebnisse der finalen Datenauswertung der PRGT-TS Verbesserungsheuristik dargestellt. Es lässt sich erkennen, dass die PRG+TS Heuristik die Ergebnisse der Referenzheuristik der LI in allen Instanzengrößen dominiert. Bei den SI ist auffällig, dass bei der Instanzengröße von $n = 400$ durchschnittlich besonders gute ZF-Werte erzielt werden können und die erzielten Ergebnisse der Multi-PRG Heuristik um ungefähr 7 Prozentpunkte verbessert werden. Bei einer Instanzengröße von $n = 800$ der SI wird jedoch nur eine nicht nennenswerte Verbesserung erzielt.

		$\bar{x}_{\text{geom}}$	LZ[s]	$\bar{x}_{\text{geom}}$	LZ[s]	$\bar{x}_{\text{geom}}$	LZ[s]	$\bar{x}_{\text{geom}}$	LZ[s]
Instanzen	Programm	$n = 200$		$n = 400$		$n = 600$		$n = 800$	
LI	PRG+TS	102.30 %	9.10	101.63 %	67.00	101.57 %	183.95	100.82 %	462.82
SI	PRG+TS	93.25 %	3.80	94.22 %	22.92	85.22 %	80.43	82.99 %	180.84

Tabelle 8.8: PRG+TS - Performance-Evaluierung

8.4 Paarweise randomisierte Greedy-Heuristik mit Genetischem Algorithmus

Bei der PRG+GA Heuristik werden die optimalen Werte des Parameters $iterGA$ durch eine Funktion bestimmt und der Parameter p_C im Zuge einer offline Parameteranpassung definiert. Die Werte für die Mutations- und Crossover-Rate p_m und p_{cr} orientieren sich an den in der Literatur vorgeschlagenen Wertebereichen und werden iterativ und online an das Maß der Konvergenz einer Generation angepasst.

Da für die Bewertung der Ergebnisse als Referenzheuristik der Genetische Algorithmus von Julstrom [2005] herangezogen wird, orientieren sich die Vortests des Parameters $iterGA$ an der von Julstrom verwendeten Methode und somit einer Anzahl $10n$ Generationen. Es zeigt sich jedoch, dass der für diese Arbeit zur Verfügung stehende Computer und die programmiertechnische Implementierung der PRG+GA Heuristik nicht in der Lage sind, den Parameter $iterGA$ in dieser Form zu bestimmen, da die Datensätze nicht in einer vertretbaren Zeit berechnet werden können. Deshalb werden bei der PRG+GA Metaheuristik höchstens $10n$ Iteration vorgenommen, jedoch die Lösungssuche frühzeitig unterbrochen nach dem Erreichen von $0.5iterGA$ aufeinanderfolgende Iterationen, in denen der ZF-Wert nicht mehr verbessert wurde. Zusätzlich dazu wird bei der Parameteranpassung von $iterGA$ überprüft, ob auch bei einer maximalen Anzahl von $5n$ Generationen vergleichbare Ergebnisse erzielt werden können.

Anhand der Tabelle 8.9 kann man jedoch erkennen, dass vor allem bei den Instanzen des Parameter-Testbeds der Größe $n = 400$ und $n = 600$ eindeutig bessere Ergebnisse durch die Funktion $iterGA = 10n$ generiert werden. Bei den größten Testinstanzen fällt die durchschnittliche Laufzeit von 800 Sekunden und daher rund 13 Minuten negativ ins Gewicht. Hinsichtlich der Optimierung des ZF-Wertes und einer Vergleichbarkeit mit dem Genetischen Algorithmus von Julstrom wird der Parameter $iterGA$ trotz schlechter Laufzeit-Performance für alle Instanzengrößen auf einen Wert von $10n$ fixiert.

	$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]
	$n = 200$		$n = 400$		$n = 600$		$n = 800$	
$iterGA = 10n$	95.78 %	11.29	95.23 %	59.38	94.07 %	359.03	91.68 %	804.97
$iterGA = 5n$	95.33 %	7.93	93.38 %	56.85	92.29 %	217.62	91.71 %	456.21

Tabelle 8.9: PRG+GA - Parameteranpassung des Parameters $iterGA$

Anders als bei der PRG+TS und der PRG+SA Verbesserungsheuristik wird der PRG+GA Algorithmus nicht auf alle Startlösungen angewandt, sondern die durch die Multi-PRG Heuristik erstellten Startlösungen werden für die Startpopulation P_0 herangezogen. Aufgrund dessen werden bei der Testung für den Parameter $\bar{a}$ nicht nur der im Kapitel 8.2 ermittelte Wert verwendet, sondern auch überprüft, ob eine größere oder kleinere Population einen positiven Effekt auf die Ergebnisse hat. Wie bereits aus der Tabelle 8.1 ersichtlich, muss der Parameter nb_C kleiner als die Anzahl der Individuen einer Population sein, da gemäß des Algorithmus auch nb_C Individuen der aktuellen Generation durch

die Nachkommen ersetzt werden. Die Größe der Population wird durch den Parameter a dargestellt und in Abhängigkeit der Variable $iterC$ definiert (siehe Formel (8.2)). Dementsprechend bestimmt sich die Anzahl der zu erstellenden Nachkommen aus einem ganzzahligen und prozentualen Anteil des Parameters $a(iterC)$ und wird in der Formel (8.3) abgebildet. Der prozentuale Anteil wird über den Prozentsatz p_C gesteuert, dessen optimaler Wert durch Einsatz einer Parametervariation in Kombination mit Variationen des Parameters a bestimmt wird (siehe Tabelle 8.10).

$$nb_C(a(iterC)) = a(iterC) p_C \quad (8.3)$$

Da für die Erstellung einer neuen Generation mindestens zwei Individuen zur Verfügung stehen müssen, welche als Elternteile herangezogen werden, muss der Fall abgefangen werden, dass $a(iterC)$ einen Wert von 1 annimmt. Sollte dies eintreffen, so wird der Wert für a auf 2 erhöht. Der Parameter p_C wird stets auf die nächstgrößte natürliche Zahl aufgerundet, falls das Ergebnis der Berechnung eine rationale Zahl ist.

Die Auswertung von der Parameteranpassung des Parameters p_C in der Tabelle 8.10 zeigt, dass bei einem Parameterwert von 0.5 für alle Instanzengrößen sehr gute Ergebnisse erzielt werden können. Dementsprechend werden fortan in jeder Generation maximal halb so viele Nachkommen wie Individuen erstellt. Die tatsächliche Anzahl der zu erstellenden Nachkommen wird zusätzlich zu dem Parameter p_C über die Crossover-Rate p_{cr} beeinflusst.

n	Parameter	$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]
		$p_C = 0.25$		$p_C = 0.5$		$p_C = 0.75$	
200	$\bar{a}=10$	94.98 %	5.60	95.74 %	5.67	94.77 %	7.22
	$\bar{a}=20$	93.25 %	5.06	94.29 %	7.00	92.97 %	7.57
	$\bar{a}=30$	95.30 %	8.46	95.85 %	9.15	95.17 %	10.56
	$\bar{a}=40$	95.54 %	14.04	95.76 %	10.85	95.79 %	16.68
400	$\bar{a}=10$	93.50 %	28.41	93.44 %	32.87	93.21 %	36.73
	$\bar{a}=20$	91.24 %	37.26	91.16 %	35.14	90.85 %	48.10
	$\bar{a}=30$	95.68 %	54.12	93.46 %	64.66	95.88 %	64.37
	$\bar{a}=40$	93.79 %	69.62	94.04 %	62.39	94.12 %	84.62
600	$\bar{a}=10$	92.53 %	143.42	94.67 %	191.06	92.45 %	198.34
	$\bar{a}=20$	89.92 %	182.53	92.83 %	270.32	89.81 %	281.38
	$\bar{a}=30$	93.39 %	255.53	93.46 %	347.02	93.00 %	406.32
	$\bar{a}=40$	95.06 %	380.46	92.38 %	420.16	92.49 %	460.25
800	$\bar{a}=10$	91.41 %	284.99	93.35 %	347.86	90.95 %	396.48
	$\bar{a}=20$	88.38 %	334.04	91.04 %	436.84	87.76 %	491.72
	$\bar{a}=30$	94.44 %	590.61	91.78 %	669.21	91.53 %	675.64
	$\bar{a}=40$	97.18 %	682.02	97.10 %	606.24	91.30 %	862.36

Tabelle 8.10: PRG+GA - Parameteranpassung des Parameters p_C

Die Parameterwerte der Parameter p_{cr} und p_m werden in jeder Generation an das Maß der Konvergenz der Population angepasst. Im folgenden Pseudocode wird gezeigt, wie diese iterative Aktualisierung der Parameterwerte erfolgt. Zur Bestimmung der Konvergenz wird die Variable $Equal_p$ herangezogen, welche in jeder Generation im Anschluss an das Ausführen der genetischen Operatoren Selektion, Re-

kombination und Mutation berechnet wird. Hierfür wird überprüft, wie oft prozentual eine eindeutige Lösung unter allen Individuen einer Generation vorkommt. Ein Wert von 1 der Variable $Equal_P$ weist darauf hin, dass die Lösungsvektoren aller Individuen ident sind. Die Werte der Tabelle 8.11 orientieren sich an den in der Literatur erprobten und vorgeschlagenen Wertebereichen der Mutations- und Crossover-Rate [Hesser u. Männer, 1991].

Konvergenz	p_m	p_{cr}
$Equal_P \leq 0.5$	0.001	0.5
$0.5 \leq Equal_P < 0.6$	0.01	0.6
$0.6 \leq Equal_P < 0.7$	0.02	0.7
$0.7 \leq Equal_P < 0.8$	0.03	0.8
$0.8 \leq Equal_P < 0.9$	0.04	0.9
$0.9 \leq Equal_P \leq 1$	0.05	1

Tabelle 8.11: PRG+GA - Parameterwerte der Parameter p_m und p_{cr}

Eine finale Datenauswertung der PRG+GA Metaheuristik erfolgt sowohl für die Single-Point-Crossover-Methode als auch für die Two-Point-Crossover-Methode. Bei beiden Verfahren werden die Crossover-Punkte auf Basis einer gleichverteilten Zufallszahl bestimmt. Aus der Tabelle 8.12 ist ersichtlich, dass die Ergebnisse bei den LI sowohl beim Single-Point-Crossover Verfahren also auch beim Multi-Point-Crossover Verfahren über alle Instanzengrößen konstant sind. Bei den SI jedoch variieren die Ergebnisse stärker. Das Single-Point-Crossover Verfahren dominiert bei einer Instanzengröße von $n = 600$ wohingegen das Two-Point-Crossover bei einer Instanzengröße von $n = 400$ eindeutig bessere Ergebnisse erzielt. Bei den kleinsten und den größten Instanzen der SI können jedoch unabhängig von der Crossover-Methode kaum Verbesserungen gegenüber der Startlösungsheuristik erzielt werden.

Instanzen	Programm	$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]
		$n = 200$		$n = 400$		$n = 600$		$n = 800$	
LI	PRG+GA	99.47 %	18.27	99.53 %	68.85	99.62 %	255.16	99.66 %	505.55
	(Single-Point-Crossover)								
	PRG+GA	99.56 %	23.29	99.69 %	83.73	99.70 %	272.03	99.79 %	663.69
	(Two-Point-Crossover)								
SI	PRG+GA	92.57 %	54.46	87.48 %	177.36	88.21 %	739.14	82.83 %	1094.80
	(Single-Point-Crossover)								
	PRG+GA	92.90 %	37.41	90.22 %	130.37	85.80 %	501.69	82.99 %	820.63
	(Two-Point-Crossover)								

Tabelle 8.12: PRG+GA - Performance-Evaluierung

8.5 Paarweise randomisierte Greedy-Heuristik mit Simulated Annealing Verfahren

Ein Vortest der Parameteranpassung bezüglich der Start- und Terminierungstemperatur hat gezeigt, dass die optimalen Werte beider Parameter stark von einer spezifischen Instanz abhängig sind. Daher wird bei der Berechnung einer Instanz eine offline Parameteranpassung besagter Parameter vor der Ausführung der Verbesserungsheuristik in einem Testlauf vorgenommen. Hierfür werden 500 zufällige Nachbarschaftsschritte nach dem Schema der PRG+SA Heuristik ausgeführt, die durchschnittlichen Zielfunktionsdifferenzen einer aktuellen Lösung und eines Nachbarschaftsschrittes ermittelt und durch Einsatz der Methode von Johnson u. a. [1989] (siehe Kapitel 7.1.1) die optimalen Parameterwerte bestimmt. Für die Berechnung der durchschnittlichen Zielfunktionsdifferenzen δ werden nur ZF-Wert verschlechternde Schritte berücksichtigt, da auch nur bei diesen die Prüfung des Metropolis-Kriteriums schlagend wird. Die Werte der Starttemperatur T_0 werden unter dem Gesichtspunkt festgelegt, dass zu Beginn der Metaheuristik die Nachbarschaftsschritte rein zufallsbasiert erfolgen sollen und demnach ein sogenannter Random Walk statt findet. Daher wird die gewünschte Akzeptanzwahrscheinlichkeit für die Bestimmung von T_0 auf $x_0 = 0.9999$ festgelegt und der Parameter durch die Division von δ mit dem natürlichen Logarithmus von x_0 ermittelt. Analog dazu wird die Terminierungstemperatur T_{BOUND} bestimmt. Da jedoch gegen Ende des iterativen Suchverlaufs keine ZF-Wert verschlechternde Schritte mehr zugelassen werden sollen, wird die gewünschte Akzeptanzwahrscheinlichkeit für die Bestimmung von T_{BOUND} auf $x_0 = 0.0001$ fixiert. Durch die Festlegung der Parameter nach dieser eben genannten Methode wird gewährleistet, dass in dem iterativen Prozess der PRG+SA Heuristik nicht unnötig viele rein zufallsbasierte Nachbarschaftsschritte untersucht werden, bevor sich die Temperatur zu reduzieren beginnt. Dadurch wird verhindert, dass die Suche frühzeitig aufgrund der Terminierungstemperatur beendet wird und attraktive Suchräume nicht erreicht werden können.

Im nächsten Schritt der Parameteranpassung wird die optimale Anzahl an Akzeptanzperioden, nach deren Erreichen eine Temperaturverringerung stattfindet und die Abkühlungsrate der geometrischen Abkühlungsmethode bestimmt. Beide Parameter nb_{acc} und α beeinflussen die Geschwindigkeit der Abkühlung, weswegen die Kombinationen beider Parameter getestet und die Ergebnisse in der Tabelle 8.13 zusammengefasst werden. Es lässt sich erkennen, dass sich das Erhöhen eines einzelnen Parameters nicht maßgeblich auf die Güte der Ergebnisse auswirkt, jedoch eine Kombination aus einem hohen Wert für nb_{acc} und α zu einer eindeutigen Verbesserung führt. Durch das Erhöhen beider Parameter nb_{acc} und α wird die Abkühlungsgeschwindigkeit verlangsamt. Bei der Festlegung der Parameter spielt jedoch auch die Laufzeit eine wichtige Rolle, welche bei einer Kombination von $nb_{acc} = 110$ und $\alpha = 0.9$ bei einer Instanzengröße von $n = 800$ mit ungefähr 9 Minuten Laufzeit sehr negativ ins Gewicht fällt. Daher werden unter dem Gesichtspunkt der Laufzeitoptimierung für die Parameter jene Werte ausgewählt, welche die zweitbeste Kombination aus nb_{acc} und α erzielen

und in der Tabelle 8.13 farblich markiert sind. Für die eben beschriebene Parametervariation wurde die Anzahl der maximal zu untersuchenden Nachbarschaftsschritte nb_{trans} innerhalb einer Iteration der aktuellen Temperatur mit dem Wert von 20.000 fixiert.

n	Parameter	$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]
		$\alpha=0.3$		$\alpha=0.5$		$\alpha=0.7$		$\alpha=0.9$	
200	$nb_{acc} = 10$	95.61 %	4.34	95.22 %	4.64	95.23 %	4.28	95.63 %	8.00
	$nb_{acc} = 30$	94.96 %	4.67	95.09 %	5.79	95.35 %	8.05	96.17 %	19.06
	$nb_{acc} = 50$	95.35 %	6.38	95.22 %	7.34	95.10 %	10.81	95.35 %	26.70
	$nb_{acc} = 70$	95.22 %	7.03	94.96 %	8.91	95.22 %	14.62	96.44 %	37.01
	$nb_{acc} = 90$	95.22 %	8.15	95.22 %	10.92	95.61 %	18.44	96.97 %	47.25
	$nb_{acc} = 110$	95.22 %	9.37	95.35 %	12.48	95.22 %	21.30	96.89 %	60.47
400	$nb_{acc} = 10$	93.63 %	9.12	93.50 %	8.85	94.06 %	10.94	93.44 %	19.51
	$nb_{acc} = 30$	93.38 %	10.94	93.56 %	13.69	93.75 %	20.95	93.32 %	52.93
	$nb_{acc} = 50$	93.81 %	14.25	93.38 %	18.01	95.08 %	29.47	94.25 %	73.31
	$nb_{acc} = 70$	93.69 %	18.74	93.44 %	22.73	93.94 %	39.45	95.59 %	104.01
	$nb_{acc} = 90$	93.38 %	19.49	94.84 %	28.07	93.63 %	50.95	95.32 %	130.16
	$nb_{acc} = 110$	93.50 %	22.15	93.38 %	30.73	94.27 %	58.04	95.93 %	171.91
600	$nb_{acc} = 10$	92.32 %	15.11	92.37 %	16.96	92.45 %	17.81	92.58 %	27.94
	$nb_{acc} = 30$	92.41 %	19.29	92.24 %	22.41	92.45 %	29.06	92.41 %	66.09
	$nb_{acc} = 50$	92.49 %	23.32	92.37 %	26.39	92.45 %	39.50	94.56 %	94.88
	$nb_{acc} = 70$	92.24 %	31.72	92.53 %	31.54	92.41 %	49.64	92.45 %	126.57
	$nb_{acc} = 90$	92.37 %	29.36	92.16 %	37.27	92.45 %	60.19	92.83 %	158.79
	$nb_{acc} = 110$	92.49 %	32.84	92.41 %	43.98	92.49 %	68.89	94.57 %	192.83
800	$nb_{acc} = 10$	91.43 %	20.73	91.14 %	23.08	91.33 %	27.04	91.20 %	52.11
	$nb_{acc} = 30$	90.95 %	28.80	91.32 %	35.17	91.22 %	51.48	91.29 %	134.72
	$nb_{acc} = 50$	90.96 %	38.47	90.99 %	45.85	91.01 %	81.94	92.75 %	248.94
	$nb_{acc} = 70$	91.11 %	44.93	91.22 %	58.33	91.14 %	109.38	93.14 %	315.72
	$nb_{acc} = 90$	90.97 %	52.08	91.25 %	75.98	91.49 %	146.60	91.98 %	396.79
	$nb_{acc} = 110$	91.27 %	63.19	91.07 %	85.26	91.10 %	163.57	95.72 %	548.60

Tabelle 8.13: PRG+SA - Parameteranpassung der Parameter nb_{acc} und α bei $nb_{trans} = 20.000$

Zum Schluss wird überprüft, ob das Erhöhen des Parameters nb_{trans} einen positiven Effekt auf die Lösungssuche hat, und die Ergebnisse werden in der Tabelle 8.14 zusammengefasst. Es lässt sich erkennen, dass die besten Ergebnisse bei einem Wert von $nb_{trans} = 40.000$ erzielt werden können und zusätzlich dazu durch ein Erhöhen des Parameters die Laufzeit reduziert wird. Der Umstand, dass die Laufzeit durch ein Erhöhen der zu untersuchenden Nachbarschaftsschritte gesenkt werden kann, mag auf den ersten Blick widersprüchlich erscheinen, lässt sich jedoch dadurch erklären, dass bei einem ausreichend hohen Wert von nb_{trans} genug Akzeptanzperioden nb_{acc} erreicht werden können und dadurch die Temperatur schneller abgekühlt wird. Die festgelegten Werte des Parameters sind erneut in der Tabelle farblich markiert.

	$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]
	$n = 200$		$n = 400$		$n = 600$		$n = 800$	
$nb_{trans} = 15.000$	96.84 %	60.35	93.88 %	120.9	92.28 %	112.81	91.52 %	364.44
$nb_{trans} = 20.000$	98.72 %	66.45	93.44 %	123	92.45 %	107.82	91.09 %	340.07
$nb_{trans} = 25.000$	96.59 %	54.36	95.44 %	118.11	92.20 %	104.31	91.22 %	343.75
$nb_{trans} = 30.000$	96.84 %	52.31	95.13 %	111.16	95.25 %	101.48	91.49 %	327.64
$nb_{trans} = 35.000$	96.01 %	54.68	95.20 %	112.64	92.37 %	100.86	91.45 %	330.54
$nb_{trans} = 40.000$	99.52 %	55.99	95.05 %	119.84	94.07 %	107.09	94.71 %	350.66
$nb_{trans} = 45.000$	99.52 %	61.93	95.36 %	118.57	92.62 %	100.33	91.63 %	335.27
$nb_{trans} = 50.000$	96.28 %	53.06	94.19 %	118.12	92.53 %	100.81	91.22 %	322.85

Tabelle 8.14: PRG+SA - Parameteranpassung des Parameters nb_{trans}

Mit den fixierten Parameterwerten wird zum Schluss die Metaheuristik anhand der LI und SI getestet und die Ergebnisse werden in der Tabelle 8.15 zusammengefasst. Durch den Einsatz der PRG+SA Verbesserungsheuristik auf die Startlösungen der Multi-PRG Heuristik kann eine eindeutige Verbesserung erzielt werden. Bei den LI liegen die erzielten Ergebnisse beinahe bei denen der Referenzheuristik für alle getesteten Instanzengrößen. Wie auch bei der Datenauswertung der anderen Heuristiken ist erneut zu erkennen, dass bei den SI die Güte der Ergebnisse mit steigendem Inputparameter n abnimmt. Jedoch werden bei der kleinsten Instanzengröße bereits durchschnittlich 96 % der Optimal-lösung erzielt.

		$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]	$\bar{x}_{geom}$	LZ[s]
Instanzen	Programm	$n = 200$		$n = 400$		$n = 600$		$n = 800$	
LI	PRG+SA	99.90 %	26.03	99.75 %	95.34	99.70 %	170.62	99.74 %	433.40
SI	PRG+SA	95.97 %	27.85	91.77 %	91.22	86.12 %	165.42	84.23 %	412.87

Tabelle 8.15: PRG+SA - Performance-Evaluierung

8.6 Performance-Vergleich (Meta-)Heuristiken

Im folgenden Kapitel wird einerseits ein Vergleich der Performance der vorgestellten Metaheuristiken zu den Ergebnissen der Startlösungsheuristik gezogen, und andererseits werden die Metaheuristiken untereinander bezüglich der Güte der Ergebnisse und der Laufzeit-Performance verglichen.

In der Tabelle 8.16 werden die Ergebnisse der finalen Datenauswertungen des vorangegangenen Kapitel zusammengefasst. Hinsichtlich der LI lässt sich zusammenfassen, dass die PRG+TS Metaheuristik eindeutig die beste Performance ausweist, da die Ergebnisse der Referenzheuristik übertroffen werden. Durch Einsatz der PRG+GA und der PRG+SA Metaheuristik werden die Ergebnisse der Multi-PRG Startlösungsheuristik nur insignifikant verbessert.

Instanzen	Programm	$\bar{x}_{\text{geom}}$	LZ[s]	$\bar{x}_{\text{geom}}$	LZ[s]	$\bar{x}_{\text{geom}}$	LZ[s]	$\bar{x}_{\text{geom}}$	LZ[s]
		$n = 200$		$n = 400$		$n = 600$		$n = 800$	
LI	Single-PRG	98.27 %	0.29	98.76 %	1.78	99.00 %	4.51	99.15 %	10.55
	Multi-PRG	99.04 %	2.00	99.30 %	11.47	99.38 %	32.19	99.46 %	78.46
	PRG+TS	102.30 %	9.10	101.63 %	67.00	101.57 %	183.95	100.82 %	462.82
	PRG+GA (Single-Point-Crossover)	99.47 %	18.27	99.53 %	68.85	99.62 %	255.16	99.66 %	505.55
	PRG+GA (Two-Point-Crossover)	99.56 %	23.29	99.69 %	83.73	99.70 %	272.03	99.79 %	663.69
	PRG+SA	99.90 %	26.03	99.75 %	95.34	99.70 %	170.62	99.74 %	433.40
SI	Single-PRG	88.32 %	0.03	84.16 %	0.15	82.68 %	0.54	80.69 %	1.50
	Multi-PRG	91.28 %	0.36	87.06 %	2.43	84.46 %	5.68	82.45 %	9.98
	PRG+TS	93.25 %	3.80	94.22 %	22.92	85.22 %	80.43	82.99 %	180.84
	PRG+GA (Single-Point-Crossover)	92.57 %	54.46	87.48 %	177.36	88.21 %	739.14	82.83 %	1094.80
	PRG+GA (Two-Point-Crossover)	92.90 %	37.41	90.22 %	130.37	85.80 %	501.69	82.99 %	820.63
	PRG+SA	95.97 %	27.85	91.77 %	91.22	86.12 %	165.42	84.23 %	412.87

Tabelle 8.16: Performance-Vergleich (Meta-)Heuristiken

Eine Datenauswertung der Ergebnisse der SI zeigt, dass die Startlösungen maßgeblich durch Einsatz der Metaheuristiken verbessert werden können. Bei der kleinsten Instanzengröße werden die besten Ergebnisse durch Einsatz der PRG+SA Metaheuristik erzielt, welche durchschnittlich bei 95 % der Optimallösung liegen. Im Vergleich zum Single-PRG können die Ergebnisse daher um rund 7.5 Prozentpunkte verbessert werden.

Bei einer Instanzengröße von $n = 400$ überragt die PRG+TS Metaheuristik mit einem geometrischen Mittelwert von rund 94 % eindeutig die Performance der übrigen (Meta-)Heuristiken. Die PRG+TS Metaheuristik erzielt nicht nur durchschnittlich die besten ZF-Werte bei dieser Instanzengröße, sondern weist auch im Vergleich zu den anderen Verbesserungsheuristiken mit ungefähr 23 Sekunden die beste Laufzeit-Performance auf. Das ist darauf zurück zu führen, dass auf Grund der Parameter-Vortests im Gegensatz zu dem PRG+GA und PRG+SA weniger Startlösungen erstellt werden. Das ist darauf zurück zu führen, dass bei den Vortests der Instanzen des Parameter-Testbeds bei einem Wert von $\bar{a} = 20$ sehr hohe Rechenzeiten benötigt werden. Die finale Auswertung aller Testinstanzen zeigt jedoch, dass unter dem Gesichtspunkt der Vergleichbarkeit der Laufzeiten der getesteten Metaheuristiken auch bei der PRG+TS mehrere Startlösungen erstellt werden könnten.

Der Einsatz PRG+GA Metaheuristik mit einem Single-Point-Crossover Verfahren erreicht bei einer

Instanzengröße von $n = 600$ durchschnittlich 88 % der Optimallösung und liefert in dieser Klasse der Instanzen demnach die besten Ergebnisse. Jedoch benötigt der PRG+GA Algorithmus hierfür ungefähr 740 Sekunden. Die PRG+SA Metaheuristik hingegen erzielt Ergebnisse, welche rund 14 % von der Optimallösung entfernt liegen, und benötigt hierfür nur ungefähr 165 Sekunden.

Bei den größten Instanzen dominiert erneut der PRG+SA Algorithmus, dessen erzielten ZF-Werte bei ungefähr 84 % der Optimallösung liegen und demnach die Startlösungen der Multi-PRG Heuristik um rund 2 Prozentpunkte verbessert werden konnten. Bei den übrigen Metaheuristiken erfolgt bei einer Inputgröße von $n = 800$ nur eine nicht nennenswerte Verbesserung der Startlösungsheuristik.

9. Fazit

Das Quadratische Rucksackproblem ist ein vielfach erforschtes Thema und weckt nicht zuletzt aufgrund seines weiten Anwendungsspektrums und der Herausforderung bei der Erstellung eines performanten Algorithmus für große Datensätze die Neugierde vieler Forscher. Das Ziel dieser Masterarbeit war es, aufgrund der vorliegenden $\mathcal{NP}$ -Schwere des Quadratischen Rucksackproblems drei geläufige metaheuristische Lösungsansätze anzuwenden und dadurch Ergebnisse von hoher Qualität in einer vertretbaren Laufzeit zu generieren. Im Hauptteil dieser Arbeit wurden die theoretischen Grundlagen der Startlösungsheuristik sowie der angewandten Metaheuristiken umfangreich behandelt und die entwickelten Algorithmen sowohl schriftlich als auch in Form von anschaulichen Pseudocodes dargestellt. Darüber hinaus wurde vorab ein Einblick in die bereits erforschten exakten sowie auch (meta-)heuristischen Lösungsansätzen des QKP geboten, um die in dieser Masterarbeit publizierten (meta-)heuristischen Methodiken dem derzeitigen Forschungsstand gegenüber zu stellen. Die erzielten Ergebnisse wurden quantitativ ausgewertet und aufgrund dessen eine Performance-Evaluierung vorgenommen.

Als Startlösungsheuristik wurde ein Paarweiser Randomisierter Greedy-Algorithmus PRG vorgestellt, welcher eine Adaption des relativen Greedy-Algorithmus von Julstrom [2005] darstellt. Dieser verfährt nach dem Prinzip eines schrittweisen Hinzufügens von Objektpaaren, welche das beste Verhältnis aus Profiten und Gewichten erzielen. Zusätzlich dazu weißt der Greedy Algorithmus eine Randomisierungskomponente auf, da nicht gezwungener das Objektpaar mit dem besten erzielten Verhältnis der Sublösung hinzugefügt wird, sondern eine Zufallsauswahl aus den k -besten Verhältnissen getroffen wird. Die durch den PRG-Algorithmus erstellten Startlösungen wurden durch Einsatz einer Tabu Suche PRG+TS, einem Genetischen Algorithmus PRG+GA und einem Simulated Annealing Verfahren PRG+SA verbessert.

Das PRG+TS Verfahren wurde im Kapitel 5.2 vorgestellt und basiert auf einer probabilistischen Tabu Suche, da die vorgenommenen Nachbarschaftsschritte durch den Tausch zweier zufällig ausgewählter inkludierter und nicht inkludierter Objekte erfolgt. Der Genetische Algorithmus PRG+GA (siehe Kapitel 6.2) zeichnet sich durch ein Single-Point-Crossover und Two-Point-Crossover Verfahren aus, durch welche in jeder Generation nb_C Nachkommen erstellt werden. Die Mutation erfolgt auf Basis einer Bit-Flip-Methode und das Auftreten von frühzeitiger Konvergenz wird durch die Steuerung der Mutations- und der Crossover-Wahrscheinlichkeit verhindert. Im Kapitel 7 wurde das Simulated Annealing Verfahren vorgestellt, bei welchem analog zum PRG+TS die Nachbarschaftsschritte auf Basis eines Tausches zweier zufällig ausgewählter Objekte erfolgt. Durch eine geometrische Ab-

kühlungsmethode wird das aktuelle Temperaturlevel iterativ reduziert bis zum Unterschreiten einer Terminierungstemperatur, welche nach der von Johnson u. a. [1989] vorgestellten Methode ermittelt wird.

Ein Blick in das Kapitel 3 zeigt, dass bereits eine Vielzahl von exakten Lösungsansätze entwickelt wurden, welche dazu in der Lage sind, Datensätze bis zu einer Inputgröße von $n = 1500$ in einer guten Rechendauer optimal zu lösen (vgl. Pisinger u. a. [2007]). Metaheuristische Methodiken, wie z. B. das GRASP+T Verfahren von Yang u. a. [2013] oder die verbesserte Version von [Alva u. Benavides, 2018] sind dazu in der Lage, für Datensätze von einer beachtlichen Größe bis zu $n = 2000$ Optimalwert-nahe Ergebnisse zu generieren. Jedoch ist es wichtig, erneut darauf hinzuweisen, dass die meisten publizierten exakten sowie auch (meta-)heuristischen Methodiken auf eine Klasse von Testinstanzen angewandt wurden, für die auch durch Einsatz einer simplen Heuristik (asymptotisch) sehr gute Ergebnisse erzielt werden können [Schauer, 2016]. Daher sei auch die Aussagekraft der publizierten Metaheuristiken infrage gestellt.

Im Zuge dieser Masterarbeit werden die entwickelten Verfahren nicht nur anhand von Instanzen getestet, welche nach der oftmals verwendeten Methode von Gallo u. a. [1980] erstellt werden, sondern auch Instanzen herangezogen, die gemäß der vorgestellten Methode von Schauer [2016] erstellt wurden. Bei dieser Methode werden sogenannten *hidden clique* Instanzen erstellt, welche hinsichtlich einer theoretischen Evidenz schwerer zu lösen sind. Darüber ist die Optimallösung dieser Instanzen mit einer hohen Wahrscheinlichkeit bekannt und es ist daher möglich, eine aussagekräftige Bewertung der erzielten Ergebnisse der Metaheuristiken zu treffen. Eine Bewertung der Performance der getesteten Metaheuristiken wurde durch die Auswertung eines um 2 geshifteten geometrischen Mittelwerts und den durchschnittlichen Laufzeiten vorgenommen. Der geometrische Mittelwert berechnet sich auf Basis des Verhältnisses eines erzielten ZF-Wertes und der Optimallösung bzw. einer Referenzheuristik. Da für die schweren Testinstanzen die Optimallösungen ermittelt werden können, wurden diese für die Berechnung des geometrischen Mittelwerts herangezogen. Für die leichten Testinstanzen wurde auf die Ergebnisse einer Adaption des Genetischen Algorithmus von Julstrom [2005], welcher nach 1400 Sekunden terminiert wird, zurückgegriffen [Schauer, 2016].

Die erzielten Ergebnisse der vorgestellten (meta-)heuristischen Verfahren wurden in Kapitel 8 präsentiert und ein besonderes Augenmerk auf die Parameteranpassung der Kontrollparameter gelegt. Bezüglich der Parameteranpassung ist es wichtig festzuhalten, dass diese nicht nur sehr zeitaufwendig ist, sondern für eine optimale Durchführung auch die Erfahrung eines Experten benötigt wird. Darüber hinaus ist die Güte von iterativen Lösungssuchen oftmals abhängig von der maximalen Anzahl von Iterationen die durchlaufen werden. Je höher dieser Wert jedoch festgesetzt wird, desto rechenintensiver ist der Algorithmus. Hierbei ist eine performante Implementierung des Algorithmus und ein leistungsstarker Computer maßgeblich bei der Festlegung dieser Iterations-Parameter. Aufgrund dessen, dass beide eben genannte Faktoren im Rahmen dieser Masterarbeit nicht gegeben waren, ist auch ein aussagekräftiger Vergleich zu den publizierten Metaheuristiken der Literatur, insbesondere

im Bezug auf die Laufzeit-Performance, nicht möglich. Jedoch konnte eine quantitative Bewertung der entwickelten Verfahren im Bezug auf eine vorliegende Optimallösung bzw. einer Referenzheuristik getroffen werden. Je nach Anwendungsfall wurden für die diversen Parameter eine online oder eine offline Parameteranpassung vorgenommen.

Die Ergebnisse der Tabelle 8.16 weisen darauf hin, dass bei den leichten Testinstanzen durch Einsatz der Multi-PRG-Heuristik bereits durchschnittlich 99 % der erzielten ZF-Werte des Genetischen Algorithmus von Julstrom [2005] erreicht werden konnten. Bei den getesteten Verfahren wurde jedoch nur durch Einsatz der Tabu Suche PRG+TS eine Verbesserung der Referenzheuristik erzielt, wie man anhand eines geometrischen Mittelwerts von über 100 % in der Tabelle 8.16 erkennen kann. Hinsichtlich der Testung der anderen Metaheuristiken im Kontext der leichten Instanzen wurde über alle Instanzengrößen konstant geometrische Mittelwerte von ungefähr 99.5 % generiert.

Für die Klasse der getesteten schweren Instanzen lässt sich erkennen, dass die Ergebnisse der Startlösungsheuristik Multi-PRG sowohl durch Einsatz des PRG+TS, des PRG+GA und des PRG+SA verbessert werden konnten. Das Simulated Annealing Verfahren PRG+SA hat bei allen Instanzengrößen, aber insbesondere bei einer Größe von $n = 200$ mit einem geometrischen Durchschnitt von rund 96 % und $n = 800$ mit rund 84 % sehr gute Ergebnisse erzielt. Bezüglich des PRG+GA Verfahren hat sich gezeigt, dass es im Vergleich zu den anderen Metaheuristiken, trotz zweier Terminierungskriterien die schlechteste Laufzeit-Performance aufweist. Ungeachtet der hohen Laufzeiten des PRG+GA und demnach im weiteren Sinne eine großen Anzahl von Iterationen der Lösungssuche, konnten im Vergleich zu den anderen Metaheuristiken nur bei einer Instanzengröße von $n = 600$ die besten Ergebnisse in einem Umfang von rund 88 % der Optimallösung erzielt werden. Wie auch bei den leichten Testinstanzen erzielt die PRG+TS Metaheuristik vergleichsweise gute Ergebnisse bei einer geringen Laufzeit. Bei einer Instanzengröße von $n = 400$ konnte z. B. durch Einsatz der PRG+TS auf die Startlösungen der Multi-PRG die Ergebnisse um rund 7 Prozentpunkte verbessert werden.

Zusammenfassend lässt sich festhalten, dass die Performance der am besten operierenden Metaheuristiken der Literatur, durch die in dieser Arbeit publizierten metaheuristischen Anwendungen nicht übertroffen werden konnten. Jedoch konnten die Ergebnisse der Referenzheuristik, im Speziellen der Genetischen Algorithmus von Julstrom [2005], durch Einsatz der angewandten Tabu Suche PRG+TS verbessert werden. Die Performance des in dieser Arbeit publizierten Genetischen Algorithmus liegt jedoch vergleichsweise nur bei 99.5 % der Ergebnisse der Referenzheuristik.

Literaturverzeichnis

- [Adams u. Forrester 2005] ADAMS, Warren ; FORRESTER, Richard: A simple Recipe for Concise Mixed 0–1 Linearizations. In: *Oper. Res. Lett.* 33 (2005), 01, S. 55–61
- [Affenzeller u. a. 2009] AFFENZELLER, M. ; WINKLER, S. ; WAGNER, S. ; BEHAM, A.: *Genetic Algorithms and Genetic Programming - Modern Concepts and Practical Applications*. Boca Raton, USA : Taylor Francis Group, LLC, 2009
- [Alva u. Benavides 2018] ALVA, L. ; BENAVIDES, A.: *An Iterated Semi-Greedy Algorithm for the 0-1 Quadratic Knapsack Problem*. EasyChair Preprint no. 437, 2018
- [Bellman 1957] BELLMAN, Richard: *Dynamic Programming*. 1. Princeton, NJ, USA : Princeton University Press, 1957
- [Billionnet u. Calmels 1996] BILLIONNET, A. ; CALMELS, F.: Linear programming for the 0-1 quadratic knapsack problem. In: *European Journal of Operational Research* 92 (1996), Nr. 2, S. 310–325
- [Billionnet u. a. 1999] BILLIONNET, A. ; FAYE, A. ; SOUTIF, E.: A new upper bound for the 0-1 quadratic knapsack problem. In: *European Journal of Operational Research* 112 (1999), Nr. 3, S. 664–672
- [Billionnet u. Soutif 2004] BILLIONNET, Alain ; SOUTIF, Eric: An exact method based on Lagrangian decomposition for the 0-1 quadratic knapsack problem. In: *European Journal of Operational Research* 157 (2004), 02, S. 565–575
- [Blickle u. Thiele 1995] BLICKLE, Tobias ; THIELE, Lothar: A Comparison of Selection Schemes Used in Genetic Algorithms / TIK Institut für Technische Informatik und Kommunikationsnetze, Computer Engineering and Networks Laboratory, ETH, Swiss Federal Institute of Technology. 2. Schweiz, Zürich, December 1995 (11). – Forschungsbericht
- [Caprara u. a. 1999] CAPRARA, A. ; PISINGER, D. ; TOTH, P.: Exact Solution of the Quadratic Knapsack Problem. In: *INFORMS Journal on Computing* 11 (1999), Nr. 2, S. 125–137
- [Chaillou u. a. 1989] CHAILLOU, Paul ; HANSEN, Pierre ; MAHIEU, Yvon: Best network flow bounds for the quadratic knapsack problem. In: *Combinatorial Optimization*. Berlin, Heidelberg : Springer Berlin Heidelberg, 1989, S. 225–235
- [Cormen u. a. 2001] CORMEN, T. ; LEISERSON, C. ; RIVEST, R. ; STEIN, C.: *Introduction to Algorithms*. 2nd. The MIT Press, 2001
- [Dantzig 1930] DANTZIG, T.: Number: the Language of Science. In: *The Economic Journal* 40 (1930), 12, Nr. 160, S. 698–699

- [Darwin 1859] DARWIN, C.: *On the Origin of Species by Means of Natural Selection*. London : Murray, 1859
- [Du u. Pardalos 1999] DU, D. (Hrsg.) ; PARDALOS, P. (Hrsg.): *Handbook of Combinatorial Optimization - Supplement Volume A*. Springer, US, 1999
- [Eilon u. Christofides 1971] EILON, Samuel ; CHRISTOFIDES, Nicos: The Loading Problem. In: *Management Science* 17 (1971), Nr. 5, S. 259–268
- [Fomeni u. Letchford 2014] FOMENI, F. D. ; LETCHFORD, A. N.: A Dynamic Programming Heuristic for the Quadratic Knapsack Problem. In: *INFORMS Journal on Computing* 26 (2014), Nr. 1, S. 173–182
- [Gallo u. a. 1980] In: GALLO, G. ; SIMEONE, B. ; HAMMER, P. L.: *Quadratic Knapsack Problems*. Bd. 12. Springer, Berlin, Germany, 1980, S. 132–149
- [Geman u. Geman 1984] GEMAN, S. ; GEMAN, D.: Stochastic Relaxation, Gibbs Distributions, and the Bayesian Restoration of Images. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* PAMI-6 (1984), Nr. 6, S. 721–741
- [Gen u. Cheng 1997] GEN, M. ; CHENG, R.: *Genetic Algorithms and Engineering Design*. 1st. New York, USA : John Wiley Sons Inc., 1997 (Engineering Design and Automation)
- [Gendreau u. Potvin 2010] In: GENDREAU, M. ; POTVIN, J.-Y.: *Tabu Search*. Boston, MA : Springer US, 2010, S. 41–59
- [Gilmore u. Gomory 1966] GILMORE, P. ; GOMORY, Ralph: The Theory and Computation of Knapsack Functions. In: *Operations Research* 14 (1966), 12, S. 1045–1074
- [Glover 1975] GLOVER, F.: Improved Linear Integer Programming Formulations of Nonlinear Integer Problems. In: *Management Science* 22 (1975), Nr. 4, S. 455–460
- [Glover 1986] GLOVER, F.: Future paths for integer programming and links to artificial intelligence. In: *Computers OR* 13 (1986), Nr. 5, S. 533–549
- [Glover u. a. 2002] GLOVER, F. ; KOCHENBERGER, G. ; ALIDAEI, B. ; AMINI, M.: Solving Quadratic Knapsack Problems by Reformulation and Tabu Search: Single Constraint Case, 2002
- [Glover u. Woolsey 1974] GLOVER, F. ; WOOLSEY, E.: Converting the 0-1 Polynomial Programming Problem to a 0-1 Linear Program. In: *Operations Research* 22 (1974), Nr. 1, S. 180–182
- [Goldberg 1989] GOLDBERG, D. E.: *Genetic Algorithms in Search, Optimization and Machine Learning*. 1st. Boston, USA : Addison-Wesley Longman Publishing Co., Inc., 1989
- [Goldberg u. Deb 1991] GOLDBERG, D. E. ; DEB, K.: A comparative analysis of selection schemes used in genetic algorithms. In: *Foundations of Genetic Algorithms*, Morgan Kaufmann, 1991, S. 69–93
- [Grainger u. Letchford 2007] GRAINGER, D. ; LETCHFORD, A.: Improving a formulation of the quadratic knapsack problem. (2007), 01
- [Grefenstette 1986] GREFENSTETTE, J. J.: Optimization of Control Parameters for Genetic Algorithms. In: *IEEE Transactions on Systems, Man, and Cybernetics* 16 (1986), Nr. 1, S. 122–128

- [Hajek 1988] HAJEK, B.: Cooling Schedules for Optimal Annealing. In: *Mathematics of Operations Research* 13 (1988), Nr. 2, S. 311–329
- [Heistermann 1994] HEISTERMANN, J.: *Genetische Algorithmen*. Stuttgart, Germany : Teubner, 1994 (Teubner-Texte zur Informatik 9)
- [Hesser u. Männer 1991] HESSER, J. ; MÄNNER, R.: Towards an optimal mutation probability for genetic algorithms. In: SCHWEFEL, Hans-Paul (Hrsg.) ; MÄNNER, Reinhard (Hrsg.): *Parallel Problem Solving from Nature*. Germany, Heidelberg : Springer Berlin Heidelberg, 1991, S. 23–32
- [Holland 1975] HOLLAND, J. H.: *Adaptation in natural and artificial systems*. The University of Michigan Press, 1975
- [Jansen 2013] JANSEN, T.: *Analyzing Evolutionary Algorithms: The Computer Science Perspective*. Springer Berlin Heidelberg, 2013 (Natural Computing Series)
- [Johnson u. a. 1989] JOHNSON, D. ; ARAGON, C. ; MCGEOCH, L. ; SCHEVON, C.: Optimization by Simulated Annealing: An Experimental Evaluation. Part I, Graph Partitioning. In: *Operations Research* 37 (1989), 12, S. 865–892
- [Johnson u. a. 1993] JOHNSON, Ellis L. ; MEHROTRA, Anuj ; NEMHAUSER, George L.: Min-Cut Clustering. In: *Math. Program.* 62 (1993), Februar, Nr. 1–3, S. 133–151
- [Julstrom 2005] JULSTROM, B. A.: Greedy, Genetic, and Greedy Genetic Algorithms for the Quadratic Knapsack Problem. In: *Proceedings of the 7th Annual Conference on Genetic and Evolutionary Computation*. New York, NY, USA : ACM, 2005 (GECCO '05), S. 607–614
- [Kellerer u. a. 2004] KELLERER, H. ; PFERSCHY, U. ; PISINGER, D.: *Knapsack Problems*. Springer, Berlin, Germany, 2004
- [Kirkpatrick u. a. 1983] KIRKPATRICK, S. ; GELATT, C. D. ; VECCHI, M. P.: Optimization by Simulated Annealing. In: *Science* 220 (1983), Nr. 4598, S. 671–680
- [Laarhoven u. Aarts 1987] LAARHOVEN, P. J. M. (Hrsg.) ; AARTS, E. H. L. (Hrsg.): *Simulated Annealing: Theory and Applications*. Norwell, USA : Kluwer Academic Publishers, 1987
- [Laoufi u. a. 2006] LAOUFI, A. ; HADJERI, S. ; HAZZAB, A.: Adaptive Probabilities of Crossover and Mutation in Genetic Algorithms for power economic dispatch. In: *International Journal of Applied Engineering Research ISSN Number* 1 (2006), 01, S. 973–4562
- [Lee u. Lee 2014] LEE, C.-Y. ; LEE, D.: Determination of initial temperature in fast simulated annealing. In: *Computational Optimization and Applications* 58 (2014), 06
- [Leung 2004] LEUNG, Joseph Y-T: *Handbook of scheduling : algorithms, models, and performance analysis*. Boca Raton, USA : Chapman Hall/CRC, 2004
- [Lorie u. Savage 1955] LORIE, James H. ; SAVAGE, Leonard J.: Three Problems in Rationing Capital. In: *The Journal of Business* 28 (1955)
- [Martello u. Toth 1990] MARTELLO, S. ; TOTH, P.: *Knapsack problems: algorithms and computer implementations*. New York, USA : John Wiley & Sons, Inc., 1990
- [Meier u. a. 2015] MEIER, A. ; VOLLMER, H. ; SCHÖNING, U.: *Komplexität von Algorithmen: Mathematik für Anwendungen Band 4*. Köln, Germany : Lehmanns Media, 2015

- [Merkle u. Hellman 1978] MERKLE, R. ; HELLMAN, M.: Hiding information and signatures in trapdoor knapsacks. In: *IEEE Transactions on Information Theory* 24 (1978), Nr. 5, S. 525–530
- [Metropolis u. a. 1953] METROPOLIS, N. ; ROSENBLUTH, A. ; ROSENBLUTH, M. ; TELLER, A. ; TELLER, E.: Equation of State Calculations by Fast Computing Machines. In: *The Journal of Chemical Physics* 21 (1953), Nr. 6, S. 1087–1092
- [Nadi u. Khader 2011] NADI, F. ; KHADER, A.: A parameter-less genetic algorithm with customized crossover and mutation operators. In: *GECCO '11*, 2011
- [Nauss 1978] NAUSS, Robert M.: The 0–1 knapsack problem with multiple choice constraints. In: *European Journal of Operational Research* 2 (1978), Nr. 2, S. 125 – 131
- [Pisinger 2007] PISINGER, D.: The Quadratic Knapsack Problem — A Survey. In: *Discrete Applied Mathematics* 155 (2007), 03, S. 623–648
- [Pisinger u. a. 2007] PISINGER, D. ; RASMUSSEN, A. B. ; SANDVIK, R.: Solution of Large Quadratic Knapsack Problems Through Aggressive Reduction. In: *INFORMS Journal on Computing* 19 (2007), Nr. 2, S. 280–290
- [Resende u. Ribeiro 2010] RESENDE, M. G. C. ; RIBEIRO, C. C.: Chapter 11 GRASP : Greedy Randomized Adaptive Search Procedures, 2010
- [Rhys 1970] RHYS, J. M. W.: A Selection Problem of Shared Fixed Costs and Network Flows. In: *Management Science* 17 (1970), November, Nr. 3, S. 200–207
- [Schauer 2016] SCHAUER, J.: Asymptotic behavior of the quadratic knapsack problem. In: *European Journal of Operational Research* 255 (2016), Nr. 2, S. 357–363
- [Scheibenpflug u. a. 2012] SCHEIBENPFLUG, A. ; WAGNER, Stefan ; PITZER, Erik ; BURLACU, B. ; AFFENZELLER, Michael: On the analysis, classification and prediction of metaheuristic algorithm behavior for combinatorial optimization problems. In: *24th European Modeling and Simulation Symposium, EMSS 2012* (2012), 01, S. 368–372
- [Shih 1979] SHIH, Wei: A Branch and Bound Method for the Multiconstraint Zero-One Knapsack Problem. In: *The Journal of the Operational Research Society* 30 (1979), Nr. 4, S. 369–378
- [Smit u. Eiben 2009] SMIT, S.K. ; EIBEN, A.: Comparing Parameter Tuning Methods for Evolutionary Algorithms, 2009, S. 399 – 406
- [Taha 2011] TAHA, H. A.: *Operations Research: An Introduction*. 4th. USA : Prentice Hall, Inc., 2011
- [Talbi 2009] TALBI, E.G.: *Metaheuristics: From Design to Implementation*. John Wiley & Sons Inc, 2009 (Wiley Series on Parallel and Distributed Computing)
- [V., Černý 1985] V., ČERNÝ: Thermodynamical approach to the traveling salesman problem: An efficient simulation algorithm. In: *Journal of Optimization Theory and Applications* 45 (1985), Jan, Nr. 1, S. 41–51
- [Vasquez u. Hao 2001] VASQUEZ, Michel ; HAO, Jin-Kao: A Hybrid Approach for the 0-1 Multidimensional Knapsack problem. In: *IJCAI International Joint Conference on Artificial Intelligence* (2001), 05

- [Witzgall 1975] WITZGALL, C.: Mathematical methods of site selection for Electronic Message Systems (EMS), 1975
- [Yang u. a. 2013] YANG, Z. ; WANG, G. ; CHU, F.: An Effective GRASP and Tabu Search for the 0-1 Quadratic Knapsack Problem. In: *Comput. Oper. Res.* 40 (2013), Mai, Nr. 5, S. 1176–1185

Abkürzungsverzeichnis

EA Evolutionärer Algorithmus

GA Genetischer Algorithmus

GRASP *Greedy Randomized Adaptive Search*

ILP Ganzzahlige lineare Programmierung

KP Rucksackproblem

LI Leichte Instanzen

LS Lokale Suche

LZ durchschnittliche Laufzeit

MKP Multidimensionales Rucksackproblem

M-KP *Multiple*-Rucksackproblem

MCKP *Multiple-Choice*-Rucksackproblem

$\mathcal{NP}$ Klasse der nicht-polynomiellen Laufzeit Algorithmen

$\mathcal{P}$ Klasse der polynomiellen Laufzeit Algorithmen

PRG Paarweiser randomisierter Greedy-Algorithmus

SA Simulated Annealing

SI Schwere Instanzen

TL Tabu Liste

TS Tabu Suche

QKP Quadratisches Rucksackproblem

QP Quadratische Programmierung

ZF Zielfunktion

Symbolverzeichnis

Symbol	Beschreibung
α	Basis der Potenzfunktion der geometrischen Abkühlungsmethode
δ	Differenz der ZF-Werte von aktueller Lösung und Nachbarschaftslösung
η	Steigung der Geradengleichung der linearen Abkühlungsmethode
λ	Dividend der Funktion der logarithmischen Abkühlungsmethode
$\omega_{i,s}$	Lösungsvektorwert an der Stelle i der Startlösung s
a	Anzahl der zu erstellenden Startlösungen
$\bar{a}$	Obere Schranke der zu erstellenden Startlösungen
all_{unique}	Boolesche Variable die auf Einzigartigkeit aller gefundenen Lösungen prüft
b	Wurzelexponent der Funktion des Parameters a in Abhängigkeit von $iterC$
C	Kapazität des Rucksacks
$change_t$	Boolesche Variable die überprüft ob Temperaturveränderung statt gefunden hat
$C_{i,d}$	Allel des d -ten Nachkommens an der Stelle i
cnt_{acc}	Zählvariable von Akzeptanzperioden
cnt_{improv}	Zählvariable der Iterationen ohne ZF-Wert Verbesserung
cnt_{temp}	Zählvariable von LS ohne Temperaturveränderung
$common_s$	Zählvariable der gefundenen Lösungen die nicht einzigartig sind
cp	Crossover-Punkt der Single-Point-Crossover-Methode
cp_1	Erster Crossover-Punkt der Two-Point-Crossover-Methode
cp_2	Zweiter Crossover-Punkt der Two-Point-Crossover-Methode
d_i	Greedy-Funktion des absoluten Greedy-Verfahrens für das Objekt i
E	Anzahl getestete Instanzen gleicher Inputgröße
$Equal_p$	Variable zur Bestimmung des Ausmaßes an Konvergenz einer Population
f	Fitnessfunktion eines Individuums einer Population
g_{ij}	Greedy-Funktionswert der Objektpaarkombination von i und j
g_{best}	k -größten Werte der Greedy-Matrix absteigend sortiert
$improv$	Boolesche Variable die auf Verbesserung durch Nachbarschaftssuche überprüft
ind_{best}	Indizes der zu den g_{best} -Werten zugeordneten Objektpaaren
ind_{BR}	Zu entfernendes Objekt des vorteilhaftesten Reparaturschritts
L	Anzahl der randomisierten Läufe einer Metaheuristik
$iterC$	Obere Schranke von maximalen Iterationen der PRG Heuristik einer Instanz
$iterGA$	Anzahl von Iterationen des PRG+GA Algorithmus
$iterTS$	Anzahl von Iterationen des PRG+TS Algorithmus
k	Zufallsauswahl aus den k -besten Greedy-Funktionswerten
m	Anzahl von Rucksäcken
n	Anzahl der verfügbaren Objekte
nb_{acc}	Anzahl von Akzeptanzperioden bei der eine Temperaturveränderung vorgenommen wird

nb_C	Anzahl der zu erstellenden Nachkommen
N_i	Set aus verschiedenen Varianten des Objektes i
N_{size}	Anzahl von zu untersuchenden exkludierten Objekten
nb_{trans}	Anzahl von maximalen Nachbarschaftsschritten in einer LS
p_N	Prozentsatz zur Bestimmung der Nachbarschaftsgröße
P_0	Startpopulation
P_t	t -te Population
$PA1_i$	Allel des ersten Elternteils an der Stelle i
$PA2_i$	Allel des zweiten Elternteils an der Stelle i
p_{acc}	Akzeptanzwahrscheinlichkeit
p_{cr}	Crossover-Wahrscheinlichkeit
p_C	Prozentsatz zur Bestimmung von Anzahl der Nachkommen pro Population
p_i	Profit des Objekts i
p_m	Mutations-Wahrscheinlichkeit
q_{ij}	Paarweiser Profit der Objektpaarkombination i und j
R	Gleichverteilte Zufallszahl
$rand_{in}$	Index des exkludierten Objekts
$rand_{ex}$	Index des inkludierten Objekts
r_i	Greedy-Funktion des relativen Greedy-Verfahrens für das Objekt i
S	Sublösung
t	Anzahl von Perioden in denen Objekt auf TL verweilt
T_0	Starttemperatur
T_{BOUND}	Terminierungstemperatur
T_t	Temperatur der aktuellen Iteration t
$unique_s$	Boolesche Variable die auf Einzigartigkeit einer Lösung s prüft
w_i	Gewicht des Objekts i
x_i	Lösungsvektorwert des Objekt i
z	Zielfunktionswert
z_{BS}	Zielfunktionswert des vorteilhaftesten Nachbarschaftsschrittes
z_R	Zielfunktionswert nach potentielltem Reparaturschritt
z_{BR}	Zielfunktionswert des vorteilhaftesten Reparaturschritts
$z_{l,e}$	Zielfunktionswert der Instanz e beim randomisierten Lauf l
z_{re}	Referenz-Zielfunktionswert der Instanz e